



**POLITECNICO**  
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE  
E DELL'INFORMAZIONE

# Marionette: Runtime Behavioural Configuration and A/B/n Testing for Microservices Systems

TESI DI LAUREA MAGISTRALE IN  
COMPUTER SCIENCE AND ENGINEERING - INGEGNERIA IN-  
FORMATICA

Author: **Matteo Sissa**

Student ID: 247064  
Advisor: Prof. Matteo Camilli  
Academic Year: 2024-25



# Abstract

Modern microservice architectures excel at structural adaptability through independent scaling and deployment, but lack mechanisms for runtime behavioural modification. When business logic or algorithms require changes, developers typically resort to redeployment cycles, complex configuration-driven branching, or maintaining multiple service variants. All approaches that introduce significant complexity and operational overhead.

This thesis presents Marionette, a framework that enables runtime behavioural configuration and automated A/B/n testing in microservice architectures. The developed system allows developers to define multiple behavioural implementations for methods within their microservices. The tool automatically transforms the codebase to support runtime behaviour switching. An additional microservice part of the Marionette ecosystem, called the Marionettist service, automatically discovers instrumented microservices, extracts their behavioural configurations, and provides a web-based interface for centralised management and real-time metric monitoring.

To demonstrate the framework's capabilities, we developed an automated A/B/n testing pipeline that systematically explores all possible combinations of behavioural variants across multiple microservices. Unlike traditional A/B testing, which compares two system versions, this approach enables multidimensional experimentation where individual method implementations are tested in combination, revealing system-wide interactions and trade-offs that would remain hidden in isolated testing. The framework was validated through comprehensive experiments using a custom microservice application. The evaluation demonstrated successful runtime behaviour modification across distributed services, with the A/B/n testing pipeline effectively identifying optimal configurations.

The Marionette framework establishes a foundation for advanced behavioural management in microservices, enabling applications ranging from automated system optimisation to energy-aware computing.

**Keywords:** microservices, runtime behaviour modification, A/B testing, distributed systems, software architecture, behavioural variants, automated testing





## Abstract in Lingua Italiana

Le architetture moderne a microservizi eccellono nell'adattabilità strutturale attraverso il dimensionamento e deployment indipendenti, ma mancano di meccanismi per la modifica comportamentale a runtime. Quando la logica di business richiede cambiamenti, gli sviluppatori ricorrono a operazioni di branching complesse o mantenimento di varianti multiple del servizio, tutti approcci che introducono complessità e overhead operativo.

Questa tesi presenta Marionette, un framework che abilita la configurazione comportamentale a runtime e il testing A/B/n automatizzato in architetture a microservizi. Il sistema consente agli sviluppatori di definire implementazioni comportamentali multiple per i metodi nei loro microservizi. Il tool trasforma automaticamente il codice sorgente per supportare il cambio di comportamento a runtime. Il servizio Marionettist scopre automaticamente i microservizi instrumentati, estrae le loro configurazioni comportamentali e fornisce un'interfaccia web per gestione centralizzata e monitoraggio delle metriche in tempo reale.

Per dimostrare le capacità del framework, abbiamo sviluppato una pipeline di testing A/B/n che esplora sistematicamente tutte le combinazioni di varianti comportamentali in un'applicazione a microservizi. Diversamente dal testing A/B tradizionale che confronta due versioni del sistema, questo approccio abilita sperimentazione multidimensionale dove implementazioni di metodi individuali sono testate in combinazione, rivelando interazioni e trade-off a livello di sistema che rimarrebbero nascosti nel testing isolato. Il framework è stato validato attraverso esperimenti utilizzando un'applicazione a microservizi personalizzata. La valutazione ha dimostrato la fattibilità della modifica comportamentale a runtime, con la pipeline di testing che identifica efficacemente le configurazioni ottimali.

Il framework Marionette stabilisce una fondazione per la gestione comportamentale avanzata nei microservizi, abilitando applicazioni dall'ottimizzazione automatica al computing energy-aware.

**Parole chiave:** microservizi, modifica comportamentale a runtime, testing A/B, sistemi distribuiti, architettura software, varianti comportamentali, testing automatizzato



# Contents

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| <b>Abstract</b>                                                                           | <b>i</b>   |
| <b>Abstract in Lingua Italiana</b>                                                        | <b>iii</b> |
| <b>Contents</b>                                                                           | <b>v</b>   |
| <b>Introduction</b>                                                                       | <b>1</b>   |
| <b>Related Work</b>                                                                       | <b>5</b>   |
| <br>                                                                                      |            |
| <b>1 System Overview</b>                                                                  | <b>15</b>  |
| 1.1 The Marionette Code Transformation Tool . . . . .                                     | 15         |
| 1.1.1 Problem Definition . . . . .                                                        | 15         |
| 1.1.2 Marionette Tool to the Rescue . . . . .                                             | 17         |
| 1.2 Central Behaviour Management with the Marionettist Service . . . . .                  | 20         |
| 1.2.1 Control Plane Architecture Pattern . . . . .                                        | 20         |
| 1.2.2 Role and Responsibilities . . . . .                                                 | 20         |
| 1.3 Application to A/B/n Testing Context . . . . .                                        | 21         |
| 1.3.1 Conceptual Foundation . . . . .                                                     | 21         |
| 1.3.2 Multi-Dimensional Experimentation . . . . .                                         | 22         |
| 1.3.3 High-level Experimental System View . . . . .                                       | 22         |
| <br>                                                                                      |            |
| <b>2 Marionette: A Development Tool for Microservices with Runtime Variable Behaviour</b> | <b>25</b>  |
| 2.1 A New Microservice Pattern . . . . .                                                  | 26         |
| 2.1.1 The Marionette Classes . . . . .                                                    | 26         |
| 2.1.2 The Behaviour Registry . . . . .                                                    | 27         |
| 2.1.3 The Behaviour Controller . . . . .                                                  | 27         |
| 2.1.4 The Generality of the Pattern . . . . .                                             | 27         |
| 2.2 Source Code Transformation Pipeline - An Abstract View . . . . .                      | 28         |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 2.2.1    | Definition of Multiple Behaviours . . . . .                          | 28        |
| 2.2.2    | Copy of Original Project . . . . .                                   | 29        |
| 2.2.3    | Marionette Class Creation . . . . .                                  | 29        |
| 2.2.4    | Source Code Injection . . . . .                                      | 33        |
| 2.3      | Concrete Usage of the Marionette Tool in Java . . . . .              | 34        |
| 2.3.1    | The Marionette XML Configuration File . . . . .                      | 35        |
| 2.3.2    | Transformation Process in a Java Environment . . . . .               | 37        |
| 2.3.3    | Sequence Diagrams . . . . .                                          | 41        |
| <b>3</b> | <b>The Marionettist Service for Centralised Behaviour Management</b> | <b>45</b> |
| 3.1      | Service Design . . . . .                                             | 46        |
| 3.2      | Service Implementation . . . . .                                     | 48        |
| 3.2.1    | A Clean Approach . . . . .                                           | 48        |
| 3.2.2    | Component Diagram . . . . .                                          | 50        |
| 3.2.3    | Interactions with External Components . . . . .                      | 51        |
| 3.2.4    | Metrics Configurability . . . . .                                    | 53        |
| 3.2.5    | The Graphical Interface and Available Functions . . . . .            | 54        |
| <b>4</b> | <b>The Automated A/B/n Testing Pipeline</b>                          | <b>57</b> |
| 4.1      | The Steps Making up the Pipeline . . . . .                           | 57        |
| 4.1.1    | Generation of System-level Configurations . . . . .                  | 58        |
| 4.1.2    | The Test Loop . . . . .                                              | 59        |
| 4.1.3    | The Uniform Traffic Assumption . . . . .                             | 59        |
| 4.1.4    | Final Ranking Generation . . . . .                                   | 61        |
| 4.2      | Concrete Implementation Details . . . . .                            | 62        |
| 4.2.1    | Metrics Configuration . . . . .                                      | 62        |
| 4.2.2    | Initiating the Test . . . . .                                        | 63        |
| 4.2.3    | High-level View of Components and Interactions During Test . . . .   | 64        |
| 4.2.4    | Retrieving the Results . . . . .                                     | 64        |
| 4.3      | How to Analyse the Results . . . . .                                 | 65        |
| <b>5</b> | <b>Experimental Results</b>                                          | <b>69</b> |
| 5.1      | The Test Subject . . . . .                                           | 69        |
| 5.2      | Single Variation Point Experiment . . . . .                          | 71        |
| 5.2.1    | Instrumentation of the Test Subject . . . . .                        | 71        |
| 5.2.2    | Metrics Configuration . . . . .                                      | 71        |
| 5.2.3    | Local Deployment Setup . . . . .                                     | 72        |
| 5.2.4    | Generation of Synthetic User Traffic . . . . .                       | 72        |

|          |                                                    |            |
|----------|----------------------------------------------------|------------|
| 5.2.5    | Execution and Analysis of Results . . . . .        | 73         |
| 5.2.6    | Summary of RQ1 (Single Variation Point) . . . . .  | 76         |
| 5.3      | Multi-Variation Point Experiment . . . . .         | 77         |
| 5.3.1    | Instrumentation of Test Subject . . . . .          | 77         |
| 5.3.2    | Metrics Configuration . . . . .                    | 78         |
| 5.3.3    | Local Deployment Setup . . . . .                   | 79         |
| 5.3.4    | Generation of Synthetic User Traffic . . . . .     | 79         |
| 5.3.5    | Analysis of Results . . . . .                      | 79         |
| 5.3.6    | Summary of RQ1 (Multi-Variation Point) . . . . .   | 84         |
| 5.4      | Evaluation of Instrumentation Overhead . . . . .   | 85         |
| 5.4.1    | Experiment Setup and Execution . . . . .           | 85         |
| 5.4.2    | Results Analysis . . . . .                         | 87         |
| 5.4.3    | Summary of RQ2 (Multi-Variation Point) . . . . .   | 90         |
| 5.5      | Analysis of Development Efficiency Gains . . . . . | 91         |
| 5.5.1    | Framework Architecture Benefits . . . . .          | 91         |
| 5.5.2    | Code Complexity Analysis . . . . .                 | 91         |
| 5.5.3    | Developer Productivity Impact . . . . .            | 92         |
| 5.5.4    | Summary for RQ3 (Multi-Variation Point) . . . . .  | 93         |
| <b>6</b> | <b>Conclusions and Future Work</b>                 | <b>95</b>  |
| 6.1      | Main Conclusions . . . . .                         | 95         |
| 6.2      | Future Work . . . . .                              | 96         |
| 6.3      | Final Remarks . . . . .                            | 97         |
|          | <b>Bibliography</b>                                | <b>99</b>  |
|          | <b>List of Figures</b>                             | <b>103</b> |
|          | <b>List of Tables</b>                              | <b>107</b> |
|          | <b>List of Symbols</b>                             | <b>109</b> |
|          | <b>Acknowledgements</b>                            | <b>111</b> |



# Introduction

Modern software systems face incrementally complex challenges. One of the most relevant ones is that they operate in increasingly dynamic environments where the ability to **adapt behaviour** without disrupting service availability has become critical. This is particularly relevant in large-scale and variable environments, where **microservice-based architectures** are largely employed. This architectural paradigm encourages the practice of breaking down the functionality of an application into small, separate services that collaborate and communicate over the network to carry out the tasks requested to the application. Microservices reveal great advantages when it comes to adapting the system to external world conditions. Indeed, this architectural approach excels at **structural adaptability**, enabling independent development, elastic scaling, fault isolation, and service composition. This adaptability comes naturally from the separation of concerns introduced by this architectural style. If each major functionality of the system is isolated in a separate service, then it is possible to develop each part separately from the others, as well as scale it in production or heal it in case of failures. Arguably, this is the main reason why current microservice frameworks primarily address infrastructure-level concerns while leaving a critical gap: **handling runtime behavioural adaptability**. This means that while these frameworks take care of scaling and healing the system by adding or restarting services, the internal behaviour of these services remains mostly static and fixed. This gap in behavioural adaptability research has been noted in the self-adaptive systems literature. [11] distinguish between structural adaptation (changing compositional topology by adding/removing components) and behavioural adaptation (replacing components and adjusting their parameters), noting that “a significant amount of effort is invested in formal approaches to structural adaptations” while “behavioural adaptations, in contrast, have received little attention.” This pattern is confirmed by [26]’s systematic literature review, which found that research predominantly focuses on infrastructure-level concerns (IoT, IaaS, Web Services) with 87% of empirical studies concentrating on parameter adaptation rather than more fundamental behavioural changes.

When the logic of a service needs to change, such as its rules, algorithms, or processing strategies, developers usually rely on one of three approaches, none of which are entirely

satisfactory. The most common solution is to **recompile and redeploy** the code, but this often interrupts the service and slows down the feedback cycle, since changes take a long time to become visible. Another option is to embed **configuration-based branching** inside the service logic. In this approach, the code is written with multiple conditional branches, each representing a possible variant of the behaviour, and external configuration files determine which branch is executed at runtime. While this makes it possible to alter the service's behaviour without redeploying, it comes at a high cost for developers: the codebase gradually fills with conditional paths, interactions between variants become harder to track, and the overall structure becomes difficult to evolve. A third approach is to **keep multiple service variants** active in the cluster at the same time, but this shifts the problem to the operational layer, where the overhead of managing and coordinating different versions can become significant.

Therefore, this thesis addresses three fundamental questions:

- Is it possible to achieve **automated runtime behavioural adaptability at the method/function level** in microservice architectures, while maintaining code maintainability, extensibility, and system performance?
- What architecture is required to enable **management of behavioural variants** across distributed microservices?
- How can we apply the results obtained to concrete use cases, in particular to a systematic and **automated A/B/n testing framework** that is able to compare behavioural combinations and reveal system-wide performance trade-offs?

Existing approaches to runtime behavioural adaptation in software systems fall into three main categories, each with significant limitations. **Infrastructure-level solutions** such as self-adaptive systems and deployment strategies (blue-green deployments, canary releases) excel at managing scalability, fault tolerance, and resource optimisation, but operate at the service level, requiring complete service duplication for each behavioural variant [7, 22]. **Application-level solutions** like feature flags and configuration management enable runtime behaviour changes without redeployment, but force developers to embed complex conditional logic throughout their codebase, creating technical debt and maintenance overhead [8, 21]. **Experimental frameworks** for A/B testing provide systematic approaches to compare behavioural alternatives, but typically operate at the deployment level, requiring separate system versions rather than enabling fine-grained experimentation within running services [14, 20].

These approaches create a fundamental **trade-off**: developers must choose between run-



time flexibility (through complex conditional logic) and code clarity (through service-level switching), while operators lack unified tools for managing behavioural variants across distributed systems. Current solutions address either structural adaptation (adding/removing services) or coarse-grained behavioural changes, leaving a critical gap in **fine-grained, method-level behavioural control** that preserves **code maintainability**.

To address the questions which this thesis focuses on, we propose a novel approach that separates behavioural variant definition from infrastructure management through **automated code transformation** and **centralised control**. Rather than requiring developers to implement switching logic or manage complex configuration systems manually, our approach automatically generates the necessary infrastructure while maintaining the original code clean. The resulting framework, called Marionette, enables developers to **define multiple behavioural variants** within a microservice codebase without worrying about the entangled conditional logic to select among the different implementations. This is automatically taken care of by the Marionette infrastructure. In this way, the user of this tool can solely focus on the application business logic and be relieved of **control infrastructure concerns**. Furthermore, the framework **simplifies operational management** of the different behavioural versions by providing an additional service, called Marionettist, that automatically discovers microservices with behavioural variants and centralises their management through an intuitive user interface.

This infrastructure provides, on its own, a foundation for diverse behavioural control applications, from manual configuration management to **automated optimisation systems**. To validate these capabilities and demonstrate the framework’s effectiveness, we focus on developing an **automated A/B/n testing** application that leverages the Marionette infrastructure to compare behavioural variants at a system-level.

**A/B/n testing** [15] extends traditional A/B testing [14] to compare multiple variants simultaneously rather than just two alternatives. This experimental methodology evaluates multiple system configurations using quantitative metrics. The goal is to identify which configuration performs best in terms of specific objectives. While A/B testing provides valuable insights for binary choices, it becomes limited when dealing with complex distributed systems where multiple components can each have several behavioural variants. Moreover, the challenge with traditional A/B testing in microservice environments is scope. Standard approaches typically evaluate two versions of a single service in isolation. However, in distributed systems, the optimal configuration often emerges from the **interaction between different services** rather than from optimising individual components separately.

Our approach enables **multi-dimensional experimentation** that overcomes these limitations. In the framework developed as part of this project, each instrumented microservice can contain behavioural variants for different methods, creating a space of possible system-level configurations that grows with the number of services and available variants. The A/B/n testing framework is able to **explore all possible combinations** of these behavioural variants to evaluate comprehensive system configurations. This exhaustive approach reveals performance interactions between services that would remain hidden when testing individual components in isolation. The framework automatically manages the complexity of coordinating tests across distributed services, switching between configurations at predetermined intervals while collecting performance metrics for each combination.

After the testing cycle, the system also produces **comparative rankings** that identify which behavioural combinations achieve optimal performance according to specified criteria. This methodology enables the discovery of **system-wide trade-offs** and **optimal configurations** that emerge from the interaction of multiple services rather than individual component optimisation.

This thesis is organised into six chapters that systematically present the development, implementation, and evaluation of the Marionette framework.

**Chapter 1** provides a comprehensive system overview, introducing the three core components of the Marionette ecosystem: the code transformation tool that enables runtime behavioural variability in microservices, the Marionettist service for behaviour management, and the application of this infrastructure to automated A/B/n testing scenarios. **Chapter 2** presents the technical details of the Marionette development tool, including the design of the microservice pattern for runtime variable behaviour and the complete source code transformation pipeline. **Chapter 3** describes the Marionettist service architecture and implementation, covering the architectural design principles, the integration of the service into an already existing cluster, and the web-based graphical interface for behaviour management. **Chapter 4** details the automated A/B/n testing pipeline, explaining the systematic generation of system-level configurations and the iterative testing loop methodology. **Chapter 5** presents comprehensive experimental results, including validation experiments on a custom microservice-based application, quantitative analysis of instrumentation overhead, and evaluation of development efficiency gains provided by the framework. **Chapter 6** concludes the work by summarising the main findings and outlining promising directions for future research.

## Related Work

This project started by surveying the state of the art concerning dynamic behaviour control for microservice-based applications. The results obtained showed that most of the literature in the context of adaptable microservice architectures focuses on different qualities of the software, with particular tendencies towards scalability, fault tolerance, self-healing, and deployment optimisation [2, 26].

Indeed, the management of dynamic behaviour in microservice-based applications represents an emerging area of research, and there are very few papers that can be considered closely related to the topic of this thesis. Nevertheless, several adjacent research areas provide foundational concepts and partially address related challenges. This section examines the current state of the art across these domains, identifies their limitations in the context of dynamic behaviour management, and positions this project's contribution within the broader research landscape.

## Self-Adaptive Systems and Autonomic Computing

**Self-adaptive systems** are designed to automatically adjust their behaviour in response to changes in their environment or internal state. The goal is to maintain or improve system performance, reliability, and other quality attributes without human intervention. This idea gained popularity with the concept of autonomic computing, introduced by Kephart and Chess in 2003 [12], which envisioned systems that manage themselves autonomously, much like the human autonomic nervous system.

A key model for building self-adaptive systems is the **MAPE-K loop** [12]. It includes four main steps:

- **Monitor:** Observe the system and environment to gather relevant data.
- **Analyse:** Interpret the data to detect deviations from goals or identify opportunities for improvement.
- **Plan:** Decide on actions to adjust the system's behaviour.

- **Execute:** Implement the planned adaptations.

All these steps use a shared **Knowledge** base that holds necessary information about the system and environment. The loop continuously repeats to keep the system aligned with its objectives, as systematically described by Arcaini, Riccobene and Scandurra [3].

This control loop architecture helps structure self-adaptive software and provides a foundation for many modern adaptation solutions. Oh, Raibulet, and Leest [19] have investigated its application in various domains, including cloud computing, the Internet of Things, and cyber-physical systems.

Most research in autonomic computing and self-adaptive systems has focused on enhancing specific quality attributes of software. The most widely explored are scalability, self-healing, resource optimisation, and reliability [5].

Florio and Di Nitto [7] propose *Gru*, a decentralised framework that introduces autonomic capabilities into microservices without modifying their implementation. Gru leverages Docker containers and employs distributed agents, each running a MAPE loop, to make local adaptation decisions such as **replication**, **scaling** and **migration** of services. This decentralised design avoids bottlenecks and single points of failure, making it suitable for large-scale systems.

Zang et al. [27] present a **fault diagnosis** method for microservices based on a **self-adaptive heartbeat detection algorithm**. Their approach, named *M-MFSA-HDA*, combines factors such as CPU utilisation, network load, and task size to tune the frequency of heartbeat messages adaptively. Compared to traditional push- or pull-based methods, their algorithm improves both the accuracy and efficiency of fault detection in complex microservice systems.

Rossi et al. [22] introduce *me-kube*, a hierarchical extension to Kubernetes for **elastic scaling** of microservices. Unlike the default threshold-based autoscaler, which relies solely on decentralised local metrics, me-kube introduces a two-level architecture. A centralised Application Manager coordinates local Microservice Managers, applying proactive and reactive control policies based on queuing theory. This hierarchical approach improves stability and performance for latency-sensitive applications while maintaining scalability.

Fu et al. [9] propose **Nautilus**, an adaptive system for efficient microservice deployment across cloud and edge environments. It optimises **resource usage** while guaranteeing quality of service through intelligent mapping, dynamic resource management, and context-aware migration strategies.

While these approaches effectively address infrastructure-level adaptation concerns such

as scaling, fault tolerance, and resource optimisation, they operate primarily at the level of **resource management** and **service orchestration**. However, they leave unaddressed the critical challenge of **runtime behavioural adaptability**, which is the ability to dynamically alter the internal logic, algorithms, or decision-making processes of a service without redeployment. This creates a significant dichotomy: modern systems are becoming agile at managing scaling and self-healing, yet remain fundamentally **rigid** in their internal behaviour. The existing work on self-adaptive systems, therefore, does not offer many instruments for tuning software behaviour in response to changing requirements or for modifications in the surrounding environment.

## Dynamic Behaviour Management in Software Systems

The most direct approaches to runtime behavioural control in software systems involve **configuration management** and **feature toggling** mechanisms. These approaches aim to modify system behaviour without requiring code redeployment, though they vary significantly in their granularity, complexity, and maintainability implications.

### Feature Flags and Runtime Configuration

**Feature flags**, also known as **feature toggles**, represent one of the most widely adopted approaches for runtime behaviour control in modern software systems. This approach enables developers to modify application behaviour without redeploying code by embedding conditional logic that responds to external configuration signals. Fowler [8] provides a comprehensive taxonomy of feature toggles, distinguishing between release toggles (for gradual feature rollout), experiment toggles (for A/B testing), ops toggles (for operational control), and permission toggles (for user-specific features). While this work establishes important patterns for runtime feature management, it primarily addresses **binary on/off switches** rather than supporting multiple sophisticated behavioural variants within a single feature.

Enterprise applications often go beyond basic feature flags by using more sophisticated **configuration-driven behaviour control mechanisms**. These systems use external configuration files, databases, or management interfaces to drive runtime decisions. Spring Boot's configuration management <sup>1</sup>, for example, allows runtime property modification through actuator endpoints, while cloud-native platforms like Spring Cloud Config <sup>2</sup> enable centralised configuration management across distributed systems. However, these

---

<sup>1</sup>Spring Boot Framework, <https://spring.io/projects/spring-boot>.

<sup>2</sup>Spring Cloud Config Server and Client, <https://spring.io/projects/spring-cloud-config>.

approaches still require developers to anticipate all possible behavioural paths and embed corresponding conditional logic throughout the codebase.

Recent empirical studies have highlighted critical limitations in traditional feature flagging approaches. Rahman et al. [21] found through their empirical study that while feature toggles provide deployment flexibility, they "also introduce **technical debt** and **additional maintenance** for developers". Their analysis revealed that feature flags often remain in codebases longer than intended, creating permanent conditional complexity. These findings highlight a fundamental limitation: traditional feature flagging approaches require **explicit conditional logic** scattered throughout the codebase, making maintenance challenging and limiting the extension of behavioural variations.

## Deployment Strategies for Behaviour Switching

When runtime configuration proves insufficient for behavioural changes, organisations often resort to **deployment-level strategies** that manage behavioural variants through infrastructure and orchestration mechanisms. These approaches treat different behaviours as separate deployments, leveraging container orchestration and traffic management to switch between behavioural alternatives.

**Blue-green deployments** and **canary releases** represent the most established patterns in this category. Humble and Farley [10] describe blue-green deployment as a technique for reducing downtime and risk by running **two identical production environments**. This approach provides clean separation between versions and enables rapid rollback capabilities. Similarly, **canary deployment strategies**, as demonstrated by Netflix's engineering team [23], enable gradual rollout of new versions to subsets of users, providing risk mitigation through incremental exposure.

Recent research has moved beyond basic deployment strategies to consider broader system goals. Dell'Oro et al. [6] created a system that automatically chooses and deploys different versions of microservices based on **environmental rules**. Their approach keeps multiple versions of each service ready, with each version optimised for various needs such as energy use or performance. A central controller watches the system and deploys the right service versions to **meet sustainability goals**. This work shows how deployment systems can do more than manage versions. The main idea is that you can change system behaviour by choosing which service versions to deploy based on current needs. For example, when energy costs are high, the system uses energy-efficient service versions. When performance is critical, it switches to faster versions.

However, these deployment-based approaches share fundamental limitations that restrict

their applicability to fine-grained behavioural control. First, they operate at **service granularity**, requiring complete service duplication for each behavioural variant. In systems with multiple behavioural dimensions per service, this leads to exponential growth in operational complexity. Second, they **lack dynamic reconfiguration** capabilities. This is because switching between behaviours requires deployment operations rather than runtime configuration changes.

## Circuit Breakers and Fault Tolerance Patterns

The **circuit breaker pattern**, popularised by Netflix’s Hystrix library [17], introduces runtime behavioural switching in response to **failure conditions**. Nygard [18] describes circuit breakers as a mechanism for preventing cascading failures by temporarily **disabling calls to failing services**. When a service experiences high failure rates, the circuit breaker automatically switches to a fallback behaviour, such as returning cached data or a default response. Circuit breakers represent an essential class of runtime adaptation mechanisms that can modify system behaviour without code changes or redeployment. However, they are inherently reactive and focused solely on **fault tolerance scenarios**. They respond to system failures rather than enabling proactive behavioural control based on operational requirements or optimisation goals. Additionally, circuit breakers typically operate with **binary logic**, switching between ‘normal’ and ‘fallback’ behaviours rather than supporting multiple sophisticated behavioural variants. Furthermore, circuit breakers operate at the service interaction level rather than within individual service implementations. They can change how services communicate with each other, but they cannot modify the internal algorithms, processing strategies, or business logic within a service. This limits their applicability to scenarios where the behavioural variation needs to occur within the service’s core functionality.

## Runtime Service Composition

Research on runtime service composition has focused on **dynamically connecting and reconfiguring distributed services** to adapt to changing conditions. This field addresses how to automatically modify which services work together and how they interact, rather than changing the internal behaviour of individual services.

Alf  rez and Pelechano [1] developed frameworks for autonomic Web service compositions using models at runtime. Their approach enables services to **automatically adjust their configurations** when problems arise in complex systems. Silva et al. [24] present the DynamiCoS framework, which focuses on helping users **find and select services**



**dynamically** while the system is running. It utilises semantic information (meaning-based descriptions) and user goals to identify which services best meet the current needs automatically, and then connects those services together.

These approaches typically work at the **workflow level**, focusing on orchestrating different services to achieve specific tasks. The adaptation occurs by changing which services are used in a workflow or by modifying how services are connected together. However, these traditional service composition approaches differ fundamentally from behavioural variant management within individual services. Existing research focuses on service-level composition. In contrast, our approach implements **method-level behavioural composition**, which allows us to adapt the internal logic within individual services while **keeping their external interfaces stable**. This represents a shift from coarse-grained service orchestration to fine-grained behavioural control. Service composition approaches can alter the interactions between services, but they cannot alter how individual services perform their internal tasks.

## A/B Testing

**A/B testing**, also known as controlled experimentation or split testing, is a **statistical methodology** for comparing two or more variants of a system component to determine which performs better according to **predefined metrics** [14]. Initially developed in agricultural and medical research as randomised controlled trials, A/B testing has become a widely used methodology in software engineering for making data-driven decisions about system improvements and algorithmic choices.

The fundamental principle of A/B testing lies in randomly assigning system instances to different algorithmic implementations and comparing their performance under **identical conditions**. Researchers can, in this way, establish causal relationships between algorithmic choices and selected system performance metrics [13]. This methodology enables the evidence-based selection of algorithmic implementations, rather than relying solely on theoretical analysis.

Kohavi et al. have established the **theoretical and practical foundations** of trustworthy online controlled experiments [14]. Their comprehensive framework addresses the entire lifecycle of A/B testing, from experimental design and statistical considerations to implementation challenges and best practices. They emphasise the importance of understanding **threats to validity**, including selection bias, instrumentation effects, and external validity concerns that can compromise experimental conclusions.



From a practical implementation perspective, A/B testing has proven valuable for validating hypotheses about system improvements before full deployment [25]. The methodology provides a systematic approach to innovation that **reduces the risk of deploying** changes that may degrade system performance.

**A/B/n testing** extends the classical A/B testing approach by allowing comparison of multiple variants (n variants) simultaneously rather than just two. This extension maintains the same statistical rigour and experimental principles while enabling **more comprehensive evaluation** of different algorithmic approaches in a single experiment.

Traditional A/B testing in software systems typically operates at the **deployment level**. When testing a new algorithm or system modification, organisations deploy different versions of the entire application to other user groups or server instances. This approach requires creating separate builds, managing multiple deployments, and coordinating infrastructure changes.

Our approach introduces **fine-grained A/B/n testing** that operates at the **method level** rather than the deployment level. Instead of deploying different versions of the entire system, we dynamically **switch between different implementations of individual methods during runtime**. This means a single running system can execute different algorithmic variants for the same functionality without requiring separate deployments.

The key difference lies in **granularity**:

- **Traditional A/B testing**: Compares entire system deployments (coarse-grained)
- **Fine-grained A/B testing**: Compares individual method implementations within the same running system (fine-grained)

This fine-grained approach offers several advantages: it eliminates the **overhead** of maintaining **multiple deployments** and allows for **rapid experimentation** with different implementations without infrastructure changes.

## Automated A/B Testing Pipelines

While traditional A/B testing requires manual setup and coordination of experiments, recent research has explored how self-adaptive systems principles can enhance the **automation of continuous experimentation pipelines**.

Quin and Weyns [20] present SEABYTE, a self-adaptive microservice system artefact that automates A/B testing using a **MAPE-K feedback loop**. Their framework automatically executes experimental pipelines by monitoring experiment results, analysing

statistical outcomes, planning transitions to subsequent experiments based on predefined rules, and executing the next experiment or concluding the pipeline. SEABYTE demonstrates how self-adaptation can **eliminate the manual coordination** overhead typically required in A/B testing by automatically transitioning between related experiments based on statistical outcomes.

However, SEABYTE operates at the service **deployment level**, requiring separate deployments for each variant being tested. While it automates the experimental pipeline execution, it still faces the fundamental limitation of deployment-based approaches: the need to **maintain** and **orchestrate** multiple service instances for each behavioural variant.

## Research Gaps and Positioning

The literature review reveals several critical gaps that limit the practical application of fine-grained behavioural control in microservice architectures.

**Granularity Mismatch** Most existing approaches operate at a **coarse level of granularity** for practical behavioural control. **Infrastructure-level solutions** like deployment strategies work at the service level, requiring complete service duplication for each behavioural variant. **Application-level solutions** like feature flags require developers to embed conditional logic throughout their codebase manually. **A/B testing frameworks** typically operate at the deployment level, comparing entire system versions rather than individual algorithmic choices. This creates a fundamental problem: developers need fine-grained control over specific methods and algorithms, but existing tools force them to choose between coarse-grained service-level switching or manual conditional logic scattered throughout their code.

**Centralized Control Limitations** Existing solutions lack **unified management capabilities**. Configuration management systems provide centralised storage but no automatic discovery of configurable behaviours. A/B testing platforms require manual setup and coordination across multiple services. This forces operators to manually coordinate behavioural changes across distributed systems, increasing complexity and error probability.

**Runtime Flexibility vs. Code Clarity Trade-off** Current approaches force developers to choose between **runtime flexibility** and **code clarity**. Flexible approaches, such as extensive feature flagging, lead to complex and hard-to-maintain code. Clean

approaches, such as separate service deployments, sacrifice runtime adaptability and increase operational overhead. There is no existing solution that maintains clean, testable code while providing fine-grained runtime behavioural control.

The Marionette framework addresses these gaps by providing a unified solution that operates at **method-level granularity** while maintaining **clean code** organisation, automatically generates **control infrastructure** without requiring manual conditional logic and provides **centralised discovery and management** of behavioural variants across distributed systems.

## Contributions

This thesis makes several key contributions to the field of dynamic behavioural control in microservice architectures. Each contribution directly addresses one or more of the research gaps identified in the previous section.

**Architectural Framework for Behavioural Variants** We design and implement the **Marionette tool** that enables developers to define **multiple behavioural implementations within a single microservice codebase** while maintaining clean separation of concerns. Instead of scattering conditional logic throughout the code (as in feature flag approaches) or duplicating entire services (as in deployment-based approaches), developers implement **multiple versions of the same method** and keep the link between them in a configuration file. The framework automates the generation of all switching logic, keeping the business code clean and testable.

**Automatic Control Infrastructure Generation** We develop a mechanism that automatically generates the **control layer** necessary for runtime behaviour modification, eliminating the need for developers to implement switching logic and external interfaces manually. The framework transforms the source code of the original service. It injects a web-based interface with the necessary REST endpoints to **export the configuration management outside of the service** itself. In this way, an external agent can take over the control of the service, offering centralised management. This eliminates the developer burden associated with feature flags and manual configuration approaches.

**Centralised Discovery and Management System** We implement a centralised service (called Marionettist) that **automatically discovers** microservices with behavioural variants across the cluster and provides a unified interface for managing behaviour changes system-wide. Unlike existing configuration management systems that require manual

setup, our control plane automatically finds services with behavioural variants, understands their available behaviours, and provides a **single interface** for managing the entire system. This eliminates the coordination overhead present in existing distributed approaches.

**Method-Level A/B/n Testing** We demonstrate the framework’s capabilities through a **complete A/B/n testing implementation** that operates at the method level rather than the deployment level. Our approach automatically generates system-level configurations, switches between behavioural variants during runtime, collects configurable metrics, and produces performance rankings. This enables **fine-grained experimentation** within running systems without the operational overhead of managing multiple deployments, extending traditional A/B testing to a more practical and flexible approach.

**Empirical Validation in Microservice Environments** We provide empirical evaluation of the framework’s **effectiveness**, **performance impact**, and **usability** in realistic microservice deployment scenarios. Our validation demonstrates that the framework introduces acceptable performance overhead while significantly **reducing the complexity** of implementing and managing behavioural variants compared to existing approaches.

Beyond A/B testing, this framework enables numerous practical scenarios. Future work could explore **automated system configuration selection** among available variants, where an external controller service applies predefined rules or leverages **machine learning models** to optimise system behaviour with respect to specific metrics. One particularly promising application would be **energy consumption optimisation**, where developers could design critical methods with variants that consume different energy levels, enabling an external controller to find optimal configurations that balance performance with sustainability objectives.

# 1 | System Overview

The Marionette framework is made up of **multiple components** that work together to achieve dynamic behaviour control of microservices. This chapter aims to give the reader a high-level view of the system and, in particular, a description of the interactions and relationships between the different parts.

## 1.1. The Marionette Code Transformation Tool

The very first contribution produced by this project is a framework that supports developers in the definition and maintenance of **multiple behavioural variants** within the same microservice. It also abstracts away the implementation of the infrastructure layer to **externalise the control** of the variants out of the microservice, so that the variant can be **chosen dynamically** from external components. To better understand the relevance of this, let us consider a simple example.

### 1.1.1. Problem Definition

Consider an image processing service that enhances the appearance of pictures displayed on the user interface. Imagine that the developers have various possible implementations for the enhancement algorithm, and they are aware that all these alternatives have different trade-offs. Some consume fewer computing resources, while others are more expensive. Some apply heavy modifications to the original, while others are more lightweight and gentle.

The traditional approach to cope with this landscape of behavioural variants is to rely on **embedding conditional logic directly within the service code**. When a microservice needs to support multiple algorithms or processing strategies, developers typically implement all variants within the same codebase and use conditional statements to select the appropriate implementation at runtime.

The most common approaches include:

- **Direct conditional logic:** Using if-else statements or switch cases throughout the code to select different implementations based on request parameters, configuration values, or environmental conditions.
- **Strategy pattern:** Organising behavioural variants as separate strategy classes while using a factory or configuration mechanism to instantiate the appropriate strategy at runtime.
- **Configuration-driven selection:** Reading external configuration files or environment variables to determine which behavioural variant to activate during service initialisation or request processing.

While these patterns can provide runtime behavioural switching, they suffer from fundamental architectural limitations that restrict their practical applicability in dynamic microservice environments.

First, the **switching logic remains fundamentally static**. The rules deciding when to use each behavioural variant are hardcoded within the microservice implementation, meaning that any modification to these rules requires code changes and service redeployment. This prevents runtime adaptation to changing conditions and eliminates the possibility of responding dynamically to evolving system requirements.

Second, these approaches suffer from a **local decision-making scope**. Each microservice makes behavioural decisions based solely on locally available information, such as request parameters, local configuration files, or service-specific metrics. This constraint prevents coordinated behavioural changes across multiple services and significantly limits the system's ability to respond to system-wide conditions or optimisation goals. When behavioural decisions need to consider global state or cross-service coordination, this localised approach becomes inadequate.

As the number of behavioural variants grows, **maintenance complexity increases exponentially**. The conditional logic becomes increasingly complex and scattered throughout the codebase, making the code harder to understand, test, and maintain. Each new variant requires additional conditional branches, and the interaction between different behavioural paths becomes difficult to reason about and debug.

Finally, these approaches create an **operational visibility gap**. There is no centralised view of which behavioural variants are available across the distributed system or which variants are currently active in each service. This lack of visibility makes it extremely difficult for operators to understand the current system-wide behaviour configuration and to manage behavioural changes in a coordinated manner across the entire microservice

ecosystem.

These solutions do not allow the system to be highly flexible to changes, and controllable by some central entity (external to individual microservices) that can maintain a higher-level perspective on the entire system state and outer world metrics.

While it is technically possible to implement this controlling layer **manually**, doing so presents significant challenges. For a web-based solution, a developer could create dedicated REST endpoints within each microservice to allow external agents to control behavioural switching. This approach would externalise the control logic from the core business functionality, enabling centralised management of behavioural variants across the distributed system.

However, this manual approach introduces **implementation complexity and maintenance overhead**. The developer must manually create and maintain controller endpoints for each behavioural variation point, implement request parsing and validation logic for behaviour switching commands and maintain synchronisation between the available behavioural variants and their corresponding control interfaces. As the number of variation points increases, it becomes quite a big burden for the developer.

More critically, this manual approach **lacks standardisation across services**. Each development team would implement their own control interface conventions, parameter formats, and error handling mechanisms. This creates a heterogeneous landscape where each microservice exposes behavioural control differently, making it extremely difficult to build unified management tools or automated control systems that can work across the entire microservice ecosystem.

The Marionette framework addresses these challenges by providing an **automated approach** to generating the external control layer. Rather than requiring developers to implement control endpoints and switching logic manually, Marionette automatically generates a consistent control interface for any microservice with behavioural variants. This ensures uniform control mechanisms across all services while eliminating the implementation burden on developers, allowing them to focus on business logic rather than control infrastructure.

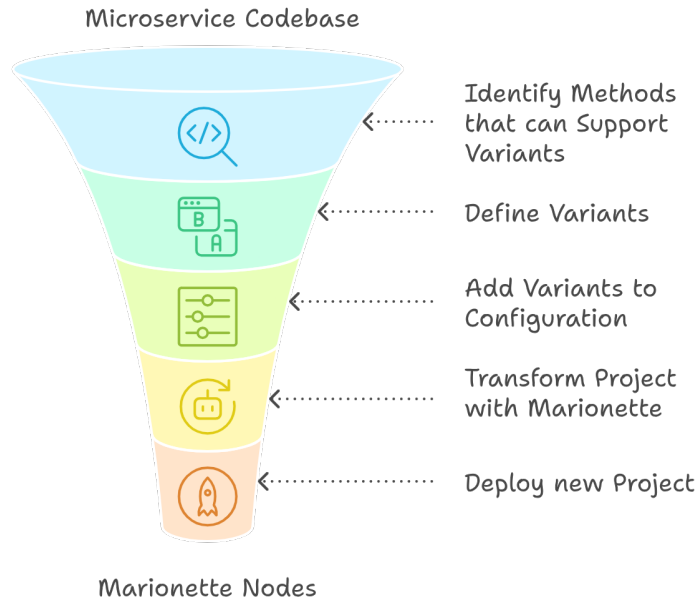
### 1.1.2. Marionette Tool to the Rescue

The solution proposed by this project is the use of a new tool that facilitates all the aforementioned critical aspects. The tool is called *Marionette* and the name was chosen based on an **analogy**. Indeed, microservices are seen as components whose behaviour can

be controlled from external agents, and so they are compared to marionettes, whose moves and dances are regulated by a marionettist. This analogy encompasses the entire work of this project. In fact, the centralised manager for behavioural display and management, introduced in Chapter 3, is called the Marionettist service.

Before starting with the high-level description of the tool usage, it is important to clarify some aspects of the **terminology** used. Currently, the Marionette tool is designed to work with Java-based projects. For this reason, during the discussion, some object-oriented specific terms may emerge, such as *method* or *class*. Nevertheless, it is important to note that the principles and transformation pipeline illustrated here can easily be ported to different frameworks as well, as they represent a **generic** and **abstract** transformation process that can work in possibly any platform of development. Moreover, the tool was initially designed to support existing projects, allowing developers to extend them with behavioural variations. For this reason, the concepts of *original method* or *original class* are often mentioned, as they refer to the implementation found in the project before the injection of behavioural variants.

To make use of the Marionette tool, developers are required to follow specific steps, which are illustrated, from top to bottom, in Figure 1.1.



**Figure 1.1:** Graphical funnel representing the steps to instrument a microservice to become a Marionette node.

First, they are asked to identify in the microservice codebase which methods require or can support behavioural variations. Then, they can add the alternative implementations



for the selected method in the same codebase. This is the first and very relevant **difference** compared with the **manual** approach. The user of this tool can define alternative implementations of a method in a **separate** class file, located anywhere in the codebase. It does not have to be in the same file as the original class, and **no complex conditional logic** is demanded to switch between variants. Simply put, for each variant implementation, the developer is tasked only with creating a new class file, maintaining the same signature as the original method, and changing the body to contain a different version of the logic.

Then, the tool needs the developer to fill in a **configuration file** at the root of the project that links all variants together. The intuitive idea behind this configuration file is that for each variant method, Marionette needs to know which file contains the implementation of the original variant and which other files contain alternative implementations.

Thanks to this basic information, the tool can create a **full copy** of the original project where some transformations are applied automatically. The choice not to work on the original code base was made to avoid **direct source code modifications** of the original project. The copy provides a clean workspace where the full transformation is observable, making the experience more transparent to the user.

After the transformation, whose implementation details are explored in depth in Chapter 2, the resulting microservice gains several powerful capabilities that fundamentally change how behavioural control can be managed. To begin with, the microservice **incorporates** all behavioural variants defined by the developer in the codebase, in such a way that it is possible to **switch** between them at runtime. Secondly, the control of which variant must be executed is automatically **exported** and **offered** to external agents through a web-based API. In simpler terms, the transformation automatically generates a standardised REST API that exposes all available behavioural variants to external management systems. This interface lays out the foundations to build a **centralised control system** that can discover, monitor, and coordinate behavioural changes across multiple microservices simultaneously, enabling system-wide optimisation and dynamic adaptation strategies that would be impossible with traditional service-level approaches. We are going to call instrumented microservices with the Marionette tool as **Marionette nodes** for simplicity.

## 1.2. Central Behaviour Management with the Marionettist Service

While the Marionette code transformation tool enables individual microservices to support multiple behavioural variants, managing these behaviours across an entire distributed system requires a centralised coordination mechanism. This is where the **Marionettist service** comes into play, serving as the control plane for behaviour management across the microservice application.

### 1.2.1. Control Plane Architecture Pattern

In distributed systems architecture, the distinction between control plane and data plane represents a fundamental **design principle** that separates management concerns from operational workloads. The **data plane** consists of the components that handle actual business traffic and user requests. In contrast, the **control plane** encompasses the systems responsible for configuration, coordination, and management of the data plane components.

This architectural pattern is widely adopted across modern infrastructure systems. In networking, routers separate packet forwarding (the data plane) from routing protocol management (the control plane). In container orchestration, Kubernetes separates workload execution (the data plane) from cluster management and scheduling (the control plane).

The Marionettist service extends this pattern to the domain of **behavioural variant management**. The Marionette-instrumented microservices constitute the data plane, processing business requests according to their configured behaviours, while the Marionettist service operates as the **control plane**, managing behaviour configurations and coordinating changes across the distributed system.

### 1.2.2. Role and Responsibilities

The Marionettist service assumes several critical responsibilities within the Marionette ecosystem.

**Service Discovery and Registration** The Marionettist service **discovers marionette nodes** within the infrastructure through automated scanning mechanisms. Upon finding a new service, it establishes communication channels and **retrieves** the service's

behavioural **configuration** schema, including available behaviours, default settings, and current state. This automated process is possible thanks to the standardised control infrastructure that is injected in all instrumented microservices by the Marionette tool, as described in Section 1.1.2. The discovery process ensures that the control plane maintains an up-to-date registry of all manageable services and their capabilities.

**Centralised and Dynamic Configuration Management** Rather than requiring manual configuration of each microservice instance, the Marionettist service provides a centralised repository for all behavioural configurations. Operators can use it directly to select the behaviour combination they desire to apply in the system. The behaviour of the discovered microservices can be changed dynamically, without requiring redeployment.

**Observability and Monitoring** The control plane aggregates customisable metrics from all managed services, providing a comprehensive view of system behaviour. This observability enables operators to understand the impact of different behavioural variants, identify performance bottlenecks, and make data-driven decisions about system configuration.

## 1.3. Application to A/B/n Testing Context

The combination of the Marionette code transformation tool and the Marionettist service creates a powerful and **versatile instrumentation framework**. This infrastructure establishes the foundation for sophisticated experimentation capabilities that can be applied across diverse contexts, from performance optimisation to feature validation. To demonstrate the full potential and practical applicability of the Marionette ecosystem, this project incorporates the development of an **automated A/B/n testing pipeline** that leverages the underlying behavioural variation infrastructure.

### 1.3.1. Conceptual Foundation

The possibility to build on top of behaviour management capabilities an infrastructure for systematic A/B/n experimentation **emerges naturally** from the architectural properties of the Marionette system. The code transformation tool enables developers to create microservices that expose behavioural variation points with method-level granularity, effectively creating multiple implementation alternatives.

Simultaneously, the Marionettist service provides **centralised visibility and control** over the behavioural configuration of each microservice within the distributed system.

This centralisation creates a unique opportunity: the ability to orchestrate behavioural changes across multiple services simultaneously and **observe the system-wide effects** of different configuration combinations.

Therefore, the various method implementations can be thought of as **experiments** that developers and operations teams want to evaluate and compare systematically. Each behavioural variant represents a hypothesis about how a particular implementation approach will impact system performance, user satisfaction, or other configurable metrics. A structured A/B/n testing pipeline built on top of the Marionette tools would allow **easy experimentation** and **comparison** of alternatives.

### 1.3.2. Multi-Dimensional Experimentation

Traditional A/B testing frameworks focus on testing changes within a single service or component, which provides limited insight into how different services interact in a distributed system. The Marionette-based A/B/n testing framework enables a fundamentally different approach: **multi-dimensional experimentation** across **multiple microservices** simultaneously.

The key innovation is the ability to test behavioural variations in **several microservices at once** and explore all possible combinations between them. For example, an experiment could simultaneously test different image processing algorithms in one service, various caching strategies in another service, and different database optimisation techniques in a third service. Rather than testing these changes in isolation, the framework systematically explores how different combinations of these behaviours work together and affect overall system performance. This multi-dimensional approach reveals important interactions that would remain hidden in traditional single-service testing. An example could be a lightweight image processing algorithm that performs well on its own but creates bottlenecks when combined with aggressive caching, or a computationally expensive processing variant that might be justified when paired with optimised database queries.

### 1.3.3. High-level Experimental System View

A full explanation of the implementation of the testing pipeline can be found in Chapter 4, as well as in the experimentation section of this paper 5. In this introduction, we aim to provide a high-level overview of the components necessary to set up an A/B/n test successfully.

First, we need to start with an existing microservice-based application, in which developers

want to inject behavioural variants and study their impact at the system level. Let us create an example with an application composed of two simple microservices. The UI service builds the interface for the user, and it interacts with the ImageStore service to obtain user-related images. The ImageStore service is responsible for receiving images, applying a complex compression algorithm to conserve space, and storing them locally for future use. Figure 1.2 shows the interactions among components.

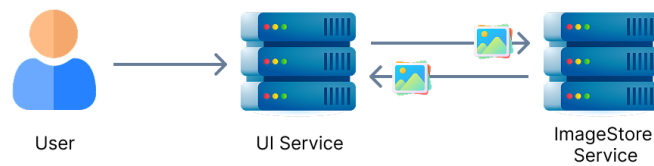


Figure 1.2: Simple microservice-based application with two services

Let us assume that the application developers want to try out different compression algorithms within the ImageStore service and test how the system responds to these various versions. They implement them in the microservice codebase and then use the Marionette tool to generate the transformed version of the project that exposes the **API for external control**, as described in Section 1.1.2. For example, they might implement three variants: a fast, low-compression algorithm, a balanced, medium-compression algorithm, and a slow, high-compression algorithm.

Once the transformed services are deployed, the Marionettist service **automatically discovers** them through the standardised control APIs and presents the user with a graphical view. Figure 1.3 shows the components involved in this scenario.

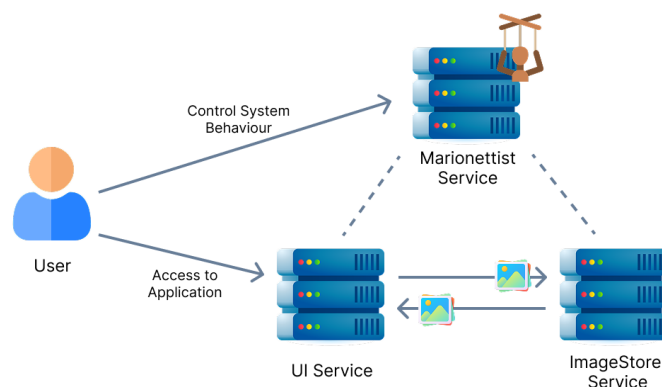


Figure 1.3: Simple microservice-based application with two services and the Marionettist controller.

At this point, it becomes easier to understand how the automatic A/B/n testing pipeline works. The user requests the Marionettist service to start a testing session with some duration, let us say two hours in total. The Marionettist service then **generates all possible combinations of variants** at the system level. In this case, there are only a few variants, which represent the different compression methodologies injected by the developers. The total duration of the experiment is equally split among the variants. In our two-hour example, each compression algorithm would be tested for 40 minutes, allowing sufficient time to collect meaningful performance data. During each time slice, the Marionettist service controls the system to **apply one behavioural configuration at a time** and **extract performance metrics** while the system is running. These metrics might include response time, CPU usage, and memory consumption, and are configurable based on the developers' optimisation needs. At the end of the experiment, the A/B/n pipeline is therefore able to present a **comparative analysis** showing how each compression algorithm performed across all measured metrics, enabling developers to make data-driven decisions about which implementation to deploy in production.

## 2 | Marionette: A Development Tool for Microservices with Runtime Variable Behaviour

The first Chapter 1 introduces some development challenges related to enabling **microservice behaviour variability** from the very beginning. To achieve the coexistence in the same codebase of multiple behaviours, it is necessary to introduce complex conditional statements scattered around the project that select the behaviour to be executed in any specific situation. Furthermore, an additional step is that of **externalising the control** over the behavioural variant that must be executed. This means to allow **external agents**, outside the microservice, to decide the behaviour to be applied. To accomplish this complementary goal, the developer is asked to implement manually a notable number of components that connect the external input (for instance, a web request) to internal logic (the selector of the behaviour). For complex projects, in which the behavioural variants might be numerous, this manual technique generates a significant burden on the developer, especially in terms of maintaining the code and variations.

As a solution to this problem, this chapter dives deeper into the internal structure and architecture of the **Marionette tool**. This can be thought of as a support for developers to reduce the complexity of managing behavioural variants through three fundamental advantages. First, the tool **automatically generates** all the necessary **infrastructure code**, eliminating the need for developers to manually implement conditional logic, configuration management, and external API endpoints. Second, it maintains complete separation of concerns by keeping business logic **clean** and focused on domain functionality. At the same time, the behavioural variant management is handled transparently through generated code that operates independently of the core business methods. Third, the tool **ensures consistency** and reduces errors by automatically generating **standardised implementations** for all behavioural variants, eliminating the manual coding mistakes and inconsistencies that commonly arise when developers implement multiple behaviour paths by hand.

## 2.1. A New Microservice Pattern

The goal of the Marionette tool is **generalising** the behaviour controlling infrastructure within a microservice codebase, to make it applicable in the same way to all the services that are configured to have multiple behavioural variants. Therefore, the work related to the Marionette tool started with the **design** of this controlling infrastructure, to identify all the components making part of it and analysing the interaction of this controlling layer with the existing customary codebase of the microservice.

We designed a **generic microservice pattern**, as the ideas and components can be reused to solve similar problems. Figure 2.1 shows the major actors and their interactions.

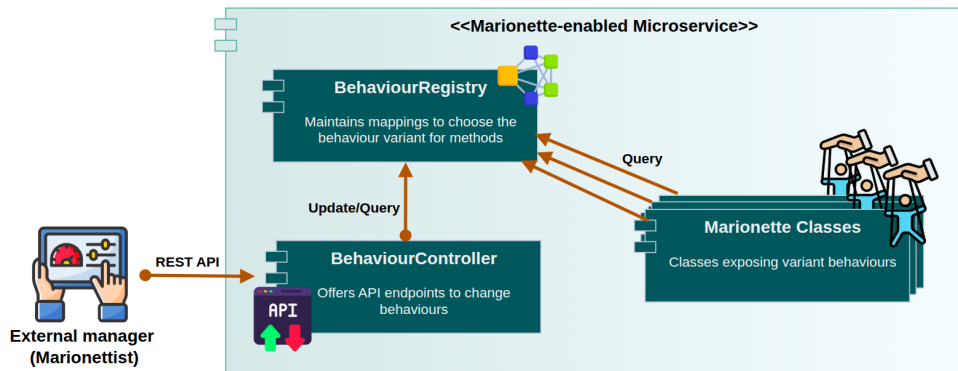


Figure 2.1: Marionette pattern with its main components, the marionette classes, the behaviour registry, and the behaviour controller

### 2.1.1. The Marionette Classes

The **Marionette classes** are the user-defined classes for which there is **at least one behavioural variant**. This means all the classes that contain at least one method for which the developer defined another variant implementation.

The Marionette classes are standard classes, but they are the main **target of modification** during the source code transformation applied by the Marionette tool. The details of how the transformation is performed are described in Section 2.2 of this chapter. At this level of abstraction, it is only important to know that the new code injected in these classes performs a **lookup** on the **behaviour registry** component (see Section 2.1.2) every time a method with variants is invoked. In this way, the marionette class **delegates** the **runtime selection** of behavioural implementations to the central registry component, allowing it to maintain a global mapping of behaviours that all instances will comply with.



### 2.1.2. The Behaviour Registry

The **behaviour registry** is a fundamental component of this pattern, as it holds all the **mappings** between **method entities** and the **behavioural variant** of each method that should be executed. This component is designed to be an **evolving data structure** that allows concurrent modifications. It is the central source of truth that is queried by all the marionette classes whenever a method with variant behaviour is invoked. This means that, as soon as one of the mappings stored in the behaviour registry is changed, all the marionette classes containing that method will automatically change in behaviour as well.

### 2.1.3. The Behaviour Controller

The **behaviour controller** represents the component of the architecture responsible for intercepting user requests related to behaviour management and turning them into **actions on the behaviour registry**. The controller enables external entities to interact with the internal system by modifying the mappings stored within the behaviour registry. At the same time, the controller exposes the mappings of the behaviour registry to the outside world, allowing other actors to understand the current behavioural configuration of the microservice.

As it is possible to see in the bottom left of the image, the main client for the interface exposed by the behaviour controller is the **Marionettist service** itself. Thanks to the controller, the Marionettist service can:

1. **Automatically detect nodes** with variable behaviours (Marionette nodes).
2. **Read the current behavioural configuration** of the Marionette nodes.
3. Expose the current behavioural configuration of the Marionette nodes to the user of the infrastructure through a simplified **user interface**.
4. Allow easy **modification of the behaviours** through the user interface.

### 2.1.4. The Generality of the Pattern

The Marionette pattern was designed to be **abstract** and **generic**. As can be seen from the figures and descriptions, there is no mention of a specific framework or language that must be used to implement it. Thus, it can be seen as an abstract description of how the code inside a microservice can be organised and structured to provide:

1. A set of classes with variable behaviour.
2. A control layer that exposes the decision on the behaviour to employ to components outside the microservice.

For the implementation of this particular project, **Java** is going to be used as the language of choice, but the same approach can be applied in other languages as well.

## 2.2. Source Code Transformation Pipeline - An Abstract View

The Marionette tool performs a set of transformations on the original project's codebase to provide the user with the whole infrastructure for runtime behaviour management. It is possible to think about the source code transformation as a pipeline of **sequential steps**, described in the following.

### 2.2.1. Definition of Multiple Behaviours

Let us assume the developer implemented a microservice and wants to make its behaviour variable and controllable from the outside with the Marionette tool. The first step is the **definition** of the different **behavioural variants**. Starting simply, the developer will identify one method in any class of the microservice for which they would like to expose alternative implementations. They will proceed by implementing the variant implementation of that same method in a **separate** class file.

Keeping the different implementation variants in separate class files is a design decision made to make the tool as **invisible** and lightweight on the original source code as possible. This approach avoids cluttering the original code with alternative definitions and instead isolates the variability in well-scoped, dedicated files. It also ensures that developers can maintain and evolve the original microservice codebase **without interference** from the adaptation framework, while still enabling Marionette to integrate and switch between variants at runtime seamlessly.

Once the variant implementation is over, the developer is asked to specify the link between the method behaviours in a **configuration file** for the tool to work. For more details on the configuration file syntax, refer to Section 2.3, which provides a deeper examination of the tool's technological and implementation-related aspects. For now, it is sufficient to know that through a configuration file, the user can specify the location of the original method in the codebase, as well as the locations of all its variant implementations. The

transformation pipeline of the Marionette tool starts from this point.

### 2.2.2. Copy of Original Project

Before any source code transformation begins, the Marionette tool creates a **complete copy** of the original project and performs all subsequent operations exclusively on this copy. This fundamental design decision serves several important purposes.

The primary motivation is **safety** and **reversibility**. Since the Marionette tool performs extensive source code modifications, working directly on the original codebase would be a risky approach. Any unexpected errors during the transformation process could potentially corrupt or damage the original source code, resulting in the loss of work or project instability. By operating on a copy, developers can always **return to their original**, unmodified codebase if needed.

This design choice also promotes **transparency** and **developer confidence**. The tool does not hide or obscure any of its transformations. Instead, developers can examine the entire transformed codebase to understand exactly what changes were made and compare the original and transformed versions side by side.

From a **workflow integration** perspective, this approach fits naturally into existing development processes. After the transformation, developers receive a fully functional copy of their project that has been **enhanced** with runtime behaviour management capabilities. This transformed project is ready for immediate **compilation, testing, and deployment** without requiring any manual intervention or additional setup steps.

### 2.2.3. Marionette Class Creation

The first manipulation applied to the project's copy is a **source code transformation step** that results in the generation of what we call Marionette classes, defined in Section 2.1.1. Indeed, thanks to the configuration file left by the developer, the tool can identify in the codebase the list of original methods, as well as all the variant implementations associated with each original method. The step of creation of the marionette classes is fundamentally a source code transformation step in which:

1. All the variant implementations of a method are **merged** into the class file of their corresponding original method.
2. The original method implementation is **substituted with a conditional statement** (switch) that selects the implementation to be executed. The selection process

is **delegated to the central behaviour registry** in this specific point of the code.

3. The extracted original method implementation becomes a **separate new method of the same class**, invoked as one of the options of the switch statement.

For this to work, **method name clashes** must be resolved, and this is achieved as follows. To make the entire infrastructure more user-friendly, the configuration file allows developers to associate a custom string **identifier** with each method implementation (original and variant). The identifiers are used to refactor the names of the methods while merging the whole code into a single class file to avoid name collision problems.

An example of the source code transformation process in pseudo-code would facilitate the understanding. The pseudo-code Section 2.1 shows a class called `Imageprocessor` that contains a method `enhanceImage` to process images and enhance their look.

---

**Algorithm 2.1:** Original class implementation

---

```
class ImageProcessor {  
  
    public BufferedImage enhanceImage(BufferedImage image) {  
  
        // Standard image enhancement processing  
        applyBrightnessCorrection(image);  
        applyContrastAdjustment(image);  
        return processedImage;  
    }  
  
    // Other methods...  
}
```

---

The developers of the microservice containing this class want to **define multiple variants** of the image processing step and **control** the way the microservice acts from an **external controller** component. To achieve this, they first define an alternative version of the previous method in a separate class called `ImageProcessorAdvanced`, as shown in the pseudo-code Section 2.2.

---

**Algorithm 2.2:** Variant implementation of the method for enhancing image looks.

---

```

class ImageProcessorAdvanced {

    public BufferedImage enhanceImage(BufferedImage image) {

        // Advanced AI-based enhancement processing
        applyNoiseReduction(image);
        applyAIBasedEnhancement(image);
        applyBrightnessCorrection(image);
        applyContrastAdjustment(image);
        return processedImage;
    }

}

```

---

When the Marionette tool processes this information and applies source code changes, it first **parses** all implementations of the method, both the original one and the variant. Then, it changes the names of each method by appending the identifiers defined by the user in the configuration file to each method. Let us assume for now that the user set up the identifier for the original method as *standard* and for the variant as *advanced*. Finally, it **merges** all these methods inside the **original class file**, so in this case, in the implementation of the class `ImageProcessor`. The pseudo-code Section 2.3 shows the final result.

---

**Algorithm 2.3:** Marionette class after the transformation applied by the Marionette tool.

---

```

class ImageProcessor {

    // Original implementation renamed with behaviour identifier
    private BufferedImage enhanceImage_standard(BufferedImage image) {

        // Same implementation as original class
    }

    // Variant implementation merged into the same class
    private BufferedImage enhanceImage_advanced(BufferedImage image) {

        // Same implementation as variant class
    }

    // Other methods remain unchanged...
}

```

---

The only missing step is the **transformation of the original method** `enhanceImage`. The original method implementation is transformed into a **dispatcher**, meaning a component that switches between possible implementations of the algorithm based on conditional logic. It **queries the central behaviour registry** component for the currently applied method implementation alternative and then dispatches the call to the correct variant. The pseudo-code Section 2.4 shows the original method implementation after the transformation.

---

**Algorithm 2.4:** Original method body after the transformation applied by the Marionette tool.

---

```
class ImageProcessor {  
    public BufferedImage enhanceImage(BufferedImage image) {  
        // Query the behaviour registry for current behaviour selection  
        String behaviorId = BehaviorRegistry.getBehaviorId("ImageProcessor",  
            "enhanceImage");  
        switch(behaviorId) {  
            case "standard":  
                return enhanceImage_standard(image);  
            case "advanced":  
                return enhanceImage_advanced(image);  
            default:  
                return enhanceImage_standard(image);  
        }  
    }  
}
```

---

Figure 2.2 shows the same transformation step visually.

What makes this code transformation trick powerful is its **simplicity** and the consequent **little impact** that it has on the external code, depending on the class being modified. The intuitive idea is the following: after this transformation on the class, no external code needs to be changed because the **interface** that the class exposes **is still the same**. The only difference now is that all the external invocations to the `enhanceImage` method will first pass through the **dispatching logic**, which in turn **delegates** the responsibility of choosing the behaviour to the behaviour registry.

The same process can be applied for all other methods, with variants defined by the user, and the final transformed project will be compilable.

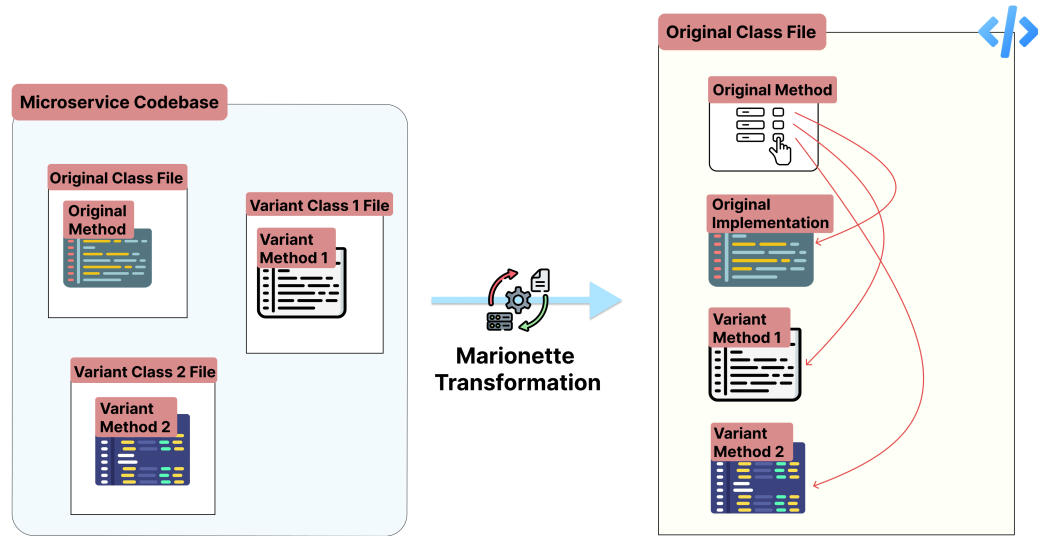


Figure 2.2: A visual representation of the source code transformation process. On the left, the code of the original project in which multiple class files hold the variant implementations. On the right, the merging process was completed on the original class file, incorporating all behavioural variants.

### 2.2.4. Source Code Injection

Examining the original design of the Marionette pattern in Section 2.1, we can observe that some components are still missing from this transformation pipeline. While the marionette classes have been created, the **behaviour control layer** still needs to be incorporated in the source code. In other words, the microservice needs to have an infrastructure of classes that allow the implementation of the central behaviour registry, as well as the behaviour controller that exposes the manipulation of the service behaviour to outside components.

To do that, the transformation pipeline goes through a phase of **source code injection**, which means that **custom classes** are injected into the newly created project in a folder specified by the user, which must be included in the compilation process.

Among the contributions of this fundamental step, the most important ones are

1. Incorporation in the project of the **behaviour registry implementation**.
2. Incorporation in the project of the **controller implementation**.
3. Injection of the **startup code** to populate the behaviour registry at system startup and activate the controller.

Figure 2.3 shows the separation between pre-existing and injected code in a marionette-enabled microservice.

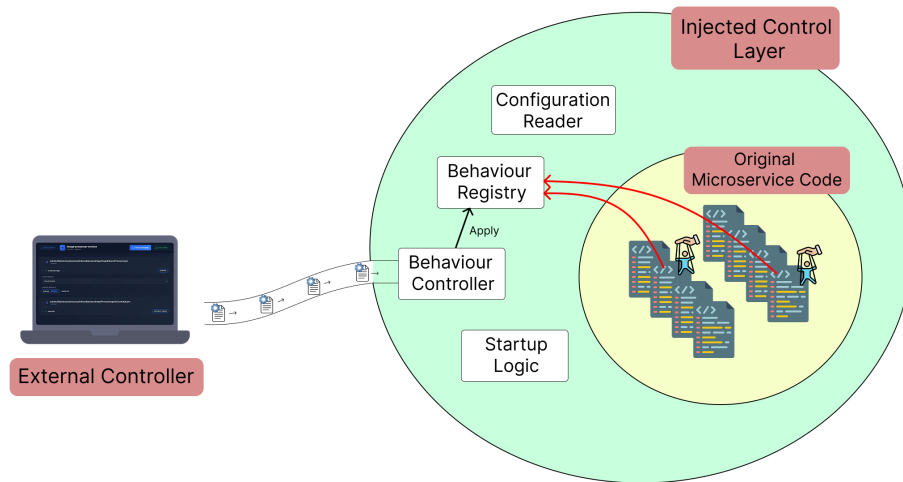


Figure 2.3: Graphical representation of the separation between pre-existing code and injected infrastructure. The green circle represents the most relevant subset of components injected in a marionette-enabled microservice. The Behaviour Registry serves as the central mapping for behaviours, while the Behaviour Controller provides an interface for external control.

The injected startup code will initiate all relevant components when the system boots up so that the new microservice is immediately **available for connections** coming from external controllers.

## 2.3. Concrete Usage of the Marionette Tool in Java

This section is dedicated to introducing the reader to the implementation details of the Marionette tool, which was developed to integrate seamlessly with a **Java Spring** microservice-based application. The core concepts and abstract design illustrated in Sections 2.1 and 2.2 form the **theoretical foundations** that enable the implementation of the same infrastructure in other frameworks as well. Java Spring was chosen as a starting point due to its production-ready features, well-written documentation, and vast developer support. Nevertheless, many of the choices taken along the path show the attention paid to make the system **portable** to other environments.



### 2.3.1. The Marionette XML Configuration File

Assuming the developer has already defined some variants for their own methods of choice, the first step that enables the automation process offered by the Marionette tool is the creation of an **XML configuration file** storing all the information needed for the transformation to happen. The choice to use XML over JSON was thoroughly assessed, and XML was favoured because it does not require **external libraries** to be parsed. The Java Standard Library provides basic functionalities for parsing an XML file, which is extremely valuable in this project. The Marionette tool performs source code modification on a custom project, and it is not possible to know in advance what dependencies the transformed project will express. While it would be possible to force the dependency on a JSON parsing library, it is not a transparent solution for the tool's users, as it would alter the project's dependencies. It could also introduce errors or bugs with library versioning and transitive dependencies if not handled properly.

Considering the example illustrated in Section 2.2 of the image processor again, it is possible to explain the mandatory fields that must be specified in the configuration file. The explanation focuses on chunks of the XML file for ease of reading.

### Global Configurations

These are configurations that apply at the project level; they are not related to a specific class and its variant behaviours.

```
<marionetteConfig>

    <microserviceName>image-processor-service</microserviceName>
    <srcRoot>src/main/java</srcRoot>
    <injectCodeRoot>com/outfit/__marionette</injectCodeRoot>
    ...

</marionetteConfig>
```

The `microserviceName` is self-explanatory. The new project copy will be created under a folder named after the service, as expressed in this field, and appended with the suffix *-marionette*. The `srcRoot` is a way to specify the **source directory** where the class files can be found. All the paths specified later in the configuration to locate class files will be relative to this root. The `injectCodeRoot` is a path relative to `srcRoot` that will be the root folder where the whole Marionette infrastructure for dynamic behaviour control will

be injected. In here, it will be possible to find the collection of classes implementing the **behaviour registry**, the **controller** component and the **startup code** to initialise the modified system.

## Marionette Classes Configuration

The most relevant part of the configuration is related to the **logical link** that the developer must maintain between the original implementation of a class and the classes containing alternative definitions of methods. In the example of the `ImageProcessor` class, the configuration would appear as follows.

```
<!-- Full definition of a marionette class with variants -->
<marionetteClass>
  <originalClass>
    <path>com/outfit/ImageProcessor.java</path>
    <behaviours>
      <behaviour>
        <name>enhanceImage</name>
        <id>standard</id>
      </behaviour>
    </behaviours>
  </originalClass>

  <variantClasses>
    <variantClass>
      <path>com/outfit/marionette/ImageProcessorAdvanced.java</path>
      <behaviours>
        <behaviour>
          <name>enhanceImage</name>
          <id>advanced</id>
        </behaviour>
      </behaviours>
    </variantClass>
  </variantClasses>
</marionetteClass>
```

The marionette class configuration is composed of an `originalClass` component and a list of `variantClasses`. In the `originalClass` section, it is possible to specify the location of the class file (relative to the global `srcRoot`), as well as the list of `behaviours` that present variants. A behaviour is essentially a **method**, with a `name` and an `id`

which is user-defined and will later be used to reference this particular configuration. The `variantClasses` follow a very similar structure; they have a `path` to be located and a list of behaviour configurations that are associated with their custom identifier.

It is possible to notice here the **flexibility** of the framework with regard to the positioning and organisation of the variant code. Indeed, it is possible to create a separate class for each variant method, or keep multiple (different) methods with variants belonging to the same class in the same class file.

### Platform-agnostic Configuration

The configuration file used by the tool was maintained as **platform-agnostic** as possible. No Java-specific notation or convention is used. For instance, the fully-qualified name of a class could be used to locate a class in a Java project (e.g. `com.outfit.ImageProcessor`). Instead, common file paths are employed to make the configuration portable to **other languages** in the future.

#### 2.3.2. Transformation Process in a Java Environment

With the configuration file ready, the user can employ the Marionette tool as a **command-line tool** that operates on the project root directory. The steps that the Marionette tool applies are the ones explained in Section 2.2 and summarised here at a very high level.

1. **Parse** the XML configuration file.
2. Create a **full copy** of the project.
3. For each `marionetteClass` in the configuration, **modify the source code** of the original class to integrate the variant versions.
4. **Inject** in the project the set of classes implementing the **behaviour control infrastructure** (startup code, behaviour registry, controller...).

In this section, we would like to provide some additional details of the **design decisions** made while developing the tool in Java. While the generation of the Marionette classes is equivalent to the one presented in Section 2.2, there is a Java-specific step that was integrated to make the tool easier to use: the `import` statements of the original and variant classes are **merged** together. The idea is that the user should be able to import additional classes and components while implementing the variant class. If some of these components are not needed in the original implementation, it is not necessary to include them there. The tool takes care of comparing the import statements lists and merging

them appropriately.

Concerning the implementation of the components making up the **behaviour control layer**, we report here the design of the **behaviour registry** and the **controller**.

## The Behaviour Registry Component

The behaviour registry was implemented as a **nested map data structure**, i.e. a map containing another map. The outermost map links each **class name** to **another map** containing the configurations of single methods. Indeed, a class can have multiple methods exposing variant behaviour. The innermost map links each **method** to a data structure holding the **method configuration**. The method configuration stores the currently used variant identifier as well as the list of available identifiers for other alternative implementations.

Returning to the example of the microservice containing the `ImageProcessor` class, the behaviour registry would maintain this association internally, as shown in Figure 2.4.

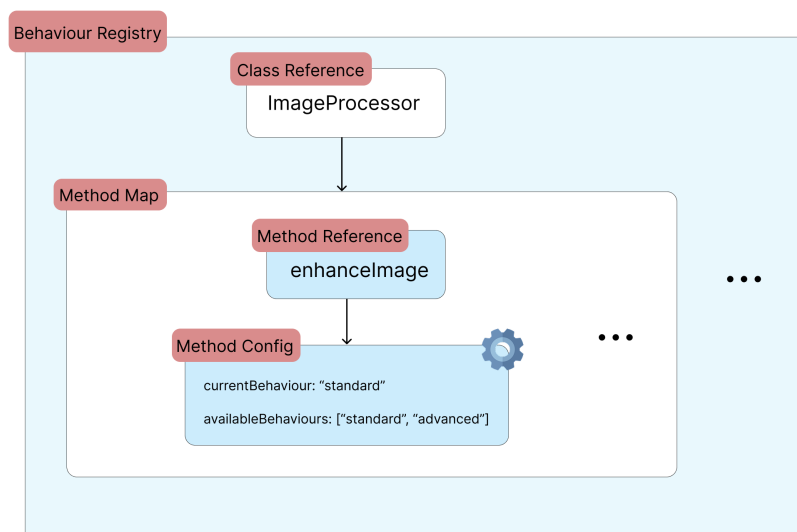


Figure 2.4: Visual representation of the state of the behaviour registry in the example of the image processor service.

The outermost rectangle represents the entire data structure, which is implemented as a map. In this case, there is an entry that links the `ImageProcessor` class with another map holding the configurations of its methods. In the innermost map, the method with variants `enhanceImage` is associated with its configuration. At the time this snapshot was taken, the method variant being executed was the *standard* one. The available variants

are identified as *standard* and *advanced*.

The three dots on the side indicate that the behaviour registry can hold multiple method configurations for the same class, as well as configurations across multiple classes if set up by the user.

## The Controller Component

Since the implementation is developed in a Java Spring environment, the controller component responsible for **exposing access** to the microservice **behaviour configuration** was implemented as a simple `@RestController`. Thanks to the automatic discovery of `@Controller` classes offered by Java Spring, it was sufficient to define the necessary endpoints and inject the class into a project package where the Java Spring runtime container performed automated scanning. This is a **requirement** for the correct functioning of the system that the user must comply with. For this reason, the folder path where all the source code is injected is **configurable** in the XML file, as shown in 2.3.1.

The three most important endpoints that are exposed by the controller are represented in Table 2.1.

**Table 2.1:** Core Marionette Framework API Endpoints on Instrumented Microservices

| Endpoint                         | Method | Parameters                               |
|----------------------------------|--------|------------------------------------------|
| /marionette/api/isMarionette     | GET    | None                                     |
| /marionette/api/getConfiguration | GET    | None                                     |
| /marionette/api/changeBehaviour  | POST   | className,<br>methodName,<br>behaviourId |

The `/marionette/api/isMarionette` endpoint serves as the primary **discovery mechanism** for the external agents that need to control the microservice. They query this endpoint to determine whether a particular service supports Marionette behavioural configuration.

**Configuration retrieval** occurs through the `/marionette/api/getConfiguration` endpoint, which returns a comprehensive JSON representation of the service's behavioural capabilities. This response includes all instrumented classes and methods, the complete set of available behaviours for each method, current runtime settings, and default configurations. The Marionettist service can use this information to build its internal model of the service state and present configuration options to administrators. The JSON format

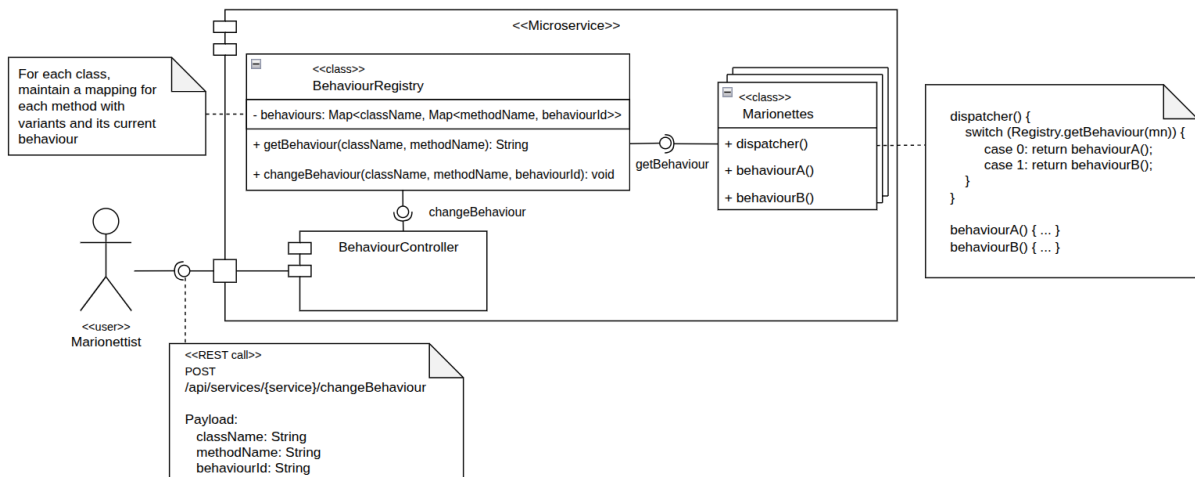


Figure 2.5: Component diagram showing the interactions between the behaviour registry, the behaviour controller, the external manager (Marionettist) and the marionette classes.

was chosen in this case to reduce the verbosity of the data transfer. The limitations on JSON parsing libraries mentioned in Section 2.3.1 are no longer a problem for the Marionettist service, as this service is completely developed as part of this project and does not need to adapt to existing custom implementations.

**Runtime behaviour modification** happens with an HTTP request to the endpoint `/marionette/api/changeBehaviour`, which accepts POST requests containing the **target method coordinates** and desired **behaviour identifier**. The request body must specify the `className` to identify the target class, the `methodName` to pinpoint the specific method, and the `behaviourId` representing the desired behaviour from the method's available options.

### UML Component Diagram

Figure 2.5 shows more formally the component diagram of an instrumented marionette node.

The pivotal element is the **BehaviourRegistry**, which is a **single class**, offering two fundamental methods to interact with the data structure. The `getBehaviour` method allows marionette classes inside the microservice to know which behaviour should be selected among the available ones. The **BehaviourController** employs the `changeBehaviour` method to **actuate the requests** coming from external users on the **BehaviourRegistry**.

On the right, it is possible to notice the **structure of a marionette class**. The original method is turned into a **dispatcher**, which queries the **BehaviourRegistry** to select the

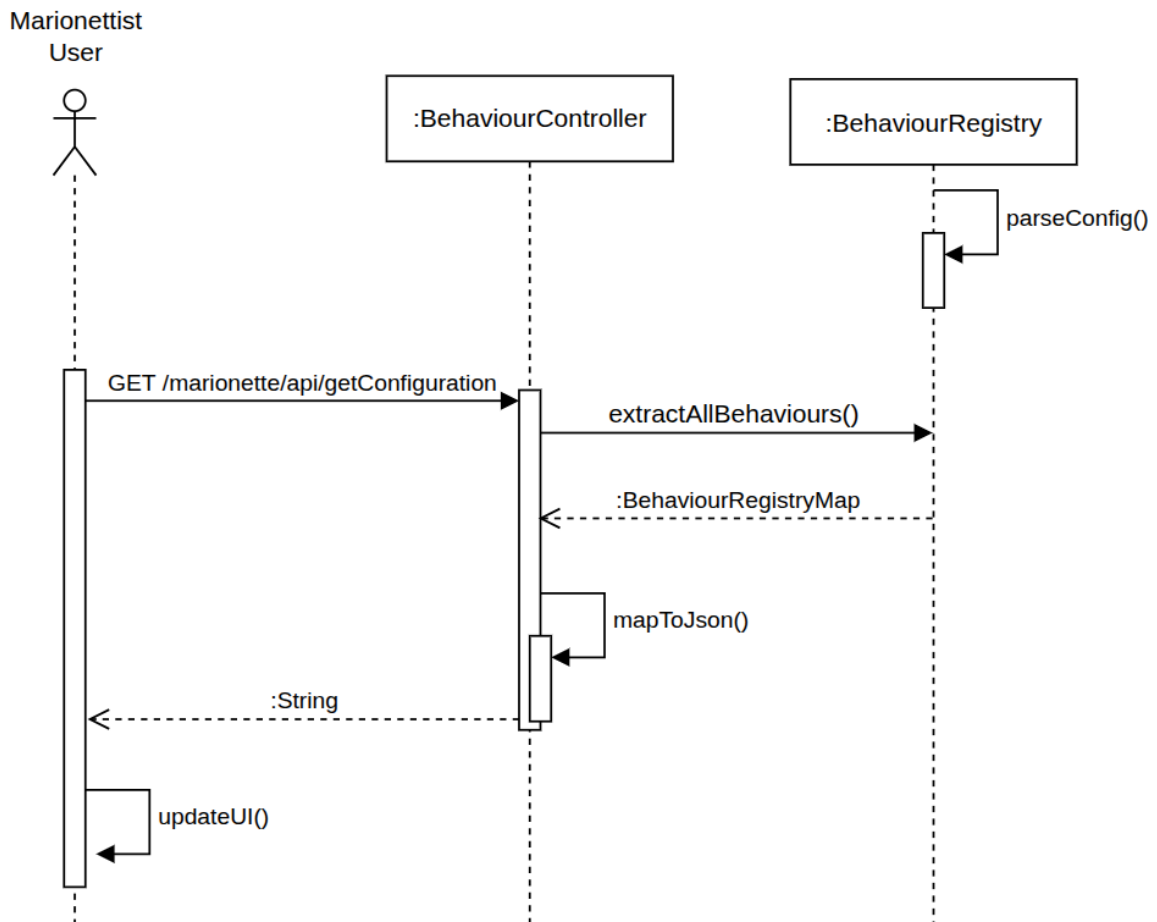


Figure 2.6: Sequence diagram illustrating the interaction between the external behaviour manager (Marionettist) and the instrumented marionette service when reading the behavioural configuration.

right execution variant.

### 2.3.3. Sequence Diagrams

After being instrumented with the Marionette tool, a microservice can interact with external controllers to **expose its internal behavioural configuration** and allow them to **change** it via HTTP requests.

Figure 2.6 illustrates the interactions that occur between the external behaviour manager (Marionettist service) and the instrumented marionette service when **reading the internal service configuration**.

At the top-right corner of the diagram, it is possible to see the **initialisation** of the `BehaviourRegistry` component, which occurs at service startup. The registry is an in-

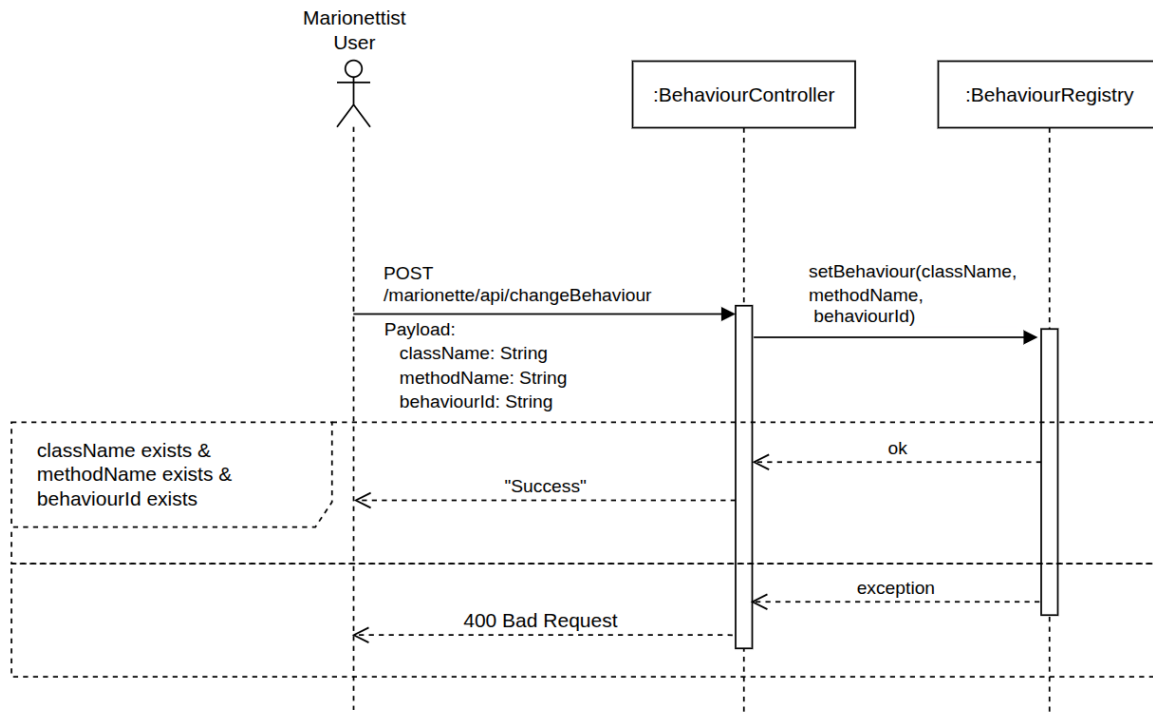


Figure 2.7: Sequence diagram showing the interactions between the external behaviour manager (Marionettist) and the marionette service components when changing the behavioural configuration.

memory structure that reads the behavioural configuration of the service from a file when the class is first loaded. Once the initialisation completes, the service can expose its configuration to external entities.

To fetch the marionette node behavioural configuration, the Marionettist service initiates communication with an HTTP request to the marionette API, requesting the full configuration export. The `BehaviourController`, injected in the marionette node, intercepts the call, extracts the data from the `BehaviourRegistry`, maps it to a JSON format and returns it to the caller. In this way, the Marionettist can update the graphical interface offered to the user with this information.

Figure 2.7 details the interactions between the Marionettist service and an instrumented marionette service when handling a request to **change the behavioural configuration**.

The Marionettist service initiates the communication with an HTTP POST request in which it specifies the class name and the method name to pinpoint a specific method with variants, as well as the **new behaviour identifier** to be applied. The `BehaviourController` intercepts the call and forwards the request to the internal `BehaviourRegistry`. If the request data is correct, which means that the class name, method name, and new be-



haviour identifiers exist in the current configuration, the request succeeds. Otherwise, an exception is raised, and the client receives a bad request HTTP response.



# 3 | The Marionettist Service for Centralised Behaviour Management

The Marionette tool illustrated in Chapter 2 supports the modification of microservices to make them easily **expose control of behavioural variants**. However, in a distributed microservices architecture comprising dozens or hundreds of services, manually coordinating behavioural changes across individual services becomes both impractical and error-prone. For this reason, we developed the **Marionettist service**, a **centralised control plane** that provides **unified management** of Marionette-enabled microservices within a distributed system.

This microservice was designed as a support to developers, and it was built to integrate as easily as possible with an existing microservice-based application. There are many advantages in using this microservice to control the instrumented marionette components. First of all, the Marionettist service can be configured to support **automatic discovery** of the available Marionette services within the cluster, and to refresh the state of the available nodes periodically. In this way, when the Marionettist service starts up, it automatically detects marionette nodes and extracts from them their current **behavioural configuration**. This process captures the full catalogue of available behavioural variants for each instrumented method. The control plane then consolidates this distributed configuration state into a unified, **centralised registry** that serves as the authoritative source of truth for the entire system's behavioural configuration.

The data collected from the marionette nodes can be used to populate a graphical **user interface** that greatly simplifies the **operational management** of behaviours. By looking at this interface, developers and operators can see the current state of all their services in one place, and they can quickly identify which services are using modified behaviours instead of their default settings. Most importantly, the interface allows users to **make changes** to individual methods within a service. This means operators can modify ex-

actly the behaviour they want to test without accidentally affecting other parts of the same service or other services in the application.

Finally, the Marionettist service automatically extracts **runtime metrics** from the marionette nodes. It shows them to the user through the graphical interface, right next to the service's current behavioural configuration. The set of metrics to be monitored is itself **configurable** to be adapted to different kinds of users' needs. This is a great feature because it allows operators to modify some method behaviour selection and see the reaction of the system right away, on the same screen.

This chapter is dedicated to introducing the original **design** of the Marionettist service at a high level, including all its components, and then to dive deeper into the **implementation details** to explore the features currently supported.

### 3.1. Service Design

The design of the Marionettist service started by collecting a set of **requirements** that the system had to fulfil. A full list of the identified requirements can be seen in Table 3.1.

Table 3.1: Requirements for the Marionettist Service

| ID | Requirement Description                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | The service shall <b>integrate</b> into an existing microservice-based application with minimal configuration required.                            |
| R2 | The service shall provide a way to <b>automatically discover</b> the marionette-enabled microservices in the cluster.                              |
| R3 | The service shall automatically <b>extract</b> from the marionette-enabled microservices their <b>behavioural configuration</b> .                  |
| R4 | The service shall provide to the user an intuitive <b>graphical interface</b> to show the system's behavioural configuration (service by service). |
| R5 | The service shall periodically extract <b>metrics</b> from the marionette-enabled services and show them to the user.                              |
| R6 | The service should provide a way to <b>configure</b> the metrics to be extracted from the marionette-enabled microservices.                        |

Starting from the requirements, we moved on to the design of the **service components**. It was pretty clear from the beginning that the system needed four essential parts:

1. Component for **automatic discovery** of marionette-enabled microservices.

2. Component for **fetching behaviour configurations** from marionette nodes and building the internal representation of the whole system.
3. Component to hold the **representation of the system state**, to be exposed to the user through the graphical interface.
4. Component to perform runtime **metric collections** of the active marionette microservices.

Therefore, it was possible to create the component diagram that can be seen in Figure 3.1.

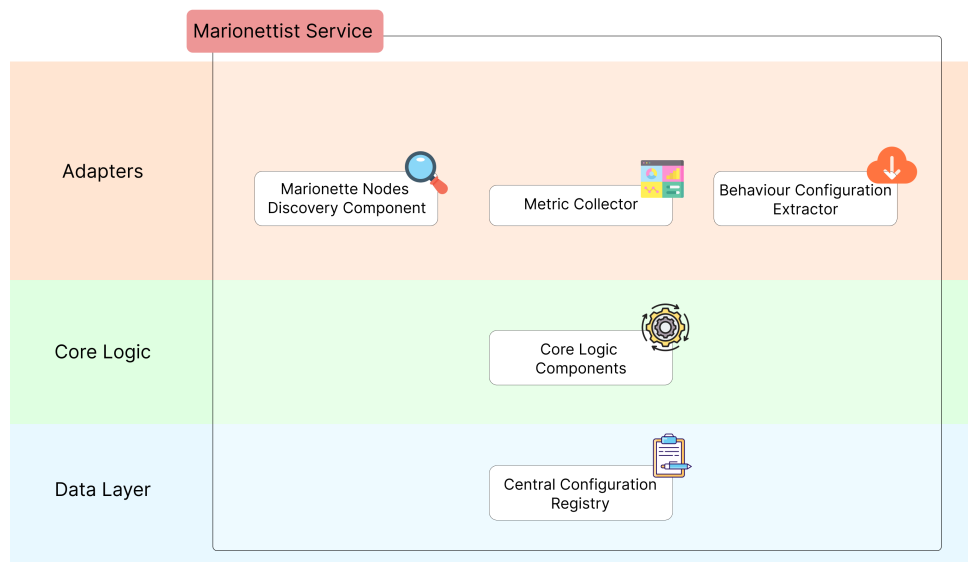


Figure 3.1: High-level view of the core components of the Marionettist service.

Starting from the bottom and working our way up, the first layer is the **data layer** (light blue), holding the central configuration registry that the Marionettist service uses as a storage for the current global system configuration. During system startup, the Marionettist collects all individual configurations from the marionette-enabled nodes and populates the central registry. Every time the user requires a modification, this is reflected in the registry for consistency.

The second layer (light green) is the **core** of the application, where the **business logic** lives. This layer was designed to separate the registry from the outermost components, which connect the application to external systems. The core logic encapsulates all the business rules and features that the service must implement as specified in the requirements. For instance, this layer validates requests coming from the user via the graphical interface, maps data structures from external representations to internal ones, and acts

as a bridge between the service’s data domain and the outside world.

Finally, the layer of **adapters** (orange) is where all the modules that connect the application business logic to **external components** sit. In particular, there is an adapter that handles the automatic discovery of Marionette nodes, a unit that collects runtime metrics, and one to fetch the runtime behavioural configurations from the detected Marionette services.

It is possible to notice the evident similarities between this architecture and the **clean architectural style** [16]. The initial high-level design effort aimed at clearly separating concerns, ensuring that the service remains both extensible and maintainable. Although the current implementation is tailored to run in a specific environment, as described in Section 3.2, the codebase has been structured to facilitate **future adaptations** and integration with new frameworks. Indeed, the product was designed to operate within a Kubernetes cluster with Prometheus, while intentionally leaving multiple points of extension available for further development.

## 3.2. Service Implementation

This section of the document details the **concrete implementation** of the Marionettist service, serving as a guide to the user for the current state of development and the available features.

### 3.2.1. A Clean Approach

During the implementation process, it became very clear from an early stage that it was necessary to **reduce the level of abstraction and generality** to make the service work in a specific environment. Indeed, numerous deployment options are available on the market, and developers have various ways to orchestrate the deployment of their microservice-based applications.

The goal of this project was to demonstrate that the framework and tools can effectively be used to enhance an existing project and **reduce the complexity of variability management**. Therefore, we restricted the domain of application to a specific environment, which is **Kubernetes** with **Prometheus**. This means that, as of now, the Marionettist service can integrate seamlessly into a deployment configuration that uses Kubernetes as the orchestrator and has a Prometheus instance active for metrics monitoring.

Nevertheless, the objective was also to structure the implementation in such a way as to make it **extensible** in the future and adaptable to a variety of other use cases. For

this reason, the **clean architectural approach** [16] seemed to be the best choice in this scenario. By structuring the codebase in a clean way, it was possible to completely separate the application core logic from the (superficial) details that integrate the core logic with external frameworks. In this way, the core logic remains stable, while the connectors can be **swapped easily** to change the external entities the system interacts with.

For example, the design of the central registry to store the current global system configuration, as well as all the classes that allow access to this registry, are part of the core logic and, as such, they will not change even if the Marionettist service is refactored to work with Docker Swarm <sup>1</sup> or even a simpler local testing environment without orchestration. On the other hand, the module that integrates the service with Prometheus is treated as a connector module in the project, which can be replaced with other types of metrics monitoring solutions.

Figure 3.2 shows a graphical representation of the main components of the architecture, divided into the typical layers of a clean architectural style.

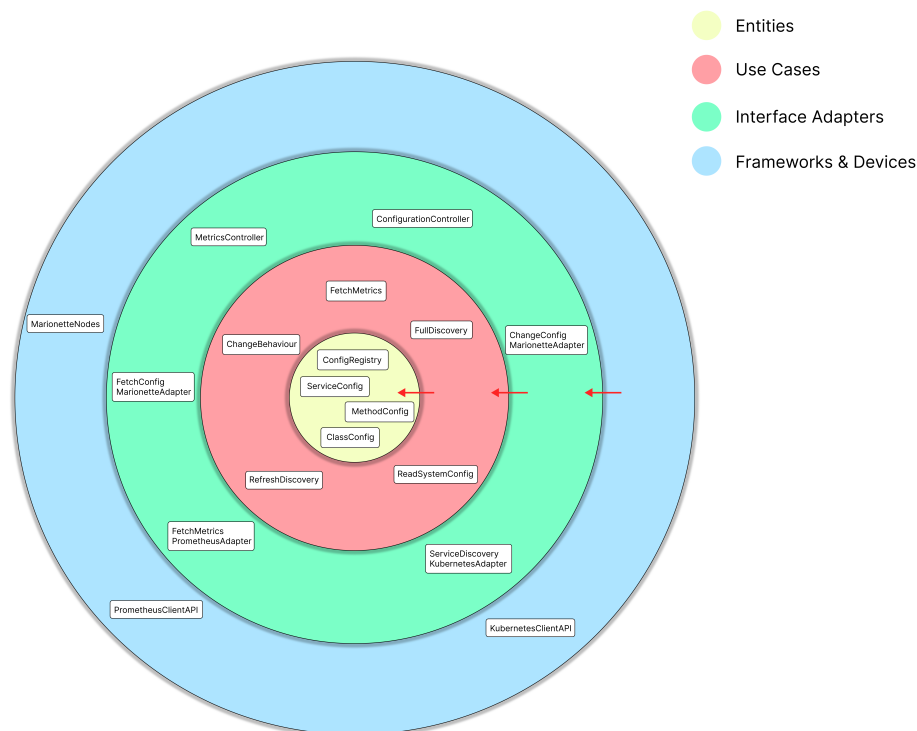


Figure 3.2: A graphical representation of the main components of the Marionettist service, divided into the typical layers of a clean architectural style.

<sup>1</sup>Docker Swarm, <https://docs.docker.com/engine/swarm/>.

It is possible to notice that the structure of the original theoretical design, shown in Figure 3.3, was maintained without many modifications. The **entities layer** (yellow) is made up of all the in-memory data structures that allow the Marionettist service to build a view of the current **runtime state** of the whole system. In particular, the `ConfigRegistry` is the central component that stores all configurations of marionette-enabled nodes.

The **use case layer** (light red) contains all the classes that offer the main **functionalities** of the system and enforce **business logic invariants**. It is possible to see in this circle components like the `FullDiscovery` use case that **detects** marionette-enabled nodes in the cluster, the `ReadSystemConfig` use case that provides **access to the data** in the `ConfigRegistry` by exporting the information in a user-friendly format, the `FetchMetrics` use case that **collects runtime metrics** of the observed marionette nodes and processes them and finally the `ChangeBehaviour` use cases which is employed every time the user requests a **change of behaviour** in any of the marionette nodes.

Moving outwards, the green circle represents the **interface adapters layer**. It is this layer that guarantees **extensibility** and **maintainability** of the codebase. Indeed, the interface adapters layer contains all the classes and components that connect the core application logic to specific **external frameworks**.

The `ServiceDiscoveryKubernetesAdapter` allows the Marionettist to connect to the Kubernetes client API for automatic service discovery. To be able to integrate with the web API exposed by Prometheus nodes to collect runtime metrics, the Marionettist service leverages the `FetchMetricsPrometheusAdapter`.

The **extensibility** of the system comes from the fact that if the Marionettist service had to be adjusted to work in a different environment, only the components in this layer of the architecture would be affected by the changes. For example, if the service needs to integrate into a Docker Swarm deployment setting, the `ServiceDiscoveryKubernetesAdapter` would fall short of this task, but it would be possible to create a new adapter, let us call it the `ServiceDiscoverySwarmAdapter`, and replace the Kubernetes one with it. The rest of the components would remain untouched.

### 3.2.2. Component Diagram

Figure 3.3 illustrates the UML component diagram of the Marionettist service.

At the centre of the figure, the **controller components** leverage the functionalities offered by different modules of the system to serve user requests. The diagram highlights **four main modules**. At the top-left, the `DiscoveryModule` is designed to discover



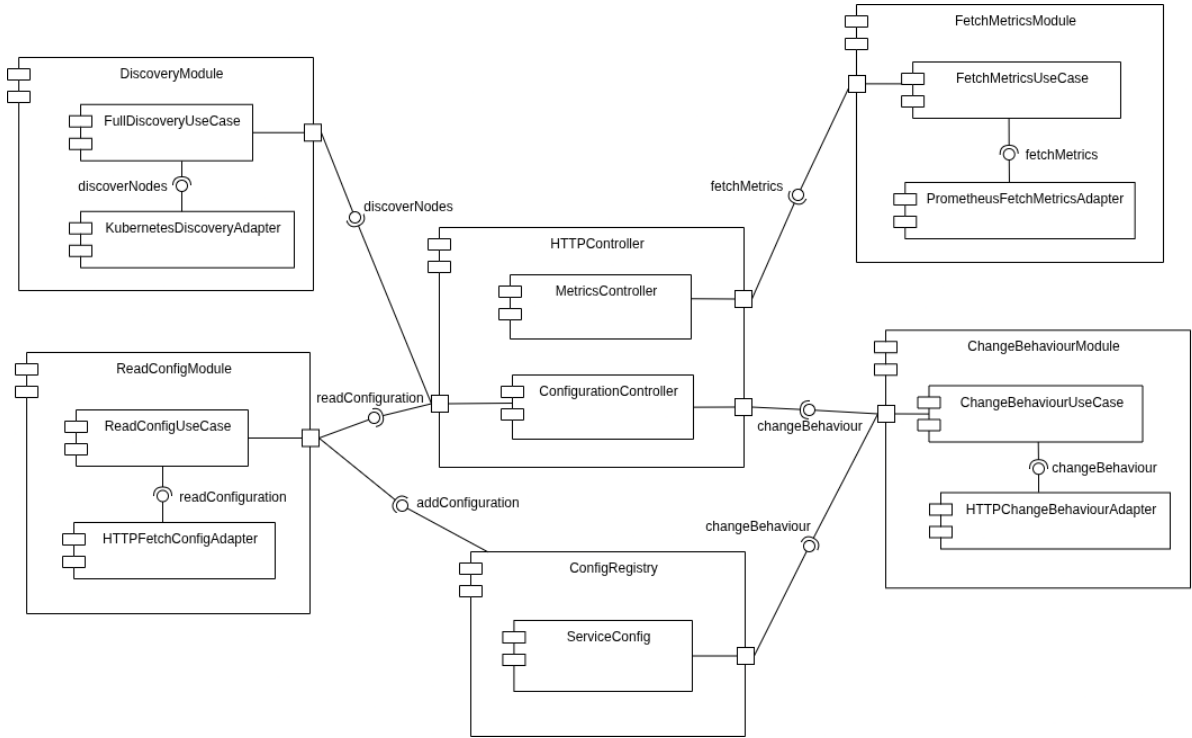


Figure 3.3: Component diagram detailing the main modules that make up the Marionettist service.

Marionette nodes in a Kubernetes cluster. The `ReadConfigModule` carries out HTTP requests to fetch the behavioural configurations of marionette nodes. At the top right, the `FetchMetricsModule` focuses on integrating the system with Prometheus for metric monitoring. Finally, the `ChangeBehaviourModule` enables the Marionettist service to contact marionette nodes whenever a behaviour change request arrives.

### 3.2.3. Interactions with External Components

We can now zoom out from the internal architectural view and see how the Marionettist service interacts with external components. In particular, the current implementation supports the deployment of the Marionettist service in a Kubernetes cluster with a Prometheus server collecting runtime metrics. In this way, the Marionettist can leverage the **Kubernetes client API** to discover automatically marionette-enabled services in the cluster, and the **Prometheus server** to query for runtime metrics.

Figure 3.4 illustrates the high-level deployment view of the architecture.

The yellow-ish circle on the left represents a **marionette-enabled application**, i.e., an application in which at least one of the microservices was instrumented with the

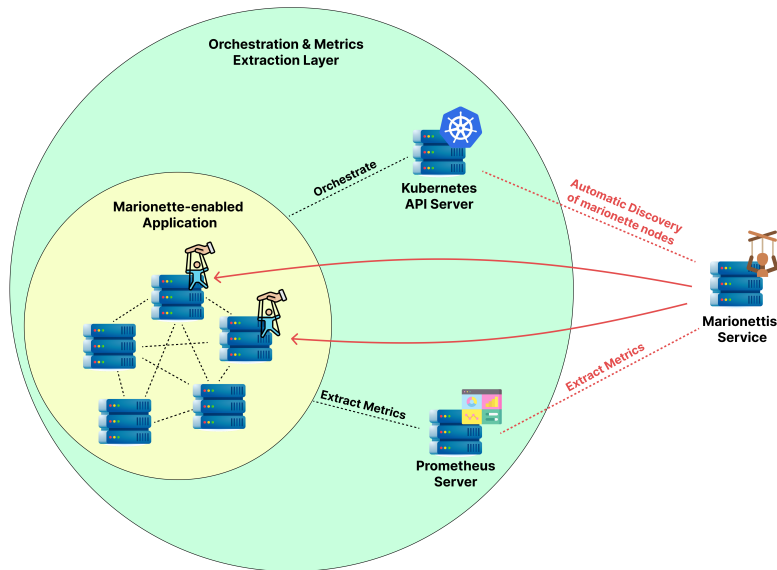


Figure 3.4: A deployment view of the full architecture. The Marionettist service integrates seamlessly with a Kubernetes environment and a Prometheus server for collecting runtime metrics.

Marionette tool to externalise control of behavioural variants. In the figure, it is possible to distinguish marionette-enabled services because they have a small marionette icon in the top-left corner.

The green-ish circle encompasses all the infrastructure that manages the microservice-based application from the outside. The Kubernetes API server allows external actors to query the **state of the deployment**, as well as to make changes to it through a web interface. The Prometheus node monitors the full system in action.

The Marionettist service **builds on top** of this complex infrastructure, as shown in the same Figure 3.4 on the right-hand side. It relies on the Kubernetes API server to query the **current state of the cluster** and discover marionette-enabled services. The solid red arrows at the centre of the figure show the successful discovery mechanism. The Marionettist service detects two marionette-enabled nodes, extracts from them their current behavioural configurations, and builds the graphical interface for the user. The Prometheus node is also leveraged to **monitor metrics at runtime** that explain how the application is handling the workload.

It is important to notice that the Marionettist service is an additional microservice that is itself deployed and **managed inside the Kubernetes cluster**. The separation in

circles shown in Figure 3.4 is just a logical separation that aims to draw the line between the already-existing services and the Marionettist for this explanation.

### 3.2.4. Metrics Configurability

The Marionettist service provides a simple system of **configuration** that enables the user to specify what metrics should be **tracked** for the detected marionette services. The configuration is achieved through **environment variables**, which are very easy to set up, especially when deploying the system in a Kubernetes environment. It is indeed possible to specify the environment variables directly in the **deployment file** in YAML, and they will be applied to the pod running the Marionettist service. We present an example of metric configuration in the YAML deployment file here.

```
- name: MARIONETTE_METRICS_QUERIES_JVM_MEMORY_DISPLAYNAME
  value: "JVM Heap Memory"
- name: MARIONETTE_METRICS_QUERIES_JVM_MEMORY_QUERY
  value: "jvm_memory_used_bytes{job=~\".*{service}.*\",area=\"heap\"}"
- name: MARIONETTE_METRICS_QUERIES_JVM_MEMORY_UNIT
  value: "bytes"
- name: MARIONETTE_METRICS_QUERIES_JVM_MEMORY_ENABLED
  value: "true"
```

The names given to the environment variables to configure a single metric follow a specific **pattern**. They should all start with `MARIONETTE_METRICS_QUERIES_`. Then comes the unique **key** of the metric, which will be used internally to separate this metric from all the other ones, so it should be unique. Finally, the name terminates with the **property** that is configured. In the first line, `DISPLAY_NAME` shows that the display name of the metric is being assigned. The most important property to define is the `QUERY` one, thanks to which the user can write the string that will be used to **query the Prometheus instance** in the cluster. The query supports a **placeholder** `{service}` to represent the name of the specific service for which the metric will be collected. This will be replaced at runtime, as the Marionettist might manage multiple services.

Another important piece of configuration that is necessary to set up to enable metrics extraction is the **URL** pointing to the Prometheus instance in the cluster. This can be done through the environment variable `PROMETHEUS_URL`.

```
- name: PROMETHEUS_URL
  value: "http://prometheus.monitoring.svc.cluster.local:9090"
```

### 3.2.5. The Graphical Interface and Available Functions

All the functions available in the Marionettist service are exposed to the user with a simple and **intuitive graphical user interface**, which facilitates the management of the marionette-enabled microservices. Figure 3.5 shows an example of how the interface looks when the user first lands on it.

The header section contains the title and on the right side two buttons: a green *Start System Test* button for initiating comprehensive **testing procedures**, and a blue *Full Discovery* button for triggering the **discovery process** of marionette-enabled services in the cluster. The functionality related to system testing is presented in Chapter 4. When the Marionettist service starts, it is configured to trigger a complete service discovery in the Kubernetes cluster automatically, so that the user can find the discovered services immediately at system startup. However, they can **refresh** the search at any time.

Below the header, a *Services Overview* section provides a comprehensive view of all the services discovered. The interface displays summary statistics at the top right, including the total number of services found, any unavailable services, and a timestamp indicating when the last discovery operation was performed.

Each discovered service is represented as a card-based component in which it is possible to see the service name as a prominent header and some service metrics like the number of classes that contain variable behaviours, the total number of methods with variants, and the number of methods whose behaviour has been changed from their default implementation via this interface. It is possible to click on any of the service cards to access **detailed service-specific** functionality.

When a user clicks on a specific service from the main dashboard, they are directed to a detailed **configuration view** that provides comprehensive management capabilities for that individual service, as shown in Figure 3.6. This interface serves as the primary workspace for configuring and monitoring the behavioural aspects of marionette-enabled methods within the selected service. Action buttons in the header provide quick access to essential operations: a *Reset to Template* function that restores **default behaviours**, and a *Show Metrics* option that displays **real-time performance data** for the service. The main content area is organised as a hierarchical structure. Each class within the service is presented as an expandable section that contains individual **methods with variants**. Each method is listed as a collapsible item that can be expanded to reveal its configuration options and displays its **current behavioural state**.

By clicking the *Show Metrics* button, the user triggers the **metrics collection func-**

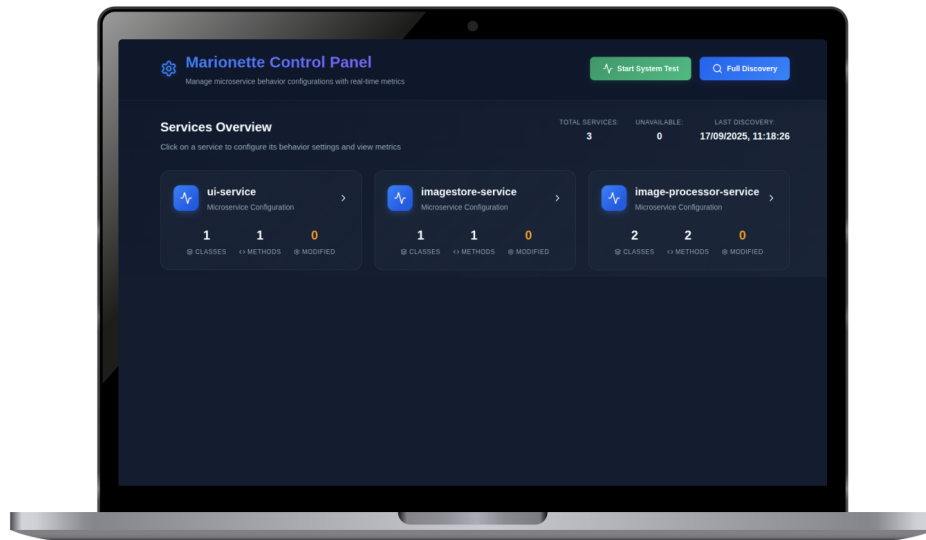


Figure 3.5: The home page of the Marionettist service graphical interface showing three discovered marionette-enabled services.

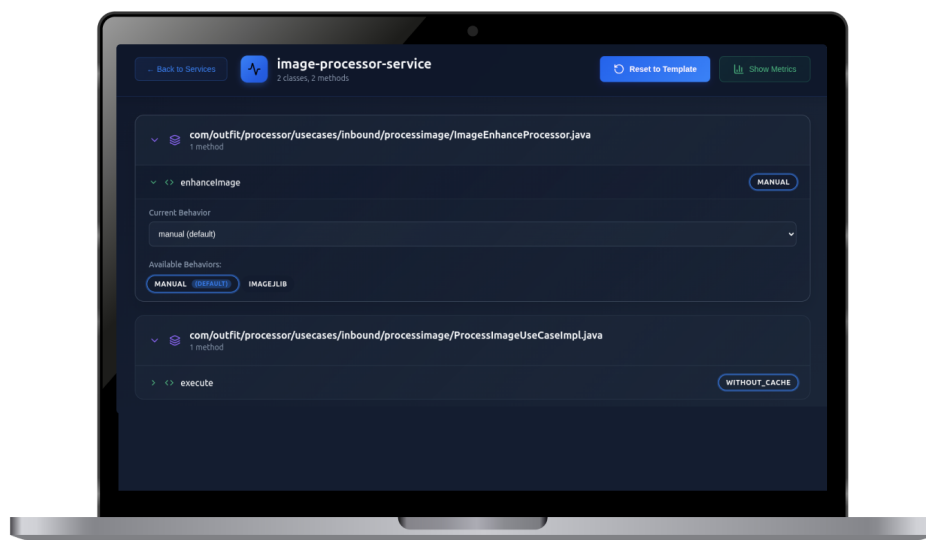


Figure 3.6: An example of a service detailed view of the Marionettist graphical interface showing two classes that expose variant behaviours.

**tionality** for the marionette-enabled service whose configuration is currently displayed in the graphical interface. Figure 3.7 shows how the metrics are displayed.

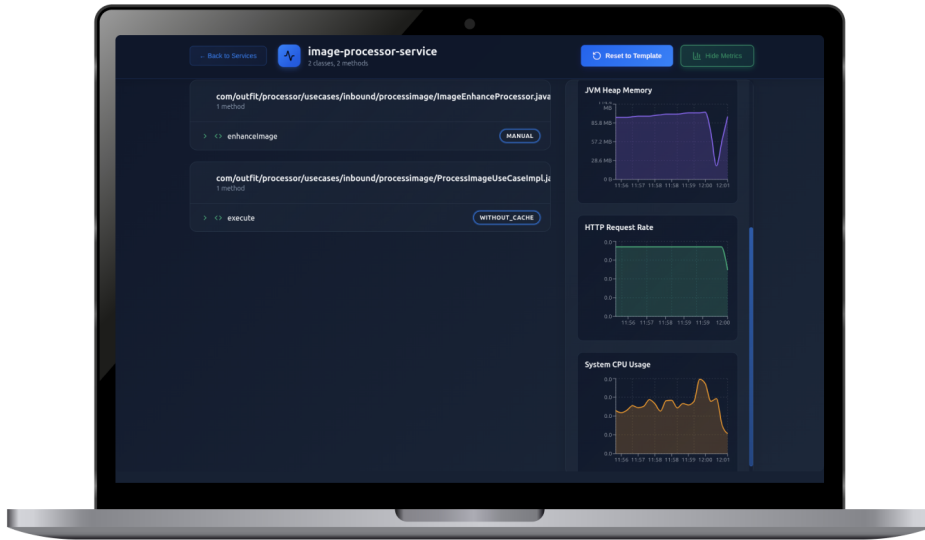


Figure 3.7: An example of how the graphical interface of the Marionettist service shows the user-defined metrics collected at runtime for a marionette-enabled service.

In this specific example, the user configured the Marionettist service to extract from Prometheus three metrics:

1. The JVM Heap Memory Consumption.
2. The HTTP Request Rate.
3. The System CPU Usage.

By having the metrics graphically visible on the same screen dedicated to the management of behavioural configuration, developers can see in **real-time** the **impact of behavioural changes** on the selected service.

## 4 | The Automated A/B/n Testing Pipeline

The Marionette tool presented in Chapter 2 and the Marionettist service illustrated in Chapter 3 form by themselves a **versatile framework** that can be applied to many **different domains**. Together, they allow developers to easily describe multiple behavioural variants within the codebase of a microservice, expose control of these variants through a web-based interface, and manage the entire system of variants via a centralised component, the Marionettist.

The possibility of having centralised control over the behaviour of individual microservices offers the basis for **many possible use cases**, from sustainability-driven goals to user-centric adaptation for better user experience and so on.

To demonstrate the potential of this infrastructure, we designed an **automated A/B/n testing pipeline** that builds on top of the Marionette tools. The idea of a testing framework comes quite naturally when we consider that the Marionette system is capable of changing the behaviour of a microservice at the method granularity at any point in time. Therefore, the intuition was to exploit this functionality to **test the marionette-enabled application with different behavioural variants activated**, collect metrics of interest, and finally select the behavioural configuration that best meets specific requirements and demands. Thus, we define this additional contribution as an **A/B/n testing pipeline** because multiple system configurations (more than two) are compared with one another. More specifically, the pipeline systematically explores **all possible combinations of behavioural variants** across multiple methods and services.

### 4.1. The Steps Making up the Pipeline

The A/B/n testing pipeline is developed as a module **inside the Marionettist service**. This architectural choice simplifies the **integration** between the testing orchestration and the behavioural configuration management, eliminating the need for external co-

ordination between separate systems. Since the Marionettist service already maintains complete knowledge of all available behavioural variants across all microservices, the testing pipeline can directly leverage this information without requiring additional discovery or configuration steps.

#### 4.1.1. Generation of System-level Configurations

The goal of the testing pipeline is to **assess how system metrics change** across different global system configurations. A global system configuration represents a specific assignment of behavioural variants to **every method** that has variants available across all microservices in the system.

The pipeline’s objective is to systematically **test each possible configuration**, measure **performance metrics** during execution, and identify which configurations deliver **optimal results** according to predefined criteria. Since each configuration is formed by selecting one behavioural variant for every instrumented method, the total number of possible configurations grows **combinatorially** with the number of methods and their available variants. For example, with ten methods, each one having three behavioural options, there would be  $3^{10} = 59,049$  possible system configurations to evaluate <sup>1</sup>.

Let us consider a **real example** to simplify the reasoning. A web-based application is built to solve mathematical equations and consists of two microservices. The first one handles building the graphical interface, while the second one calculates the results. Figure 4.1 shows the high-level architecture.

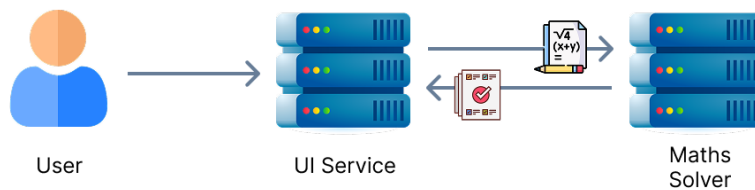


Figure 4.1: Example architecture of the mathematical equation solver web application.

The developer of the maths solver application employs the Marionette tool described in Chapter 2 and injects a behavioural variant in the *Math Solver* microservice—for instance, a different and presumably faster algorithm to solve second-order equations. From a

<sup>1</sup>This exponential growth of possible configurations is inherent to A/B/n testing, and it is up to developers to handle it appropriately. It is not a flaw of the Marionette framework.



global system perspective, there are two system configurations: one with the original implementation of the solver for second-order equations and one with the new version.

Then, the developer introduces a variant in the service, taking care of the graphical interface by implementing a different graphical interface that utilises a new frontend development framework. This variant increases the global system configurations to four, as each version of the graphical interface should be tested in combination with each variant of the equation solver. Hence, the configuration space grows **combinatorially** for all new points of variation added in the system.

Moving back to the main topic, the first step in the A/B/n testing pipeline is the **generation of all possible system-level configurations**. This allows the tool to have a comprehensive view of all the global behavioural configurations that the pipeline must test and compare. Once this generation process is complete, the test loop can begin.

#### 4.1.2. The Test Loop

The test loop iterates over each global configuration and always repeats the same steps:

1. Compare the configuration with the current state of the system to **find differences**.
2. **Apply changes** to the system behavioural configuration where needed to make the system comply with the configuration under test. This step can be carried out with the built-in functionalities of the Marionettist service.
3. **Wait** for the time slot allocated for a single configuration. The user defines the total time available to the test, and it is simply **split equally** among all configurations.
4. At the end of the time slot, **collect the metrics** that summarise and assess the current configuration's behaviour.

The cycle goes on until all configurations have been tested. A graphical representation of the steps of the loop can be found in Figure 4.2.

#### 4.1.3. The Uniform Traffic Assumption

The A/B/n testing pipeline operates under a key **architectural assumption**: it does not generate or control user traffic to the system under test. Instead, the pipeline focuses exclusively on configuration management and metric collection, while relying on **external traffic sources** to drive system activity.

This design decision stems from the recognition that different applications require vastly

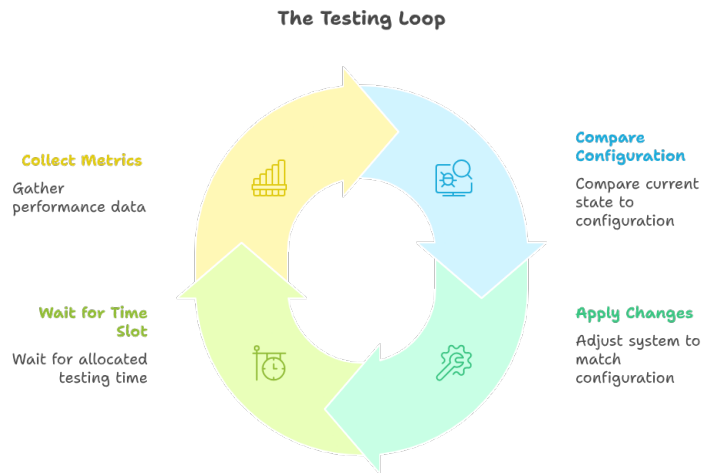


Figure 4.2: Graphical representation of the steps of the testing pipeline.

**different types of user interactions.** Rather than attempting to create a generic traffic generator that might not suit specific application needs, the framework delegates traffic generation to the system administrators or developers who understand their application’s usage patterns.

Traffic can originate from multiple sources depending on the testing scenario:

- **Production traffic:** Real user interactions when testing is performed on live systems.
- **Synthetic traffic:** Custom scripts or load testing tools that simulate realistic usage patterns.
- **Replay traffic:** Previously recorded user sessions replayed during testing.

However, it is important to note that the current implementation does not normalise collected metrics based on traffic characteristics or volume. The pipeline assumes that traffic patterns **remain consistent across all configuration test periods**. If traffic varies significantly between test intervals, the metric comparisons between configurations may not be meaningful. Therefore, ensuring uniform traffic distribution throughout the testing process is essential for obtaining reliable comparative results. This is particularly easy in the case of synthetic traffic generation. For real production traffic, a solution would be to extend the testing over a large period of time to average out the uneven distribution of traffic.

4.1.4. Final Ranking Generation

After all configurations have been tested and their metrics collected, the final step of the A/B/n testing pipeline involves generating a comprehensive **ranking** of all tested system configurations. This ranking enables users to identify which behavioural configuration combinations deliver the **best performance** according to the optimisation goals and metrics they configured the system to monitor during the test.

Since each configuration test involves multiple microservices, the collected metrics must first be **consolidated** to obtain **system-wide performance indicators**. Individual service metrics are aggregated using averaging to produce unified measurements that represent the overall system behaviour under each configuration. A visual example is offered in Figure 4.3. This aggregation step is essential because comparing configurations requires consistent, system-level metrics rather than disparate service-specific measurements.

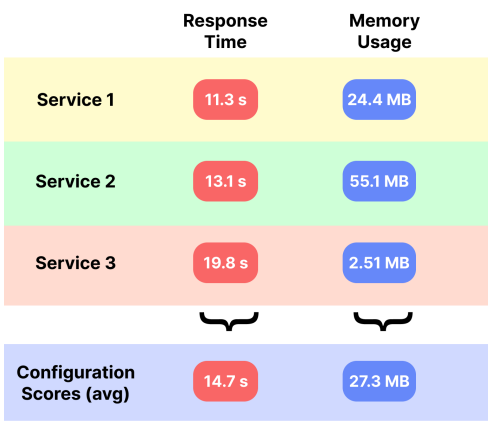


Figure 4.3: Visual aggregation of the metrics, from the service-level results to the system-level results.

The ranking mechanism employs a **lexicographic** comparison approach that enables **multi-criteria optimisation** with explicit **priority ordering**. Users define which performance metrics are most critical for their specific use case and specify whether higher or lower values indicate better performance for each metric. The **lexicographic strategy** compares configurations by first evaluating the highest-priority metric. Only when configurations perform identically on the primary metric does the algorithm consider the second-priority metric, and so forth. This approach ensures that the most important performance characteristics drive the ranking decisions while still considering secondary factors when primary metrics result in ties. Each metric includes a **optimisation direction** specification that indicates whether the goal is minimisation or maximisation. For instance, metrics like response time and error rates are typically minimised, while through-

put and success rates are maximised. The ranking algorithm automatically accounts for these different optimisation objectives.

The output of the ranking process is an ordered list of the **three most performant configurations**, where each one receives a numerical rank indicating its relative performance. The ranking includes both the position of each configuration and the specific metric values that determined its placement, providing transparency into the decision-making process. This detailed presentation allows users to understand not only which configurations performed best overall, but also how different behavioural variants impact specific aspects of system performance. Users can identify configurations that excel in particular areas or represent good **compromises** across multiple performance dimensions.

## 4.2. Concrete Implementation Details

This section provides additional insights into implementation-related aspects of the A/B/n pipeline and can also serve as a user guide to assist in setting up and utilising the tool. As discussed in Chapter 3, the Marionette framework was developed to integrate into a **Kubernetes orchestrated cluster** with a **Prometheus** server instance for metrics collection. The A/B/n testing pipeline builds on top of these tools, so it was implemented to achieve the desired functionality in this **same environment**.

### 4.2.1. Metrics Configuration

To make the testing pipeline adaptable to various types of systems, the configuration of the metrics to be monitored is completely customisable and left to the user. The strategy to achieve this configurability is very similar to the one introduced in Section 3.2.4 as it relies again on **environment variables** that must be set for the Marionettist service. An example of configuration is reported here for the CPU usage metric.

```
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_QUERY
  value: "avg(process_cpu_usage{job=~\".*{service}.*\"}) or vector(0)"
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_ORDER
  value: "3"
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_DIRECTION
  value: "lower"
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_DISPLAYNAME
  value: "Process CPU Usage"
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_UNIT
  value: "(ratio 0-1)"
- name: MARIONETTE_METRICS_CONFIG_CPU_USAGE_DESCRIPTION
```

```
value: "Average CPU usage ratio across all service instances (0.0 to 1.0)"
```

Multiple environment variables are used to configure a single metric, and the names of the variables follow a specific **pattern**. `MARIONETTE_METRICS_CONFIG_` is always the prefix. Then, the unique **key** that will be used to reference the metric inside the system is appended, in this case `CPU_USAGE`. Finally, the name ends with the **property** that is configured through the variable value. For instance, `DISPLAYNAME` to set up the name to be displayed when referring to this metric, the `UNIT` for the unit of measurement, and so on. There are two noticeable properties. The first one is the `DIRECTION` property, which can take on two values, *lower* or *higher*, and it represents the **direction of optimisation**. A metric with *lower* direction is considered to be more optimised if it assumes lower values and vice versa. Then, the `QUERY` property is used to allow the user to specify a custom **Prometheus query** to fetch the desired metric. To make the design flexible, the definition of the `QUERY` property supports two **placeholders**: `{service}` is used instead of the service name, which is replaced at runtime based on the monitored service, while `{timespan}` can be used as a placeholder to indicate the time slot dedicated to test a single configuration in the pipeline loop. This must also be replaced at runtime in the query because it depends on the total amount of time dedicated to the test and the number of global configurations.

Another important piece of configuration necessary for A/B/n testing is the **list of services to be monitored**. Indeed, in the microservice-based application under test, it is possible to have some microservices that are marionette-enabled and some that are not. The Marionettist automatically discovers the marionette nodes, but the other ones remain **out of the configuration space**. When it comes to synthesising global system configurations, however, all the services that make up the application are relevant. Therefore, the set of microservices to be observed is also exposed as a user-customizable configuration.

```
- name: MARIONETTE_METRICS_INCLUDED_SERVICES
value: "image-processor-service, ui-service"
```

#### 4.2.2. Initiating the Test

To initiate an A/B/n test, the user can exploit the graphical interface offered by the Marionettist service. As mentioned in Section 3.2.5, the home page of the graphical interface presents on the top right a button called *Start System Test*. By clicking on it, the user is prompted with a dialog to set up the **total duration** of the test, as shown in

Figure 4.4.

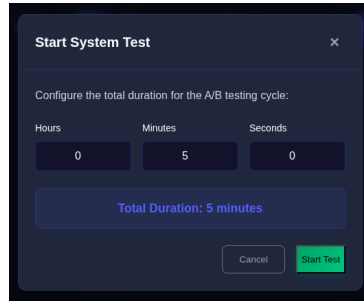


Figure 4.4: The graphical user dialog used to set up the total amount of time for the A/B/n test.

### 4.2.3. High-level View of Components and Interactions During Test

During the testing phase, the components that interact in the system are almost the same as the ones illustrated in Figure 3.4. The only difference is that while the testing loop is going on, the Marionettist service is **periodically changing the behavioural configuration** of the system under test to traverse all possible global configurations.

Another important consideration is that the Marionettist service expects some **traffic** flowing through the microservice-based application to be able to extract relevant metrics and **make comparisons** among configurations. Therefore, a more comprehensive view of all the components involved in the architecture while executing the A/B/n testing pipeline is offered in Figure 4.5.

Compared to the components illustrated in Chapter 3, we can see on the left-hand side that a **group of users** is connecting and generating traffic on the application under test. This traffic can be **synthetic** or **real** traffic, depending on the use cases. Furthermore, the visual representation highlights that the Marionettist service on the right-hand side undergoes a cycle for each configuration, as previously described in Section 4.1.2.

### 4.2.4. Retrieving the Results

When the test ends, the Marionettist service **generates the final ranking** of the best performing system configurations, as described in Section 4.1.4. It is possible to download a **JSON file** containing the full ranking and measurements through a button that appears in the graphical interface after the end of the testing period, labelled *Download Last Test Results*.

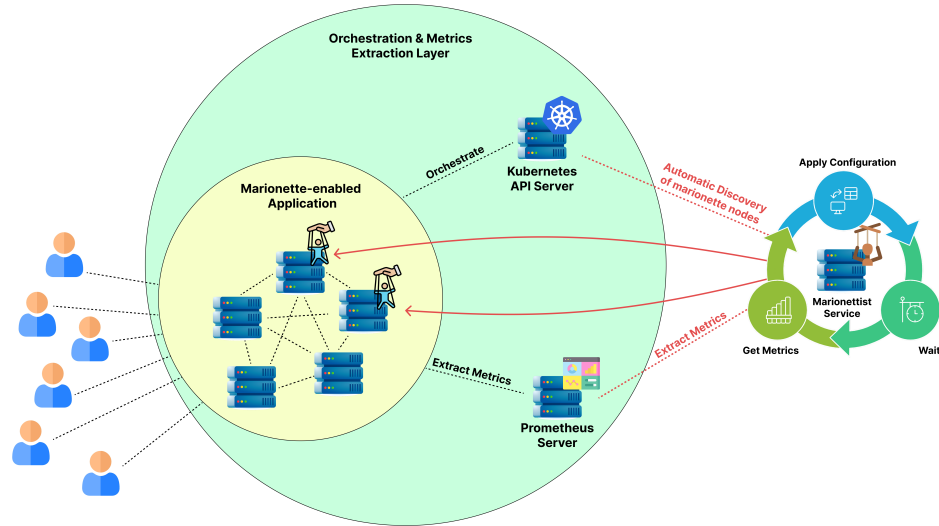


Figure 4.5: High-level view of the components interacting when initiating an A/B/n testing phase.

### 4.3. How to Analyse the Results

Analysing the results of the experiment by reading the JSON file directly can be pretty cumbersome, making it difficult to find correlations in the dataset. Therefore, the Marionette ecosystem comes with a simple **Python script** that can read the JSON configuration returned by the Marionettist service and generate a set of useful **plots** to compare metrics, services, and global system configurations effectively. The set of generated plots is made up of the following representations:

**Radar Chart & Multi-line Chart for System Configuration Comparisons** The radar chart and multi-line chart offer a **high-level comparison** of all the **system-level configurations**. They are the quickest way to analyse the major differences between behavioural configurations at a coarse granularity.

The **radar chart** shows a comprehensive multi-dimensional performance comparison. An example can be found in Figure 4.6. Each system configuration is represented by a **different coloured polygon**. The metrics are normalised to a 0-1 scale, with 1 (outer edge) representing the best performance. It has the purpose of quickly identifying which system configurations excel in which areas and overall performance patterns.

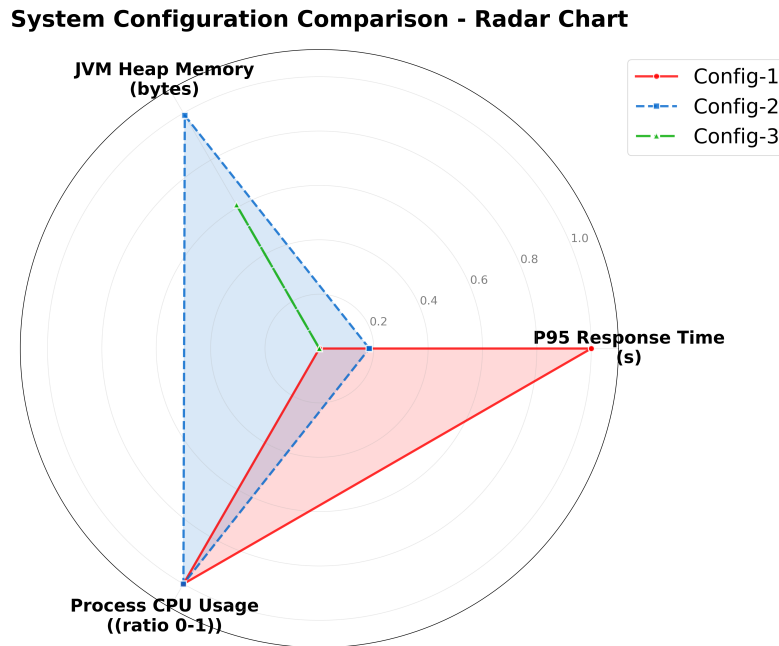


Figure 4.6: Example of a radar chart being produced at the end of A/B/n testing.

The **multi-line chart** represents individual metrics comparison across the different system configurations, as it is possible to see in Figure 4.7. On the x-axis, the selected metrics appear. Each system configuration takes a different colour to differentiate. As for the radar chart, the metrics are normalised to a 0-1 scale, with 1 (higher edge) representing the best performance.

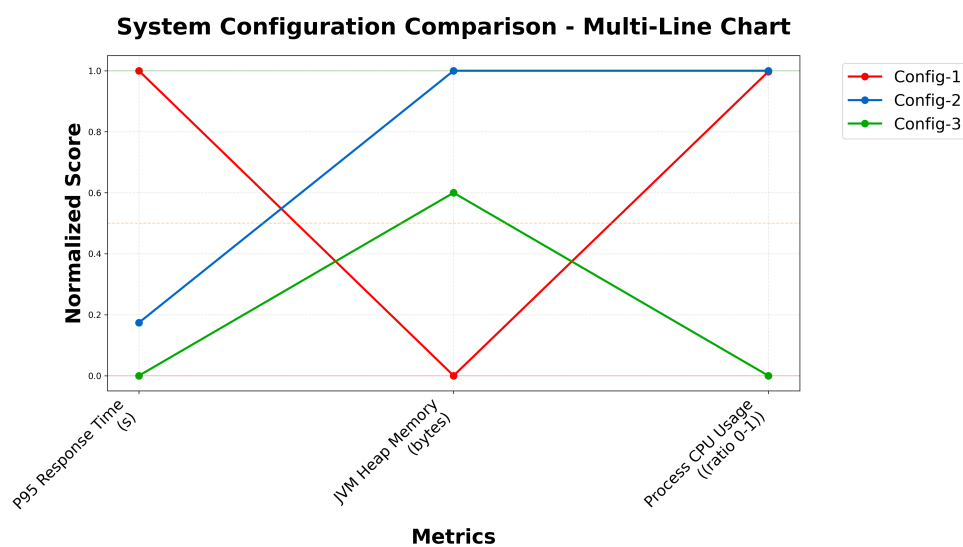


Figure 4.7: Example of a multi-line chart being produced at the end of A/B/n testing.

To interpret these plots correctly, it is essential to understand that the displayed values



are **normalised** to enable **comparison across different types of metrics**, which can have vastly different scales and units, so direct comparison would be meaningless.

The normalisation process involves two steps:

1. **Min-max scaling**: This allows for rescaling each metric to a range between 0 and 1.
2. **Direction alignment**: Since some metrics are “better” when lower (response time, memory usage) and others when higher (availability), all metrics are **aligned** so that higher normalised values consistently represent better performance. For metrics where lower values are preferable, the normalised values are inverted using:

$$aligned\_value = 1 - normalized\_value$$

As a result, in all plots, **values closer to 1.0 indicate better performance**, while **values closer to 0.0 indicate worse performance**, regardless of the original metric type.

**Single-metric Line Chart & Bar Plot** The single-metric line charts, and their equivalent bar plot, can be used to analyse **each metric trend** separately at the **system-level granularity**. In these plots, there is a sub-plot for each metric. Inside the plot dedicated to a single metric, the x-axis displays the list of system configurations, and the y-axis shows the values they scored for the metric under analysis. Figure 4.8 shows an example of a line chart.

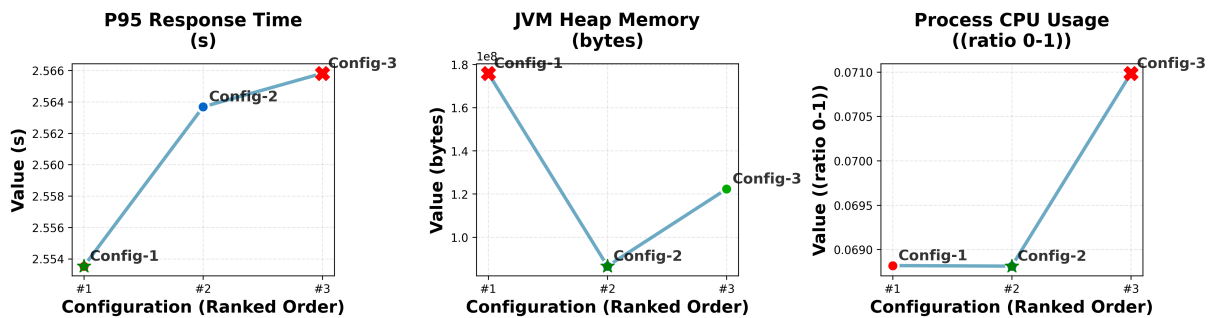


Figure 4.8: Example of a single metric line chart being produced at the end of A/B/n testing.

An equivalent representation can be offered by the bar plot, as illustrated in Figure 4.9.

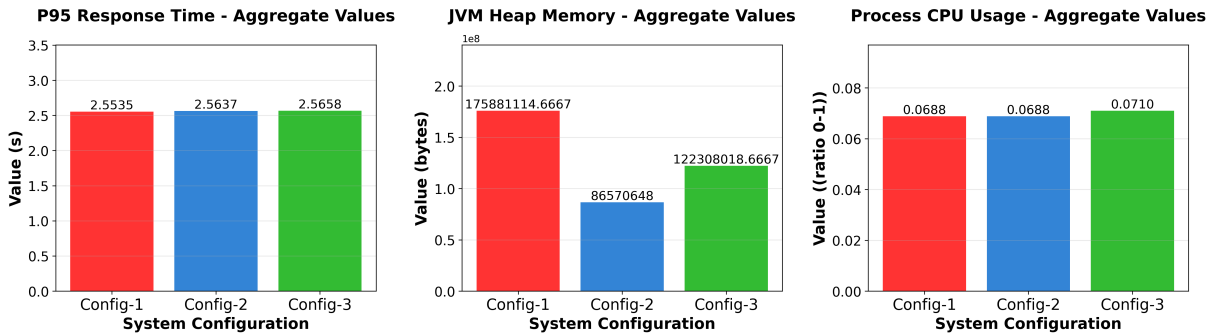


Figure 4.9: Example of a single metric bar chart being produced at the end of A/B/n testing.

**Service-level Bar Plots** The service level bar plots are useful to break down the analysis at the **level of single services**. Indeed, the final aggregate values used to compare system-level configurations are obtained by averaging the values of the services that comprise the application. This plot enables the user to identify which services in each winning configuration were crucial for the configuration to outperform the others.

The plot is divided into **sub-plots**, one for each metric. Focusing on a single subplot, on the x-axis, it is possible to see the list of services that make up the application. For each service, there is a number of bars going up equal to the number of system-level configurations being compared (the top-performing ones). Each bar represents the value that the service scored for that metric in that specific system-level configuration. Figure 4.10 illustrates an example of this plot type.

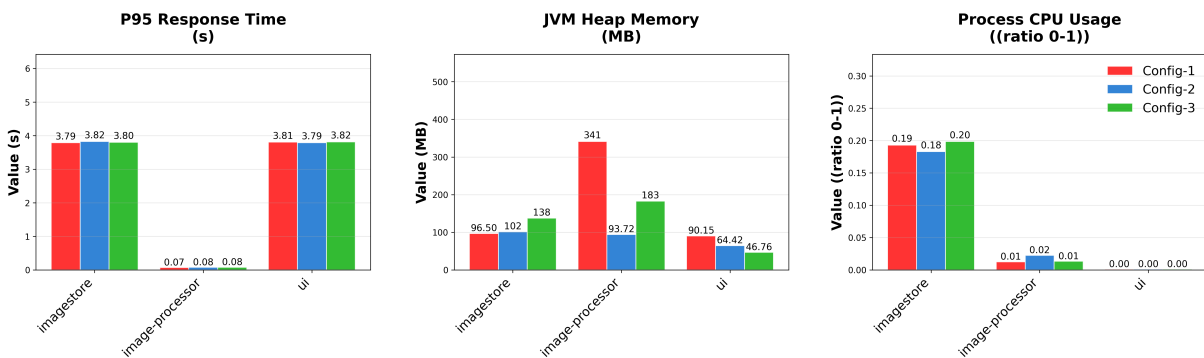


Figure 4.10: Example of a service-level bar plot being produced at the end of A/B/n testing.

## 5 | Experimental Results

Experimentation is a critical step in validating the **practical viability** of the Marionette behaviour injection framework. While theoretical analysis and architectural design provide the foundation for understanding system capabilities, empirical evaluation is essential to demonstrate **real-world applicability**.

The primary objectives of this experimental evaluation are structured as three core research questions.

1. **RQ1:** what is the applicability of Marionette to operate **fine-grained runtime behaviour modification** to microservices systems?
2. **RQ2:** What is the **performance overhead** introduced by the Marionette framework?
3. **RQ3:** To what extent does the Marionette framework reduce **developer workload**?

This chapter presents the experimental setup, details of the test environment implementation, quantitative results, and an analysis of the findings. The results demonstrate the practical feasibility of the Marionette approach while identifying specific scenarios where performance considerations may influence deployment decisions.

### 5.1. The Test Subject

The *Outfit App* is a relatively simple, microservice-based application developed as part of this project to provide **full control** over the test subject and to instrument it effectively. The primary goal of the application is to showcase a gallery of potential **outfit ideas**. Users can interact with the system by uploading an image of their favourite outfit or by scrolling through the gallery of pictures to get inspired by other people's ideas. It is made up of three microservices. The *Image Store* service is responsible for storing images locally when users upload them. The *Image Processor* service extracts images from the *Image Store* whenever requested and processes them to apply enhancements and improve their

appearance on the graphical interface. The *UI* service builds the graphical interface, and it leverages the *Image Processor* service to enhance images on the fly. Figure 5.1 shows the graphical layout of the application.

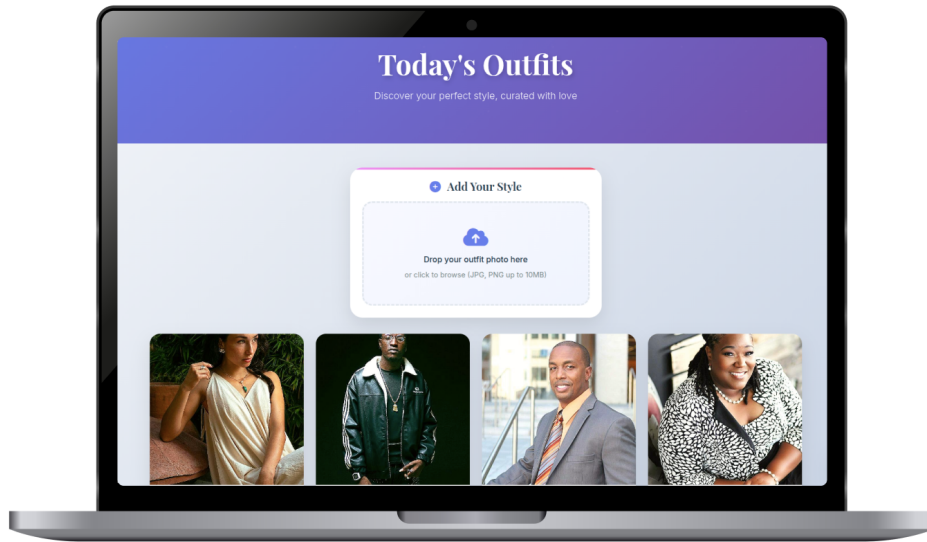


Figure 5.1: The home page of the *Outfit App* showing four outfit examples.

From an architectural standpoint, Figure 5.2 shows the three microservices and their interactions.

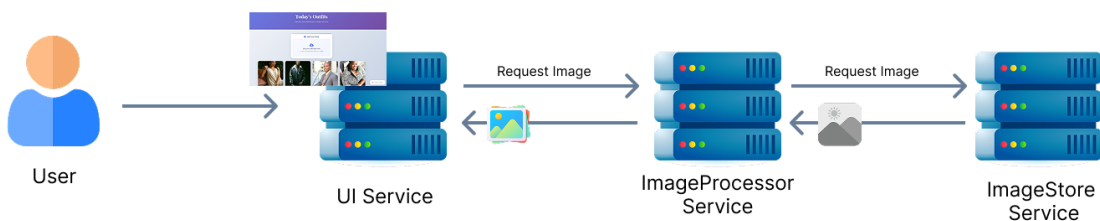


Figure 5.2: The component diagram of the *Outfit App* showing the interactions between the three microservices

The system works in a **pipeline**. When the user asks for an image, the *UI* service contacts the *Image Processor* service, which in turn extracts the raw image from the *Image Store* service, processes it, and returns it to the *UI* service. The *Outfit App* was developed with **Java Spring**, as it was the target framework for the Marionette tool, as described in Chapter 2.

## 5.2. Single Variation Point Experiment

The first part of the experimentation focuses on employing the Marionette framework in a real microservice-based application to demonstrate the **feasibility** of the tool (RQ1), as well as the possibility of performing **effective A/B/n testing** on the system. The application that is used as a test subject is the previously mentioned *Outfit App*. This first experiment was designed to be very **simple** to showcase the tools and their correct integration. Therefore, **one** single **variation point** was defined in the system, meaning a single variant implementation for a method was used.

### 5.2.1. Instrumentation of the Test Subject

It was first necessary to identify in the application one microservice that could support a variation of behaviour relevant to testing. We decided on the *Image Store* service and injected a variation point that changed the way images uploaded by users were saved. The default implementation used a **compression algorithm** to greatly reduce the size of the files and save on network bandwidth. The behavioural variant modified a single method to prevent any compression on the images entering the service.

In this way, the A/B/n testing pipeline could show the **impact** that compression had on the application metrics. The Marionette tool was used to generate the new version of the *Image Store* microservice, and this latest version **replaced** the previous one without behavioural variants in the deployment process.

### 5.2.2. Metrics Configuration

As for the **metrics configuration** of the A/B/n test, we selected three fundamental metrics to be monitored, which are reported in Table 5.1.

Table 5.1: Metrics configuration for performance monitoring.

| Display Name      | Unit        | Description                                                         |
|-------------------|-------------|---------------------------------------------------------------------|
| P95 Response Time | s           | 95th percentile response time averaged across all service instances |
| JVM Heap Memory   | bytes       | Total heap memory used across all service instances                 |
| Process CPU Usage | (ratio 0-1) | Average CPU usage ratio across all service instances (0.0 to 1.0)   |

As illustrated in Section 4.2, the metrics are extracted from the **Prometheus** instance

running in the **Kubernetes** cluster, which in turn fetches them from individual services of the *Outfit App*.

### 5.2.3. Local Deployment Setup

As described in Chapter 4, the system was deployed in a Kubernetes-managed cluster in which a Prometheus instance was active to extract runtime service metrics. In particular, the services were deployed locally, using Minikube on a laptop with the following specifications:

- **Processor:** Intel Core i7-1165G7 (4 cores, 8 threads) @ 2.80 GHz
- **Memory:** 8 GB DDR4-3200 RAM ( $2 \times 4$  GB modules)
- **Storage:** 476.9 GB NVMe SSD
- **Operating System:** Ubuntu 24.04.2 LTS (kernel 6.11.0-17-generic)
- **Container Runtime:** Docker Engine 27.5.1 with containerd 1.7.25
- **Minikube:** v1.36.0 with 3 CPU cores, 4 GB RAM, and 30 GB disk allocated
- **Kubernetes:** v1.34.1 (client version)

The **experimental testbed** comprises three primary components: the target **microservice application** (*Outfit App*), the **Marionettist service**, and the metrics observability stack (**Prometheus**).

### 5.2.4. Generation of Synthetic User Traffic

To simulate user interactions with the *Outfit App*, a simple **Python script** was implemented. It was designed to be configurable through the command-line launcher. It allows the user to specify:

- The **number of users** to simulate. These will be treated as concurrent threads in the script.
- The **cycle duration**. This is the amount of time (in seconds) dedicated to each system configuration. At the end of a cycle, the script takes a brief break to allow the application to stabilise on the new configuration.

When the script starts, the generated threads enter a loop in which they either request a specific page of the image gallery from the *UI* service (containing images) or upload an image of an outfit. The probability was set at 35% for image uploading and 65% for gallery

viewing. To make the traffic as **uniform** as possible among different configurations, only **one image** was used for uploading. Thus, all threads would always upload the same image throughout the test. This avoids uneven distribution of traffic or computation caused by the random choice of images of **different sizes** from the candidate set.

For this experiment, the script was run with **two users** and a **cycle duration of an hour**, aligning it with the timings of the A/B/n testing pipeline.

### 5.2.5. Execution and Analysis of Results

The A/B/n test was set up to run for **two hours**. Since there was a single variation point in the whole system with two alternatives, there were **two possible global system configurations**: one with image compression enabled and the other without image compression. Each system configuration was executed for one hour, and at the end of each cycle, the Prometheus server was queried to aggregate metrics and draw conclusions.

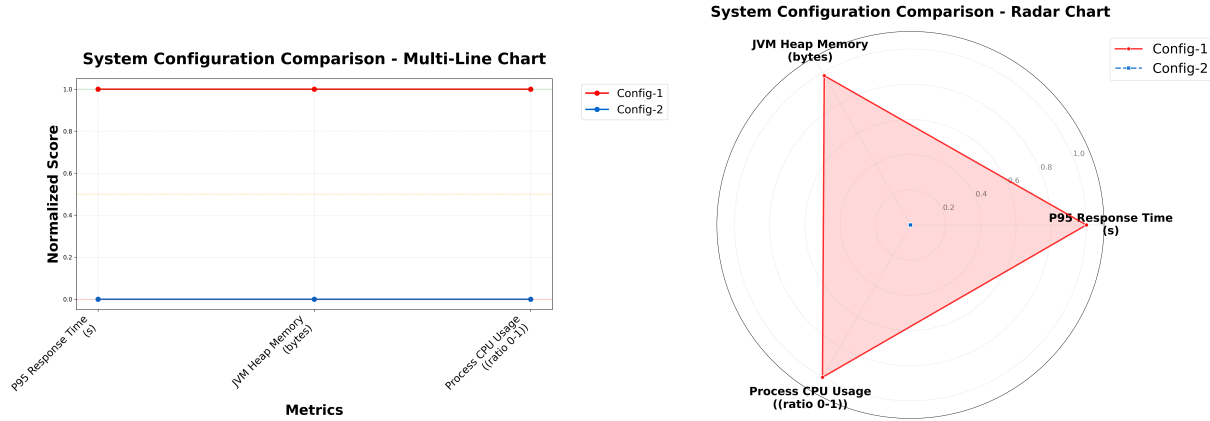
We present here the **results** of this first experiment by showing the plots generated by the Python visualiser script provided as part of the Marionette framework, which works on the data set produced by the tests, as described in Section 4.3.

First off, to understand the representations in the plots, it is important to remark that among the two configurations proposed, the one that best performed was the one **with compression enabled**. This is what we expected, as reducing the image size might increase the computational expense at the *Image Store* node; however, it is extremely beneficial for the entire image transfer process that must pass through the *Image Processor* service and the *UI* service.

**System Configurations Comparison with Line and Radar Charts** The two plots represented in Figure 5.3 illustrate a high-level comparison between the **global system configurations**.

As explained in Section 4.3, to interpret these plots, it is essential to understand that the displayed values represent **normalised measurements**, which enable comparison across different types of metrics. Since response time (in seconds), memory usage (in bytes), and CPU usage (as a ratio) operate on completely different scales, they are normalised to a common scale. Moreover, they are aligned in direction, meaning that values closer to one are always considered "better".

As a direct consequence of the normalisation applied to the metrics, the plots in Figure 5.3 demonstrate **clear performance differences** between the two system configurations.



(a) Multi-line chart to show system-level comparison of the simple experiment.

(b) Radar chart to show system-level comparison for the simple experiment.

Figure 5.3: System-level configurations comparison, shown with two different types of charts.

The **line plot** on the left shows that Configuration-1 (red line) achieves **superior performance across all three metrics** compared to Configuration-2 (blue line). Since the experiment evaluated only two system configurations, the normalisation process produces a binary outcome where one configuration scores 1.0 (optimal) and the other scores 0.0 (worst) for each metric. This polarisation is a mathematical consequence of min-max scaling applied to a two-element dataset, where the better-performing configuration receives the maximum normalised score and the worse-performing configuration receives the minimum.

The **radar chart** presents the same normalised data using a polar coordinate system, where values closer to the outer edge represent better performance. Configuration-1 forms a triangle that extends to the chart's perimeter across all three metrics, while Configuration-2 appears as a single point at the chart's centre because all its normalised values are 0.0. This visualisation effectively illustrates the **performance dominance** of Configuration-1, which demonstrates lower response times, reduced memory consumption, and decreased CPU utilisation compared to Configuration-2.

While these plots provide a **high-level view** of the global configurations comparison, they might result in too **abstract** a view to make practical conclusions on how the whole application reacted when a certain behavioural configuration was applied. For this reason, more specific plots are offered.

**Service-level Comparison with Bar Plot** Figure 5.4 breaks down the comparison into **service-level metrics**, so that it is possible to make additional considerations on



the results and draw connections between service behaviours.

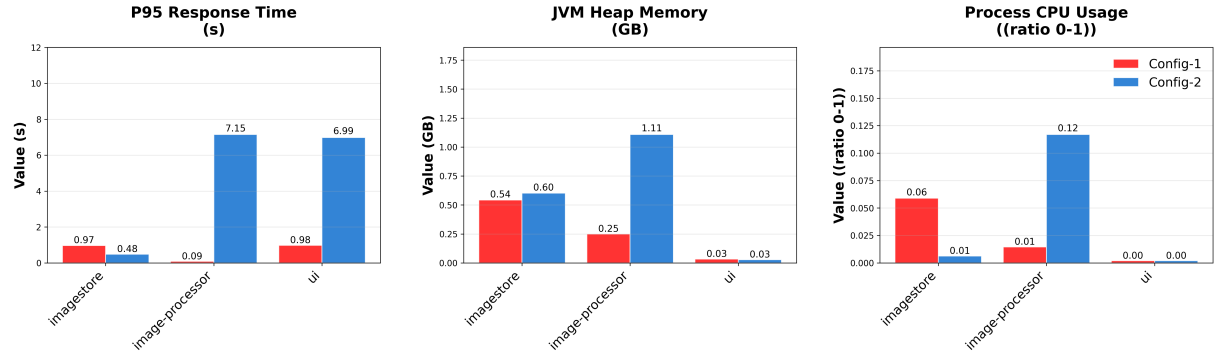


Figure 5.4: Service-level comparison of the behavioural configurations for the simple experiment

The illustration presents a separate plot **for each metric**. Looking at a single sub-plot, it is possible to see on the x-axis the list of microservices making up the *Outfit App*. For each service, there is a number of bars equal to the number of system configurations tested. Each bar therefore represents the **value for a specific metric of one of the services**, when the system applied a defined behavioural configuration. In simpler terms, all red bars represent results when the system had image compression enabled (configuration 1), while the blue bars show the results for the system without image compression (configuration 2).

It is possible to make **more insightful conclusions** on the impact of the compression strategy by looking at these plots. First off, the **response time plot** at the top right shows a clear trend: enabling compression (red) **increases** the response time of the *Image Store* service, while it greatly **reduces** the response time of the other services in the cluster. This is very aligned with the original expectations. Indeed, the response time at the *Image Store* node increases in the case of compression because additional computation is required when users upload images. On the contrary, applying compression significantly reduces the workload on other services, as well as saving a substantial amount of bandwidth when transferring images over the network.

The plot focusing on the **JVM heap memory** shows a huge **spike** in memory usage at the *Image Processor* service if compression is **disabled**. Again, this is quite reasonable as the service must work on larger images and apply pixel-wide enhancing operations.

The **CPU usage plot** aligns perfectly with the rest of the representation. Applying compression **increases** the CPU usage observed at the *Image Store* node, while it significantly **reduces** the same metric on the *Image Processor* node. Disabling compression

has the opposite effect: it reduces CPU usage at the *Image Store* node, but it increases it at the *Image Processor* node.

#### 5.2.6. Summary of RQ1 (Single Variation Point)

This initial experiment successfully demonstrated the **practical viability** of the Marionette framework for A/B/n testing in microservice environments. The framework proved capable of **injecting behavioural variations** without requiring code redeployment, while the comprehensive metrics pipeline provided clear insights into **performance trade-offs**. The image compression experiment revealed meaningful system-wide impacts: enabling compression increased CPU usage at the Image Store service but dramatically reduced resource consumption at downstream services. This demonstrates the value of observing how behavioural changes **propagate** through entire service chains rather than examining services in isolation. Results aligned with theoretical expectations, validating both the experimental methodology and measurement infrastructure. This experiment establishes a repeatable process for evaluating behavioural alternatives in microservice-based applications. The results encourage **expanded experimentation** with more complex behavioural variations and larger configuration spaces.

### 5.3. Multi-Variation Point Experiment

With the foundations of the simple experiment illustrated in Section 5.2, it was possible to set up a more complex test in which behavioural variants were injected **in all the microservices** making up the *Outfit App*, creating a total of **16** system-level configurations to compare <sup>1</sup>.

#### 5.3.1. Instrumentation of Test Subject

The three microservices of the *Outfit App* were all instrumented in this test.

The *Image Store* service was left untouched with respect to the case illustrated in Section 5.2, so it still exposes a variant that applies image compression and the other one without any form of compression.

The *Image Processor* service was enriched with **two variation points**. The first one is related to the **image enhancement algorithm**. The default implementation employed a manual approach, in which image pixels were manipulated one by one using plain Java code. The introduced variant achieved the same result, but using a third-party library called *ImageJ*, which specialises in image processing. Then, the second variation point was about the implementation of an **in-memory cache** for recently-processed images. The original implementation of the *Image Processor* service constantly re-fetches images from the *Image Store* service for each new request coming in. This means that, regardless of which images users are requesting, the processor service would still repeat the steps of getting the images from the *Image Store* and reprocessing them. The introduction of a cache with a maximum capacity of fifty images could help the service speed up the computation. The cache can contain up to fifty images already enhanced that can be returned immediately to the *UI* service, without any interaction with the *Image Store* service. The second variation point in the *Image Processor* service consists of **enabling or disabling the cache** feature.

Finally, a behavioural variant was also introduced in the *UI* service. This service is capable of building internally **two types of graphical interfaces**. The default one is the one shown in Figure 5.1. The added variant is a great simplification of the default version. It is not as colourful and disables most of the visual effects offered by the original implementation. Figure 5.5 shows what the simplified graphical interface looks like.

In total, there are four methods with behavioural variations, and each method presents two

---

<sup>1</sup>Replication package: <https://github.com/sissamatteo29/MarionetteReplicationPackage>

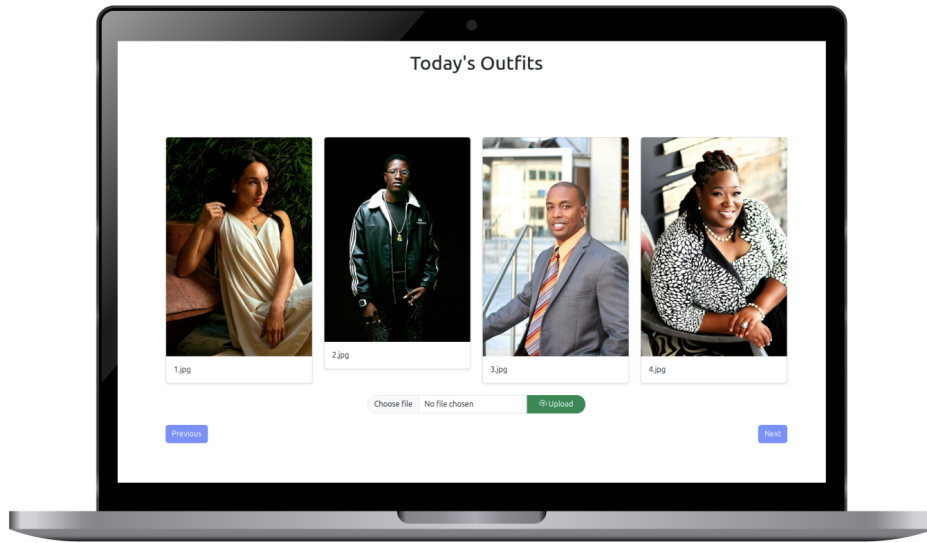


Figure 5.5: Simplified graphical interface of the *Outfit App*.

variants. This means that the total number of global system configurations is calculated as  $2^4 = 16$ .

Table 5.2 shows a summary of the behavioural variants.

Table 5.2: Summary of behavioural variants in the Outfit App microservices.

| Service         | Behavioural Aspect    | Available Options                     |
|-----------------|-----------------------|---------------------------------------|
| Image Store     | Image compression     | compression (default), no compression |
| Image Processor | Enhancement algorithm | manual (default), ImageJ library      |
|                 | Result caching        | without cache (default), with cache   |
| UI Service      | Interface complexity  | beautiful UI (default), simple UI     |

After defining the variants in the codebase, all microservices of the *Outfit App* were **transformed** with the use of the Marionette tool, and these new versions of marionette-enabled microservices replaced the previous ones in the deployment process.

### 5.3.2. Metrics Configuration

The configuration of the metrics to be monitored during the A/B/n test is **completely equivalent** to the one shown in the simpler experiment in Section 5.2.2. Table 5.3 reports the metrics list for clarity.

Table 5.3: Metrics configuration for performance monitoring

| Display Name      | Unit        | Description                                                         |
|-------------------|-------------|---------------------------------------------------------------------|
| P95 Response Time | s           | 95th percentile response time averaged across all service instances |
| JVM Heap Memory   | bytes       | Total heap memory used across all service instances                 |
| Process CPU Usage | (ratio 0-1) | Average CPU usage ratio across all service instances (0.0 to 1.0)   |

### 5.3.3. Local Deployment Setup

The deployment setup for this complex experiment follows the same **configuration** and **technological support** as described in Section 5.2.3. The microservices of the *Outfit App*, as well as the Marionettist service, were deployed in a Kubernetes cluster hosted locally by a Minikube instance. Prometheus was also present for metric monitoring.

The experiment was initiated with a **total duration of eight hours**, meaning that each global configuration was tested for **thirty minutes**, since there are sixteen configurations overall.

### 5.3.4. Generation of Synthetic User Traffic

Thanks to its **flexible** and **parametrizable** design, it was possible to simulate the users of the application using the Python script introduced in Section 5.2.4. The parameters passed to the script were adjusted to the timings of this experiment (thirty minutes per configuration), and the **number of users was set to three** to increase the workload on the system a bit.

### 5.3.5. Analysis of Results

After eight hours, the A/B/n testing pipeline had successfully switched between all sixteen available system configurations and collected aggregate metrics.

The **top three most performant configurations** are illustrated in Figure 5.6.

It is possible to notice some connections between the three configurations already. Firstly, no winning configuration selected the variant for the *Image Store* service **without compression**, clearly indicating that compressing images at the system's entrance is always the best choice to achieve better performance across these metrics.

## Top System Configurations

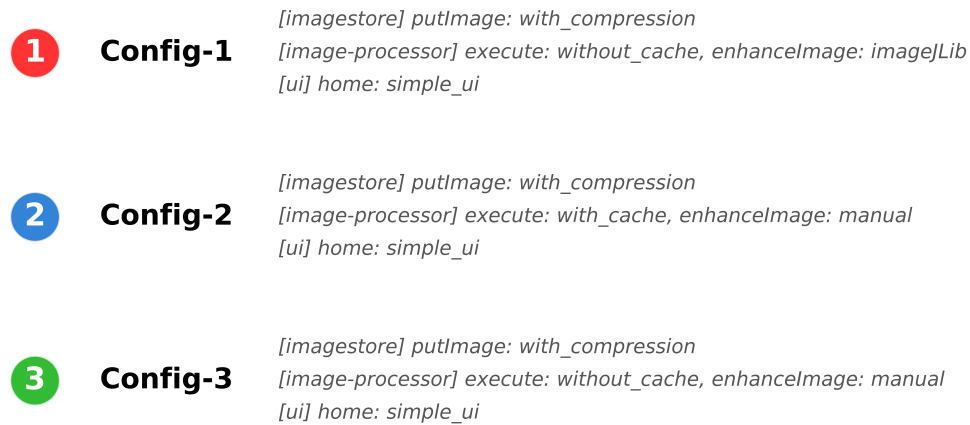


Figure 5.6: Ranking of the top three configurations with their selected behavioural variants.

Secondly, all configurations use the *simple\_ui* variant in the *UI* service, which helps in reducing the response time and CPU usage by **simplifying the graphical interface** load.

As for the *Image Processor* service, it is possible to observe that the first configuration in the ranking does not utilise the cache; indeed, the variant chosen is *without\_cache*. At first, this result seemed to be **counter-intuitive**, and we perceived it as a possible mistake in the metrics collection or aggregation strategy. However, by repeating the experiment multiple times, the same result consistently appeared: the system configurations that did not use the cache in the *Image Processor* service were always ranked first.

Investigating the code implementing the cache more closely, we gained a better understanding of the reason and motivation behind these outcomes. Indeed, the cache functionality is implemented in a thread-safe way, making use of **locks**, which greatly increases the response time of the system when multiple write operations are executed concurrently on the cache. In this code snippet, it is possible to see the properties of the class implementing the cache:

```
public class InMemoryImageCache implements ProcessedImageCache {

    private final ConcurrentHashMap<String, CacheEntry> cache;
    private final ReentrantReadWriteLock lock;    // Read-write lock
```

```

    ...
}

```

The read-write lock allows concurrent read requests, but it allows one write at a time **sequentially**. Furthermore, while a write is in progress, read requests are also temporarily blocked. As a clear consequence of this, the cache indeed reduces the response time and computational expenses by avoiding fetching the images from the *Image Store* service, but it is also true that if we consider **cache misses** and the **locking-based mechanism**, the results of the experiment sound more and more reasonable. They clearly show that the implementation of the cache may not provide all the desired benefits and should perhaps be refactored using lock-free techniques or by increasing the cache size.

Let us now move on to the **analysis of the actual metrics** extracted from the running system. Here, we report the plots produced from the JSON-formatted report compiled by the Marionettist service.

**System-level Aggregated Metrics** Figure 5.7 shows the **aggregate metrics** at the system-level for the top three most performant configurations, i.e., the metrics computed by averaging the service-level metrics within each system behavioural configuration.

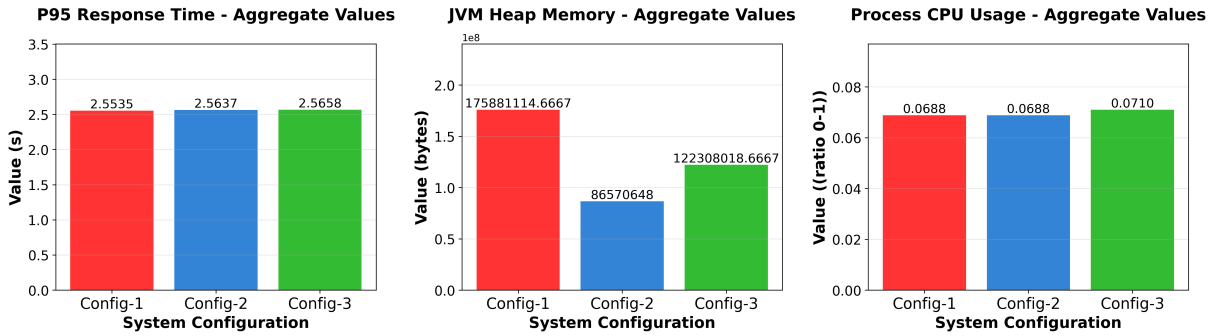


Figure 5.7: Bar plot with aggregate metrics extracted for each winning system configuration.

It is possible to notice that the top three configurations are very **close** to one another for P95 quantile response time and CPU usage. In contrast, they **differ** significantly in the measurements of JVM memory consumption. By analysing the main differences between the configurations, it is possible to identify the reasons for the significant gap in memory usage. The second configuration in the ranking is **the only one employing the cache** in the *Image Processor* service. While this feature does not contribute much to optimising the response time and CPU usage, its impact is clearly visible in the central plot representing the JVM memory usage. By employing the cache, it is possible to

**share already processed images** across multiple user requests. This allows for partially reducing the number of times the image processor needs to fetch an image from the *Image Store* service and enhance it locally, occupying memory.

The same information can be clearly observed with the line plots shown in Figure 5.8, where the differences between the values are better zoomed in.

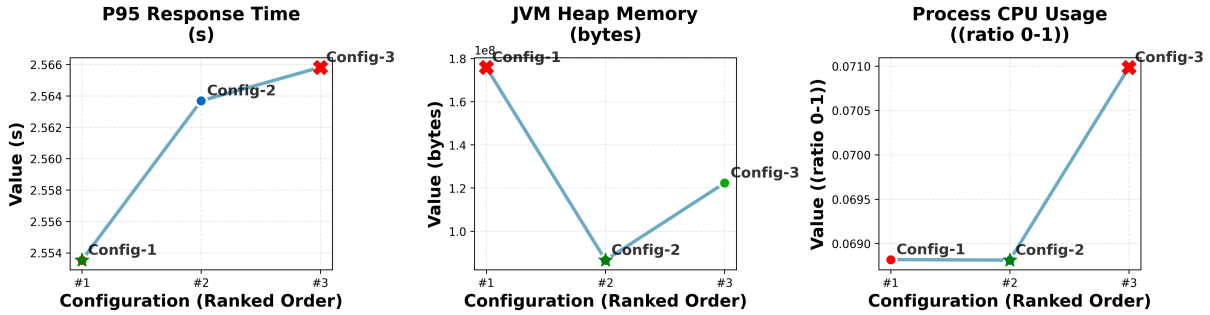
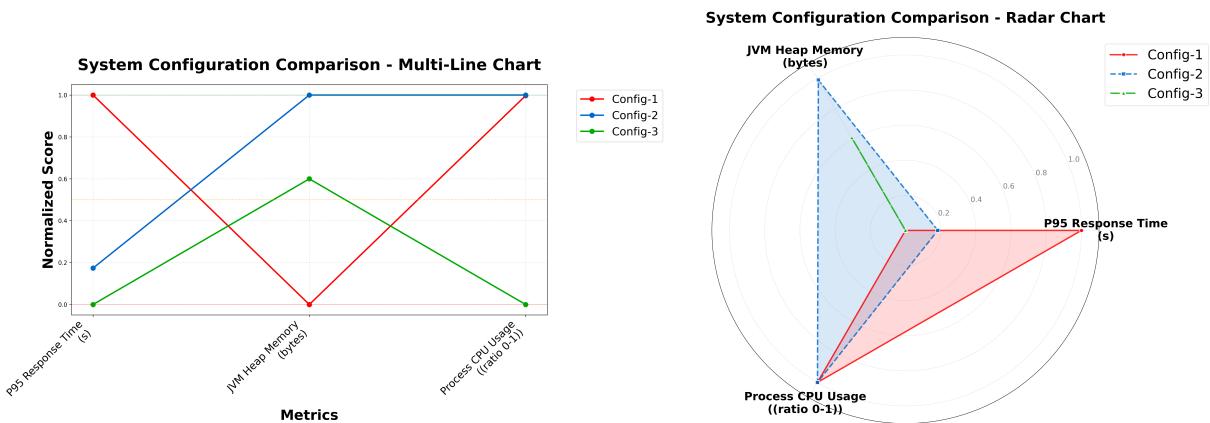


Figure 5.8: Line plot with aggregate metrics extracted for each system configuration.

### Normalised System Configurations Comparison with Line and Radar Charts

Figure 5.9 shows the system-level configurations comparison charts. These charts make it easy to compare the top three configurations across the metrics collected. Indeed, by applying min-max **normalisation** and **alignment of the optimisation direction**, these plots are capable of highlighting the relative distance between system configurations for each monitored metric.



(a) Multi-line chart to show system-level comparison of the complex experiment.

(b) Radar chart to show system-level comparison for the complex experiment.

Figure 5.9: System-level configurations comparison, shown with two different types of charts.

It is possible to notice some details from these illustrations. The first-ranked configura-



tion has the **best response time**, but it is also the **worst**-performing configuration for **memory usage**. On the other hand, the second-ranked configuration presents a **worse response time** (higher in value). Still, it is the **top-performing** one for **memory usage** and ties with the first-ranked configuration for CPU usage. This aligns well with the considerations made in the previous paragraphs on the impact of the cache in the *Image Processor* service. The third-ranked configuration is the worst in terms of response time and CPU usage, while it exhibits mediocre performance in memory usage.

**Service-level Comparison with Bar Plot** Ultimately, it is interesting to see how the aggregated metrics shown in the previous plots are obtained by breaking down the measurements at the service level. Figure 5.10 shows the metrics extracted for each service in the top-performing configurations.

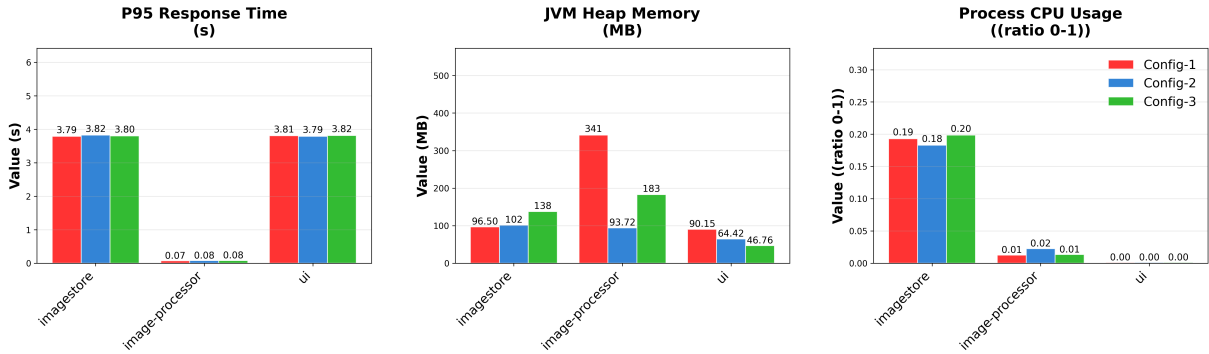


Figure 5.10: Bar plot showing the observed metrics at the service level for the complex experiment.

The analysis that was carried out in the previous paragraphs on the **impact of the cache** is consistently **reflected here** as well. The response time and CPU usage do not show great differences across system configurations. But if we focus on the JVM memory usage, we can clearly see that the bars in the plot look much more uneven. The most critical service is definitely the *Image Processor* service, which exhibits **spikes** in memory usage for configurations 1 and 3, while showing a significant **reduction** (more than halved) for configuration 2. The reason for this is that the first and third configurations do not utilise the cache, which forces the system to continuously fetch the images from the *Image Processor* service and reprocess them.

Furthermore, it is possible to observe an additional **difference** between the **first** and **third** configurations. The two configurations have different behavioural variants for the image enhancement algorithm. Configuration 1 utilises the third-party image processing library *ImageJ*, whereas Configuration 3 relies on raw pixel manipulation. From the

final results, it is clear that the third-party library has quite a **negative impact on memory consumption**, while it might help to slightly speed up the response time, since configuration 1 has a lower response time overall.

A final observation is on the CPU usage plot. It is evident that for the *Image Processor* service, the internal image cache usage **doubles the CPU usage metric**, on average. This is reasonable as the system becomes more complicated and must **maintain a central data structure** for image retrieval.

### 5.3.6. Summary of RQ1 (Multi-Variation Point)

The second and most complete experiment on a real microservice-based application showed the potential of employing the Marionette ecosystem to perform systematic A/B/n testing and compare system configurations.

From this analysis, it is possible to recognise various **advantages of this approach** that are not always guaranteed by traditional A/B testing practices. First of all, the tests work at a very **fine granularity**, enabling the system to compare different method implementations inside a single service. Usually, A/B testing works on a coarser scope, by comparing two different versions of a full-service.

Moreover, the **exhaustive investigation** of all global system configurations allows the user of the framework not only to make comparisons between alternatives, but to **discover trade-offs** and nuanced details that may remain obscure otherwise. This experiment showcases this quite well. The injection of a cache mechanism was designed as an **improvement** to the system, but it turned out to be not that beneficial after all, for the first metric in the configuration, the response time. On the other hand, by examining aggregate metrics at various levels (system and service levels), developers may discover that certain behaviours have a stronger influence on other unexpected metrics. In this case, the use of the cache resulted in a **beneficial reduction of the memory footprint** of the program.

To sum up, in some cases, the final top-performing configurations allow us to establish a **definitive decision** for a variation point. For example, the use of compression on images and a simplified graphical user interface has consistently demonstrated the ability to increase system performance, as they are **shared across all top configurations**. In other cases, the final ranking reveals trade-offs that developers should consider when adopting a solution over another, as illustrated by the cache example.

## 5.4. Evaluation of Instrumentation Overhead

While the main application of the Marionette framework introduced in this paper focuses on an A/B/n automated test, it is also possible to employ the infrastructure in other domains, thanks to its **flexibility** and **generality**. The tool can be used in production environments to manually (or, in the future, automatically) control the internal behaviour of microservices at the method level from a centralised platform. Consequently, we find it quite relevant to perform some experimental studies on the **computational overhead** (RQ2) that the instrumentation generated by the Marionette tools introduces in the system and in particular in the **execution of the methods with behavioural variants**.

### 5.4.1. Experiment Setup and Execution

The idea behind this experiment revolves around **comparing** a microservice **without behavioural variations** and its counterpart that must be **instrumented** with the Marionette tool described in Chapter 2. The *Outfit App* was used again to conduct this evaluation, specifically the instrumented version with four variation points that was used to perform the tests illustrated in Section 5.3.

#### Measurement of Baseline

The first step in the process was to obtain the **measurement of the baseline**, i.e. the execution time of some methods **before instrumenting them with the Marionette tool**.

Three of the four variation points of the *Outfit App* were selected for this purpose: the **compression strategy** in the *Image Store* service, the **cache implementation** in the *Image Processor* service and the **picture enhancement algorithm** in the *Image Processor* service again. The method with variants, designed for the *UI* service, was excluded from the experiment because it incurred no real computational expense. It was a simple selector of the type of user interface to be returned to the user, and as such, it would not have provided much insight into the computational overhead.

The three selected methods were artificially modified to **track the start time** at the very first line and **the end time** before terminating and to log the difference between the timestamps in nanoseconds on a custom file. Each method had a separate output log file to keep the results distinct.

The modified *Outfit App* was redeployed on the Kubernetes local cluster, and synthetic user interactions were generated with the Python script described in Section 5.2.4. The

experiment was terminated after the log files had already collected **at least 1.500 execution time samples** for each of the methods mentioned above.

## Measurement of Marionette-instrumented System

Once the baseline had been measured, it was necessary to evaluate the execution time for the same methods after instrumentation with the Marionette tool. Consequently, this second part of the experiment focused on the same microservices but **after the source code modifications** applied by the Marionette tool.

As described in Section 2.2, when injecting behavioural variants for a specific method, the Marionette tool turns the original method definition into a dispatcher, i.e. a single switch statement that dynamically selects the alternative implementation to be executed. To make the measurements **logically comparable** with the baselines, only the **original method conditional path** was timed among the options present in the switch statement. This means that, even if, after source code manipulation, the method body has been changed to dispatch execution to multiple possible implementations, the purpose of the experiment remains to assess the overhead introduced by the instrumentation in the time to execute **the same original logic**. The different behavioural variants for the same method can vary greatly in execution time, but this depends on the variability of the code itself, which is not part of this evaluation. For example, this is the time measurement infrastructure manually injected into the `enhanceImage` method, which is used in the *Image Processor* service for picture enhancement.

```
public byte[] enhanceImage(byte[] originalImageData) {

    // Track start time
    long startTime = ExecutionTimer.start();
    switch(BehaviourRegistry.getBehaviourId("ImageEnhanceProcessor", "enhanceImage"))
    ↪ {

        case "imageJLib":
            return enhanceImage_imageJLib(originalImageData);

        default:
        case "manual":
            byte[] enhanced = enhanceImage_manual(originalImageData);
            // Log execution time for default path -> original implementation
            ExecutionTimer.end(startTime, "enhanceImage-marionette.log");
            return enhanced;
    }
}
```

It is possible to notice that the variant implementation of the method, identified as *image-JLib*, is not considered in the measurement, as the comparison must be consistent with the baseline. Therefore, it must **assess the overhead introduced on the original implementation**.

The marionette-enabled *Outfit App* with the modifications for time measurement was then deployed on the cluster, and synthetic user interactions were generated with the Python script described in Section 5.2.4. It is also important to notice that to make the experiment as realistic as possible, the microservices were stressed during the overhead measurement by **periodically changing their behavioural configuration**. Another Python script was running in the background, and it was changing the behaviour configuration for the three selected methods. This guarantees realistic measurements as the final purpose of the tool is to gain control of the individual microservices' behaviour and switch between variants freely. Again, the experiment stopped once it obtained **at least 1.500 samples for each method** under analysis.

#### 5.4.2. Results Analysis

At the end of the experiment cycle, two log files were produced for each method: the first file reported execution time samples for the method without Marionette instrumentation, the second file for the method transformed by the Marionette tool. It was therefore possible to **produce plots** to analyse the results of this test and draw conclusions about the **overhead measurement**.

Figure 5.11 presents three plots that illustrate different perspectives on the results for the **enhanceImage** method, which is the technique employed within the *Image Processor* service to perform picture enhancement.

At the top left, the **box plot** shows the distribution of the obtained samples. The original implementation (on the left) is faster and exhibits less variability, whereas the samples collected for the marionette-instrumented version are more dispersed and have a higher median.

In the **plot at the top right**, it is possible to see the aggregated results of the experiment. The first bar on the left represents the **mean time computed on the baseline**, averaging all the execution times collected for the original version of the method. The yellow bar on the right shows the **absolute time overhead in milliseconds** introduced by the marionette instrumentation. This measure was obtained by first averaging all the

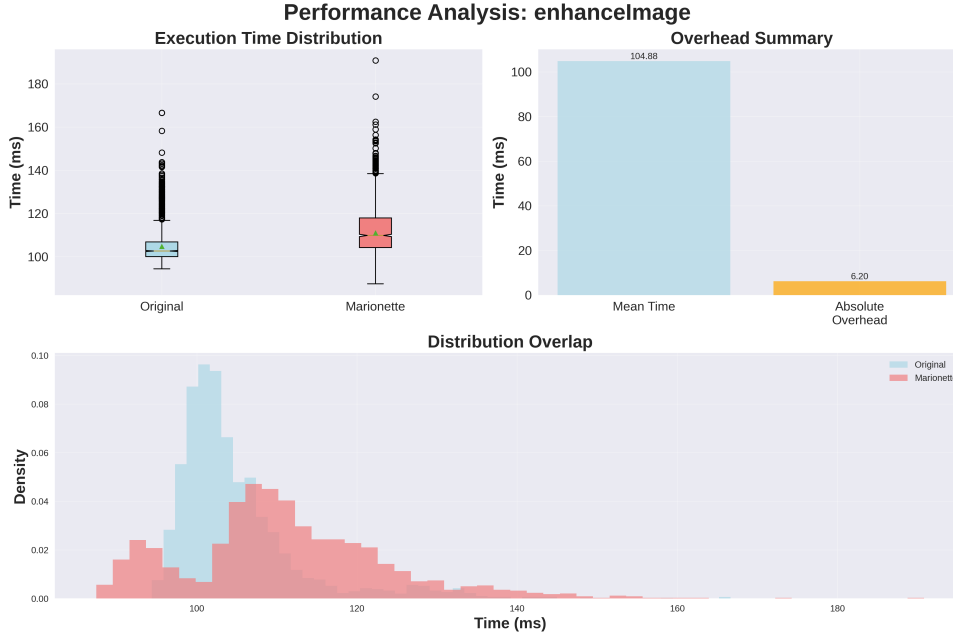


Figure 5.11: Plots representing the findings on the overhead measurements for the `enhanceImage` method used in the *Image Processor* service to select the picture enhancement implementation.

samples of execution time for the marionette version of the method and then computing the difference with the baseline average.

The **plot at the bottom** of the figure shows the **distribution overlap** between the two datasets of samples. The x-axis represents possible execution time buckets in milliseconds, and the y-axis shows the density, so a relative measure of how many samples were found belonging to that bucket. As already noticed in the box plot, the blue distribution (original method) is **less variant** and usually **faster**. In contrast, the red distribution (marionette version) shows more **dispersion** of samples and **slower** execution time on average.

Figures 5.12 and 5.13 show the same results for the other two methods selected for the experiment. Figure 5.12 corresponds to the results for the method in the *Image Processor* service to select the cache strategy, while Figure 5.13 is related to the method in the *Image Store* service to decide on the image compression strategy.

By observing the experimental findings and **aggregating results** among the different methods selected for the test, it is possible to see that the overhead introduced by the Marionette instrumentation is quite **stable** across measurements. In particular, if we calculate the percentage relative overhead observed for the three methods, we find that the overhead is stable **around 4-6% of the original method execution time**.

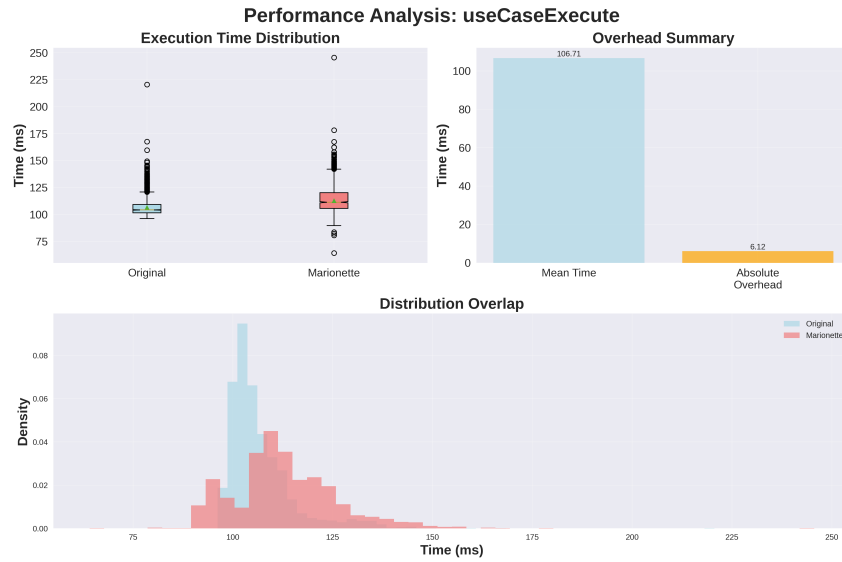


Figure 5.12: Plots representing the findings on the overhead measurements for the method used in the *Image Processor* service to process an image retrieval request, either with the support of the in-memory cache, or without.

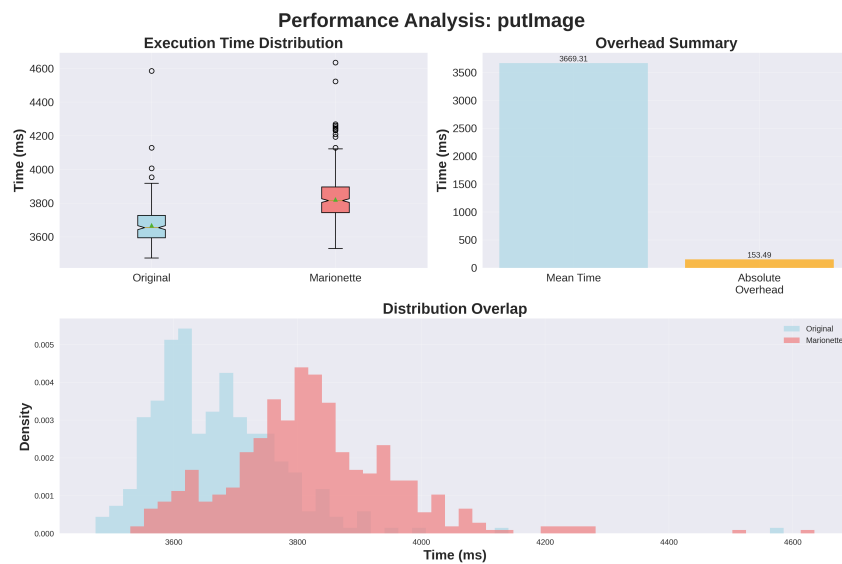


Figure 5.13: Plots representing the findings on the overhead measurements for the method used in the *Image Store* service to store a user-uploaded image locally, either performing compression on it or without any compression applied.

Table 5.4 shows the relative overhead in percentage for each method.

Table 5.4: Marionette Framework Performance Relative Overhead

| Strategy                   | Relative Overhead (%) |
|----------------------------|-----------------------|
| Compression Strategy       | 4.18                  |
| Image Enhancement Strategy | 5.91                  |
| Cache Strategy             | 5.73                  |

### 5.4.3. Summary of RQ2 (Multi-Variation Point)

The experimental evaluation of the Marionette framework’s computational overhead reveals that the instrumentation introduces a **consistent performance penalty** across different method implementations. The measured relative overhead **ranges from 4.18% to 5.91%**, with an average of approximately 5.27% across the three evaluated variation points.

These findings must be understood within the context of the Java ecosystem’s performance characteristics and the **inherent costs of runtime flexibility**. As Blackburn et al. [4] demonstrated in their analysis of real-world Java applications, enterprise systems commonly exhibit performance variability in the 3-10% range when introducing abstraction layers, with the benefits of modularity and configurability typically justifying such overhead in production environments.

In conclusion, the consistent overhead across different operational contexts, combined with the framework’s capabilities for runtime behaviour modification, makes it a **viable solution** for **production environments** where the benefits of dynamic configurability outweigh the modest performance cost.



## 5.5. Analysis of Development Efficiency Gains

The Marionette framework significantly **reduces development overhead** by providing pre-built infrastructure components that would otherwise require **substantial manual implementation**. It also helps **standardising** the process of exposing control of behavioural variants through a unified and generic strategy, as described in Section 2.2. This section quantifies the development efficiency gains achieved through framework adoption, answering RQ3.

### 5.5.1. Framework Architecture Benefits

The Marionette ecosystem consists of two main components that work together to eliminate the need for custom control plane development:

- **Marionette Tool:** Provides the runtime behaviour injection and control capabilities within each microservice.
- **Marionettist Service:** Offers centralised behaviour configuration management and service discovery across the distributed system.

When developers adopt Marionette, they only need to implement their **core business logic** and behavioural variants. The framework automatically handles **service discovery, configuration management**, and **runtime behaviour switching**, relieving the developer from this workload.

### 5.5.2. Code Complexity Analysis

Table 5.5 presents the lines of code analysis for the complete Marionette ecosystem, demonstrating the infrastructure complexity that developers receive without additional implementation effort.

Table 5.5: Lines of Code Analysis for Marionette Components

| Component                             | Lines of Code |
|---------------------------------------|---------------|
| Frontend Control Panel (Marionettist) | 1,163         |
| Marionettist Backend Java Code        | 5,841         |
| Marionette Tool Code                  | 1,816         |
| <b>Complete Ecosystem</b>             | <b>8,820</b>  |

The same information is represented graphically in Figure 5.14.

The analysis reveals that the complete Marionette ecosystem comprises **8,820 lines** of

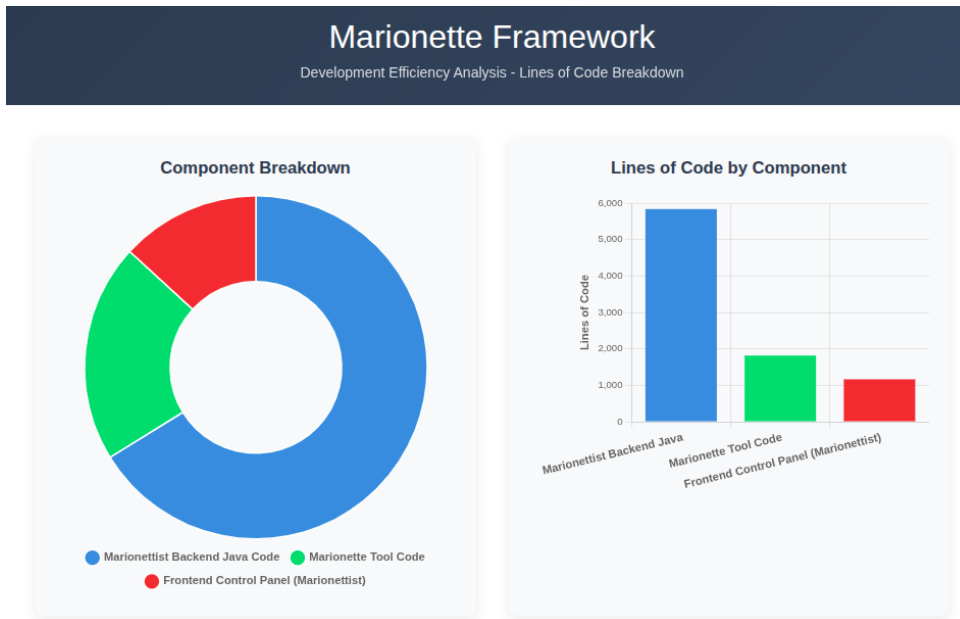


Figure 5.14: Visual representation of the number of source code lines in the implementation of the Marionette framework.

production code. This represents substantial infrastructure complexity, including:

- Marionette **service discovery** and registration mechanisms.
- Behavioural **configuration extraction**.
- Web-based **management interface**.
- **Kubernetes** integration.
- **Metrics collection** and monitoring integration.

### 5.5.3. Developer Productivity Impact

Without the Marionette framework, developers would need to implement equivalent functionality for each distributed system project manually. The framework eliminates this recurring development overhead by providing:

1. **Zero Infrastructure Code:** Developers write only business logic and behavioural variants.
2. **Automatic Service Integration:** Framework handles service registration and discovery.
3. **Built-in Management Interface:** Pre-built web console for configuration man-

agement.

4. **Production-Ready Monitoring:** Integrated metrics and observability features.

#### 5.5.4. Summary for RQ3 (Multi-Variation Point)

The Marionette framework delivers substantial **developer workload reduction** by providing **8,820 lines** of pre-built infrastructure code, eliminating the need for custom control plane development. Developers can focus exclusively on **business logic** and behavioural variants while the framework automatically handles service discovery, configuration management, and runtime behaviour control. This represents a significant **productivity improvement** for distributed systems requiring behavioural flexibility and centralised management.



## 6 | Conclusions and Future Work

This thesis presented **Marionette**, a framework that enables **runtime behavioural configuration** and **automated A/B/n testing** in microservice architectures. The work addressed the critical gap between structural adaptability and behavioural adaptability in modern distributed systems, providing developers with tools to define, manage, and test multiple behavioural variants within their microservices.

### 6.1. Main Conclusions

The experimental evaluation demonstrated several key findings that validate the practical viability and effectiveness of the Marionette framework.

**Framework Effectiveness** The Marionette framework empowers microservice-based applications with **fine-grained behavioural control** that does not require code redeployment. The experiments with the *Outfit App* demonstrated that the framework can inject behavioural variants at the method level across multiple microservices. Additionally, **runtime behaviour switching** operates correctly in distributed environments. The Marionettist service provides effective and **centralised control** of behaviour changes, while automated service discovery reduces operational overhead by eliminating manual configuration steps.

**A/B/n Testing Capabilities** The automated A/B/n testing pipeline revealed significant advantages over traditional testing approaches. Through **multi-dimensional experimentation**, the framework can test behavioural variants across multiple microservices simultaneously, exploring all possible combinations to identify optimal system-level configurations. Furthermore, **trade-off discovery** became evident as testing revealed unexpected trade-offs, such as the cache implementation that improved memory usage but increased CPU consumption and response time due to locking mechanisms. The **comprehensive analysis** approach, with exhaustive testing of all configurations, provided insights that would remain hidden in traditional binary A/B testing. Finally, **system-**

**wide impact assessment** was achieved as changes to individual methods were evaluated across the entire service chain, revealing propagation effects that isolated testing would miss.

**Performance Characteristics** The overhead analysis provided a clear quantification of the framework’s performance impact. The instrumentation introduces a **consistent 4-6% performance overhead** across different method implementations. Moreover, this overhead remains stable regardless of the complexity of the behavioural variants. The performance cost is comparable to other enterprise Java abstraction layers and is acceptable for most production scenarios.

**Development Productivity** The framework significantly reduces development complexity. Developers write only business logic and behavioural variants, avoiding the whole implementation of the infrastructure that the framework provides out of the box. The tool **eliminates** manual implementation of conditional logic, configuration management, and external API endpoints, which are error-prone and time-consuming practices. The standardised approach ensures consistency and reduces errors across different behavioural implementations.

## 6.2. Future Work

The Marionette framework establishes a foundation for several promising research and development directions that could extend its capabilities and applicability.

**Intelligent Behavioural Configuration** One of the most significant opportunities lies in developing automated system configuration selection mechanisms. Rather than requiring manual behavioural choices, future work could explore **rule-based controllers**, which are external controller services that apply predefined rules to select optimal behavioural configurations based on real-time system conditions automatically. Additionally, **machine learning optimisation** could be investigated through integration of ML models that learn optimal configurations from historical data and automatically adapt system behaviour to changing workloads. Furthermore, **multi-objective optimisation** represents another avenue, involving automated systems that balance competing objectives (performance, energy consumption, cost) when selecting behavioural variants.

**Energy Consumption Optimization** The framework provides an ideal foundation for sustainability-focused applications. Developers could implement behavioural variants

with different energy consumption profiles for critical methods. Building on this foundation, an external controller could dynamically select energy-efficient configurations while maintaining performance requirements. Moreover, integration with power monitoring systems could enable **real-time energy-aware behavioural adaptation**.

**Platform and Language Extensions** The current implementation focuses on Java Spring microservices orchestrated by a Kubernetes cluster equipped with a Prometheus server, but the core concepts are platform-agnostic. **Multi-language support** could be achieved by extending the Marionette tool to support other programming languages such as Python, Go, or Node.js. Additionally, **alternative orchestration platforms** could be targeted by adapting the Marionettist service to work with Docker Swarm, Apache Mesos, or serverless platforms. Furthermore, **cloud-native integration** could provide enhanced integration with cloud-specific services and managed platforms.

**Advanced Testing and Analysis** The A/B/n testing pipeline could be enhanced with additional capabilities. **Statistical significance testing** would involve integration of statistical methods to determine when enough data has been collected for reliable conclusions. **Adaptive testing duration** could provide dynamic adjustment of testing periods based on traffic patterns and metric convergence. Additionally, **advanced traffic management** could be implemented through integration with service mesh technologies for more sophisticated traffic routing and load balancing during tests.

**Security and Compliance** Future versions should address enterprise security and compliance requirements. Authentication and authorisation mechanisms for behavioural configuration changes would be essential for **enterprise adoption**. Additionally, audit logging and compliance reporting for regulatory environments would be necessary. Furthermore, integration with enterprise security frameworks and identity management systems would ensure seamless adoption in corporate environments.

### 6.3. Final Remarks

The Marionette framework demonstrates that behavioural adaptability can be achieved in microservice architectures without sacrificing **code maintainability** or **system performance**. The experimental results validate the approach's effectiveness, while also revealing opportunities for further improvement.

The framework's modular architecture, with its clear separation between the code transformation tool and the centralised control plane, provides a solid foundation for the future

developments outlined above. As microservice architectures continue to evolve and organisations seek greater **operational flexibility**, tools like Marionette become increasingly valuable for managing the complexity of distributed behavioural configuration.

The combination of automated infrastructure generation, centralised management capabilities, and comprehensive testing tools positions the Marionette framework as a practical solution for organisations seeking to implement dynamic behavioural adaptation in their microservice-based applications.



# Bibliography

- [1] G. H. Alf  rez and V. Pelechano. Achieving autonomic web service compositions with models at runtime. *Computers and Electrical Engineering*, 63:332–352, 2017. doi: 10.1016/j.compeleceng.2017.08.004.
- [2] A. Angelis and G. Kousiouris. A survey on the landscape of self-adaptive cloud design and operations patterns: Goals, strategies, tooling, evaluation and dataset perspectives. *arXiv preprint arXiv:2503.06705*, 2025. URL <https://arxiv.org/abs/2503.06705>.
- [3] P. Arcaini, E. Riccobene, and P. Scandurra. Modeling and analyzing mape-k feedback loops for self-adaptation. In *Proceedings of the 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, pages 13–23. IEEE, 2015.
- [4] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, et al. The dacapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*, pages 169–190. ACM, 2006. doi: 10.1145/1167473.1167488.
- [5] B. H. C. Cheng, R. de Lemos, H. Giese, and et al. *Software Engineering for Self-Adaptive Systems: A Research Roadmap*, pages 1–26. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. doi: 10.1007/978-3-642-02161-9\_1. URL [https://doi.org/10.1007/978-3-642-02161-9\\_1](https://doi.org/10.1007/978-3-642-02161-9_1).
- [6] S. Dell’Oro. An adaptive deployment approach for sustainable microservice-based applications. Master’s thesis, Politecnico di Milano, 2021.
- [7] L. Florio and E. D. Nitto. Gru: An approach to introduce decentralized autonomic behavior in microservices architectures. In *2016 IEEE International Conference on Autonomic Computing (ICAC)*, pages 357–362. IEEE, 2016. doi: 10.1109/ICAC.2016.25.
- [8] M. Fowler. Feature flag. *Martin Fowler’s Bliki*, 2010. URL <https://martinfowler>.

- `com/articles/feature-toggles.html`. Foundational blog post on feature flags, updated 2016 and 2023.
- [9] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo. Adaptive resource efficient microservice deployment in cloud-edge continuum. *IEEE Transactions on Parallel and Distributed Systems*, 33(8):1825–1840, 2022.
  - [10] J. Humble and D. Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Signature Series. Addison-Wesley Professional, 2010. ISBN 978-0321601919. Winner of the 2011 Jolt Excellence Award. Foundational work on continuous delivery practices.
  - [11] N. H. Kacem, A. H. Kacem, and K. Drira. A formal approach to enforcing consistency in self-adaptive systems. In M. A. Babar and I. Gorton, editors, *Software Architecture*, volume 6285 of *Lecture Notes in Computer Science*, pages 279–294. Springer Berlin Heidelberg, 2010. doi: 10.1007/978-3-642-15114-9\_21. URL [https://doi.org/10.1007/978-3-642-15114-9\\_21](https://doi.org/10.1007/978-3-642-15114-9_21).
  - [12] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003. doi: 10.1109/MC.2003.1160055.
  - [13] R. Kohavi, R. Longbotham, D. Sommerfield, and R. M. Henne. Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1):140–181, 2009. doi: 10.1007/s10618-008-0114-1. Foundational survey establishing best practices for web-based controlled experiments.
  - [14] R. Kohavi, D. Tang, and Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, Cambridge, UK, 2020. ISBN 978-1108724265. Comprehensive guide to experimental design and statistical analysis for online controlled experiments.
  - [15] N. Larsen, J. Stallrich, S. Sengupta, and A. Deng. Statistical challenges in online controlled experiments: A review of a/b testing methodology. *The American Statistician*, 2024. doi: 10.1080/00031305.2023.2257237. URL <https://doi.org/10.1080/00031305.2023.2257237>.
  - [16] R. C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN 9780134494272. URL <https://www.amazon.com/Clean-Architecture-Craftsmans-Software-Structure/dp/013449427X>. Also available on <https://www.oreilly.com/library/view/clean-architecture-a/9780134494272/>.

- [17] Netflix, Inc. Hystrix: Latency and fault tolerance library. GitHub Repository, 2012. URL <https://github.com/Netflix/Hystrix>. Open source library for latency and fault tolerance in distributed systems.
- [18] M. T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Programmers. Pragmatic Bookshelf, 1st edition, March 2007. ISBN 978-0978739218. Seminal book introducing the Circuit Breaker pattern and other stability patterns.
- [19] J. Oh, C. Raibulet, and J. Leest. Analysis of mape-k loop in self-adaptive systems for cloud, iot and cps. *Self-adaptive Systems: Design and Field Studies*, pages 130–141, 2023.
- [20] F. Quin and D. Weyns. Seabyte: A self-adaptive micro-service system artifact for automating a/b testing. In *Proceedings of The 17th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2022)*, pages 77–83, New York, NY, USA, 2022. ACM. ISBN 978-1-4503-9305-8. doi: 10.1145/3524844.3528081.
- [21] M. T. Rahman, L.-P. Querel, P. C. Rigby, and B. Adams. Feature toggles: Practitioner practices and a case study. In *Proceedings of the 13th International Conference on Mining Software Repositories*, pages 201–211, 2016.
- [22] F. Rossi, V. Cardellini, and F. L. Presti. Hierarchical scaling of microservices in kubernetes. In *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pages 28–37. IEEE, 2020. doi: 10.1109/ACSOS49614.2020.00023.
- [23] B. Schmaus. Deploying the netflix api: how our process has evolved. Technical report, Netflix Technology Blog, August 2013. URL <https://netflixtechblog.com/deploying-the-netflix-api-79b6176cc3f0>.
- [24] E. Silva, L. F. Pires, and M. van Sinderen. Supporting dynamic service composition at runtime based on end-user requirements. In *Proceedings of the Workshop on Architectures, Concepts and Technologies for Service Oriented Computing*, Portugal, 2010.
- [25] D. Siroker and P. Koomen. *A/B Testing: The Most Powerful Way to Turn Clicks Into Customers*. Wiley, Hoboken, NJ, 2013. ISBN 978-1118536094. Practical guide to implementing A/B testing methodologies in business contexts.
- [26] T. Wong, M. Wagner, and C. Treude. Self-adaptive systems: A systematic literature review across categories and domains. *Information and Software Technology*, 148:

- 106934, 2022. doi: 10.1016/j.infsof.2022.106934. URL <https://doi.org/10.1016/j.infsof.2022.106934>.
- [27] X. Zang, W. Chen, J. Zou, S. Zhou, L. Huang, and R. Liang. A fault diagnosis method for microservices based on multi-factor self-adaptive heartbeat detection algorithm. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, pages 165–171. IEEE, 2018. doi: 10.1109/QRS-C.2018.00043.

## List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                             |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Graphical funnel representing the steps to instrument a microservice to become a Marionette node. . . . .                                                                                                                                                                                                                                                                   | 18 |
| 1.2 | Simple microservice-based application with two services . . . . .                                                                                                                                                                                                                                                                                                           | 23 |
| 1.3 | Simple microservice-based application with two services and the Marionettist controller. . . . .                                                                                                                                                                                                                                                                            | 23 |
| 2.1 | Marionette pattern with its main components, the marionette classes, the behaviour registry, and the behaviour controller . . . . .                                                                                                                                                                                                                                         | 26 |
| 2.2 | A visual representation of the source code transformation process. On the left, the code of the original project in which multiple class files hold the variant implementations. On the right, the merging process was completed on the original class file, incorporating all behavioural variants. . . . .                                                                | 33 |
| 2.3 | Graphical representation of the separation between pre-existing code and injected infrastructure. The green circle represents the most relevant subset of components injected in a marionette-enabled microservice. The Behaviour Registry serves as the central mapping for behaviours, while the Behaviour Controller provides an interface for external control. . . . . | 34 |
| 2.4 | Visual representation of the state of the behaviour registry in the example of the image processor service. . . . .                                                                                                                                                                                                                                                         | 38 |
| 2.5 | Component diagram showing the interactions between the behaviour registry, the behaviour controller, the external manager (Marionettist) and the marionette classes. . . . .                                                                                                                                                                                                | 40 |
| 2.6 | Sequence diagram illustrating the interaction between the external behaviour manager (Marionettist) and the instrumented marionette service when reading the behavioural configuration. . . . .                                                                                                                                                                             | 41 |
| 2.7 | Sequence diagram showing the interactions between the external behaviour manager (Marionettist) and the marionette service components when changing the behavioural configuration. . . . .                                                                                                                                                                                  | 42 |
| 3.1 | High-level view of the core components of the Marionettist service. . . . .                                                                                                                                                                                                                                                                                                 | 47 |

|      |                                                                                                                                                                                          |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.2  | A graphical representation of the main components of the Marionettist service, divided into the typical layers of a clean architectural style. . . . .                                   | 49 |
| 3.3  | Component diagram detailing the main modules that make up the Marionettist service. . . . .                                                                                              | 51 |
| 3.4  | A deployment view of the full architecture. The Marionettist service integrates seamlessly with a Kubernetes environment and a Prometheus server for collecting runtime metrics. . . . . | 52 |
| 3.5  | The home page of the Marionettist service graphical interface showing three discovered marionette-enabled services. . . . .                                                              | 55 |
| 3.6  | An example of a service detailed view of the Marionettist graphical interface showing two classes that expose variant behaviours. . . . .                                                | 55 |
| 3.7  | An example of how the graphical interface of the Marionettist service shows the user-defined metrics collected at runtime for a marionette-enabled service. . . . .                      | 56 |
| 4.1  | Example architecture of the mathematical equation solver web application. . . . .                                                                                                        | 58 |
| 4.2  | Graphical representation of the steps of the testing pipeline. . . . .                                                                                                                   | 60 |
| 4.3  | Visual aggregation of the metrics, from the service-level results to the system-level results. . . . .                                                                                   | 61 |
| 4.4  | The graphical user dialog used to set up the total amount of time for the A/B/n test. . . . .                                                                                            | 64 |
| 4.5  | High-level view of the components interacting when initiating an A/B/n testing phase. . . . .                                                                                            | 65 |
| 4.6  | Example of a radar chart being produced at the end of A/B/n testing. . . . .                                                                                                             | 66 |
| 4.7  | Example of a multi-line chart being produced at the end of A/B/n testing. . . . .                                                                                                        | 66 |
| 4.8  | Example of a single metric line chart being produced at the end of A/B/n testing. . . . .                                                                                                | 67 |
| 4.9  | Example of a single metric bar chart being produced at the end of A/B/n testing. . . . .                                                                                                 | 68 |
| 4.10 | Example of a service-level bar plot being produced at the end of A/B/n testing. . . . .                                                                                                  | 68 |
| 5.1  | The home page of the <i>Outfit App</i> showing four outfit examples. . . . .                                                                                                             | 70 |
| 5.2  | The component diagram of the <i>Outfit App</i> showing the interactions between the three microservices . . . . .                                                                        | 70 |
| 5.3  | System-level configurations comparison, shown with two different types of charts. . . . .                                                                                                | 74 |
| 5.4  | Service-level comparison of the behavioural configurations for the simple experiment . . . . .                                                                                           | 75 |

|      |                                                                                                                                                                                                                                        |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.5  | Simplified graphical interface of the <i>Outfit App</i> . . . . .                                                                                                                                                                      | 78 |
| 5.6  | Ranking of the top three configurations with their selected behavioural variants. . . . .                                                                                                                                              | 80 |
| 5.7  | Bar plot with aggregate metrics extracted for each winning system configuration. . . . .                                                                                                                                               | 81 |
| 5.8  | Line plot with aggregate metrics extracted for each system configuration. .                                                                                                                                                            | 82 |
| 5.9  | System-level configurations comparison, shown with two different types of charts. . . . .                                                                                                                                              | 82 |
| 5.10 | Bar plot showing the observed metrics at the service level for the complex experiment. . . . .                                                                                                                                         | 83 |
| 5.11 | Plots representing the findings on the overhead measurements for the <b>enhanceImage</b> method used in the <i>Image Processor</i> service to select the picture enhancement implementation. . . . .                                   | 88 |
| 5.12 | Plots representing the findings on the overhead measurements for the method used in the <i>Image Processor</i> service to process an image retrieval request, either with the support of the in-memory cache, or without. . . . .      | 89 |
| 5.13 | Plots representing the findings on the overhead measurements for the method used in the <i>Image Store</i> service to store a user-uploaded image locally, either performing compression on it or without any compression applied. . . | 89 |
| 5.14 | Visual representation of the number of source code lines in the implementation of the Marionette framework. . . . .                                                                                                                    | 92 |





## List of Tables

|     |                                                                          |    |
|-----|--------------------------------------------------------------------------|----|
| 2.1 | Core Marionette Framework API Endpoints on Instrumented Microservices    | 39 |
| 3.1 | Requirements for the Marionettist Service . . . . .                      | 46 |
| 5.1 | Metrics configuration for performance monitoring. . . . .                | 71 |
| 5.2 | Summary of behavioural variants in the Outfit App microservices. . . . . | 78 |
| 5.3 | Metrics configuration for performance monitoring . . . . .               | 79 |
| 5.4 | Marionette Framework Performance Relative Overhead . . . . .             | 90 |
| 5.5 | Lines of Code Analysis for Marionette Components . . . . .               | 91 |



# List of Symbols

| Symbol      | Description                                                       | Unit  |
|-------------|-------------------------------------------------------------------|-------|
| $RT_{95}$   | 95th percentile response time (averaged across service instances) | s     |
| $M_{heap}$  | JVM heap memory usage across service instances                    | bytes |
| $CPU_{avg}$ | Average process CPU usage ratio across service instances          | (0–1) |



## Acknowledgements

I want to express my sincere gratitude to all those who supported and contributed to this thesis work. This project was developed through a collaboration between two universities: initially at the Polytechnic University of Valencia (UPV) during a study exchange program, and later continued and finalised at the Polytechnic University of Milan.

Therefore, I would like to thank my supervisors, Professor Matteo Camilli at the Polytechnic University of Milan, Professor Silvia Abrahão, and Professor Emilio Insfran at the Polytechnic University of Valencia, for their guidance, continuous support, and insightful feedback throughout this research journey. Their expertise and encouragement were of incredible importance in shaping this work and pushing me to achieve the results illustrated in this document.

A particular note of merit also goes to Daniel Gaspar Figueiredo and Luisa Fernanda Gomez Muñoz, two PhD students at the Polytechnic University of Valencia, who guided me at the very beginning of the process to shape the project idea, which was also based on their current research focus.

I would also like to express my gratitude to my family for their consistent support, patience, and understanding during the demanding periods of this work. Their encouragement provided the foundation that made this achievement possible.

I want to acknowledge my colleagues and fellow students at Politecnico di Milano for creating a stimulating academic environment and for the numerous productive discussions that helped refine my ideas and approaches.

I am also thankful to the open-source community and the developers of the tools and frameworks that made this research technically feasible, particularly the developers of the JavaParser tool for source code modification, as well as Kubernetes and Prometheus, for the infrastructure support required by this project.

This thesis would not have been possible without the collective support of all these individuals and communities.

